Universidad Mariano Gálvez de Guatemala

Sede El Naranjo

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Ing. Nancy Guadalupe Portillo Pinto



“Proyecto ACS”

Wendy Celeste Nathaly Estrada González

9490-22-3630

Aseguramiento de Calidad de Software

Guatemala 23 de Octubre del 2026

Índice

Introducción..... **5**

Carta de Presentación..... **6**

1. Antecedentes del problema..... **7**

1.1. Contexto Histórico del Transporte Público en el Área Metropolitana..... 7
1.2. La Crisis de Inseguridad y el Colapso del Modelo Tradicional..... 7
1.3. El Surgimiento del Sistema Transmetro como Medida Alternativa..... 7
1.4. El Desafío de la Expansión Metropolitana y la Descentralización..... 8
1.5. Brecha Tecnológica y la Necesidad de Aseguramiento de la Calidad (Software)..... 8

2. Definición de la situación actual..... **10**

2.1. Contexto General y Despliegue Operativo..... 10
2.2. Arquitectura de Rutas, Estaciones y Jurisdicciones..... 10
2.3. Control de Accesos y Seguridad Física en Estaciones..... 10
2.4. Gestión de los Buses e Infraestructura de Parqueo..... 11
2.5. Dinámica de Aforos, Tiempos de Espera y Reglas de Negocio en Tiempo Real..... 11
2.6. Estado de la Infraestructura Tecnológica y Telecomunicaciones..... 12
2.7. Registro y Control del Capital Humano Central..... 12

3. Deficiencias de los controles actuales..... **13**

3.1. Vulnerabilidades en el Control de Buses y Asignación..... 13
3.2. Deficiencias en el Monitoreo de Aforo y Tiempos..... 13
3.3. Riesgos en la Seguridad de Datos y Personal..... 13

4. Planificación..... **14**

4.1. Metodología de trabajo..... 14
4.2. Calendario de entregas..... 14
4.3. Diagrama de Gantt..... 15
4.4. Sprints del proyecto..... 15
4.5. Seguimiento del cronograma..... 16

5. Análisis de Requerimientos..... **17**

Requerimientos funcionales..... 17
RF-01. Gestión de Municipalidades..... 17
RF-02. Gestión de Líneas y Recorridos..... 17
RF-03. Gestión de Estaciones..... 18
RF-04. Gestión de Accesos y Molinetes..... 18
RF-05. Gestión Integral de la Flota..... 19
RF-06. Gestión de Parqueos..... 19
RF-07. Gestión del Personal Operativo..... 19
RF-08. Gestión de Asignación de Personal..... 20
RF-09. Monitoreo Operativo de la Estación..... 20
RF-10. Gestión de Pasajeros y Ocupación..... 21
RF-11. Gestión de Buses de Refuerzo y Límite de Flota..... 21
RF-12. Gestión de Tiempos de Espera..... 22
RF-13. Gestión de Viajes y Seguimiento de Recorridos..... 22
RF-14. Métodos de Pago y Transacciones..... 22
RF-15. Gestión de Alertas Operativas..... 23
RF-16. Gestión de Usuarios, Roles y Permisos..... 23
RF-17. Gestión de Auditoría..... 24
RF-18. Comunicación entre Estaciones y Servicios Centrales..... 24
RF-19. Gestión de Reportes Operativos..... 25

RF-20. Mapa de Rutas y Monitoreo en Vivo....25
RF-21. Tarjeta Ciudadana....25
RF-22. Horarios de Servicio....26
RF-23. Simulación de la Operación....26
RF-24. Fallas y Mantenimiento de Buses....26
Matriz de trazabilidad de requerimientos funcionales....27
Requerimientos No Funcionales....28
RNF-01. Seguridad....29
RNF-02. Integridad de información....29
RNF-03. Disponibilidad....30
RNF-04. Rendimiento....30
RNF-05. Escalabilidad....31
RNF-06. Usabilidad....31
RNF-07. Auditoría....32
RNF-08. Respaldo....32
RNF-09. Mantenibilidad....33
RNF-10. Compatibilidad....34
RNF-11. Conectividad....34
RNF-12. Protección de Datos Personales....35
RNF-13. Portabilidad e Instalación....35
Relación con el modelo de calidad ISO/IEC 25010....36
6. Estudios de Factibilidad....37
6.1. Factibilidad técnica....37
6.2. Factibilidad operativa....38
6.3. Factibilidad económica....38
6.4. Factibilidad legal....40
6.5. Factibilidad ambiental....41
6.6. Resumen del estudio de factibilidad....41
7. Propuesta Monetaria....42
7.1. Revisión de la estimación original....42
7.2. Clasificación corregida de las funciones....43
7.3. Archivos lógicos....44
7.4. Cálculo de puntos de función....45
7.5. Costos del proceso de integración....46
7.6. Costos de operación en la nube (anual)....46
7.7. Resumen de la propuesta....46
7.8. Inversión en equipo de estaciones (a cargo de la municipalidad)....47
7.9. Forma de pago....47
8. Modelo Entidad Relación....48
8.1. Cambios respecto de la versión anterior....49
8.2. Relaciones y reglas del enunciado....49
9. Script de la estructura de base de datos....51
9.1. Script de creación de estructuras (DDL)....51
9.2. Consultas SQL de los reportes....61
10. Diagramas UML....64
11. Presentación de Stack Tecnológico....72
11.1. Base de datos: PostgreSQL....72
11.2. Metodología de desarrollo: Scrum....73
11.3. Lenguaje de programación: PHP 8.4....74

11.4. Framework: Laravel 13.....74
11.5. IDE: Visual Studio Code.....75
11.6. Plataforma de despliegue: Render y Supabase.....75
11.7. Herramientas de calidad.....76
11.8. Integración del stack tecnológico.....76
12. Métricas de Software.....78
12.1. Objetivo general.....78
12.2. Objetivos específicos.....78
12.3. Marco conceptual.....78
12.4. Métricas de proceso.....79
12.5. Métricas de proyecto.....80
12.6. Métricas de producto.....81
12.7. Métricas de impacto.....83
12.8. Evaluación de las métricas.....84
12.9. Plan de recolección de métricas.....85
12.10. Resultados esperados.....86
13. Implementación de Estándares de Seguridad.....87
13.1. Estándares de referencia.....87
13.2. Controles frente al OWASP Top 10.....87
13.3. Controles por capa.....88
13.4. Verificación de la seguridad.....88
14. Plan de Pruebas.....89
14.1. Objetivos.....89
14.2. Alcance.....89
14.3. Niveles y tipos de prueba.....90
14.4. Entorno de pruebas.....90
14.5. Procedimiento.....90
14.6. Criterios de entrada y de salida.....91
14.7. Gestión de defectos.....91
14.8. Roles y cronograma de pruebas.....91
14.9. Casos de prueba.....92
14.10. Resultados de las pruebas.....93
14.11. Conclusión de los resultados.....95
15. Criterios de Aceptación.....96
15.1. Criterios generales.....96
15.2. Criterios funcionales.....96
15.3. Criterios no funcionales.....97
15.4. Procedimiento de aceptación.....98
15.5. Condiciones para la aceptación.....98
16. Carta de Aceptación del Proyecto.....99
17. Conclusiones y Recomendaciones.....100
17.1. Conclusiones.....100
17.2. Recomendaciones.....100
18. Repositorio del Proyecto.....101

Introducción

El Transmetro se ha convertido en la alternativa de transporte público seguro para la Ciudad de Guatemala y sus municipios aledaños. Su expansión a nuevas líneas, cientos de estaciones y una flota cada vez mayor ha hecho evidente la necesidad de un sistema de información que controle de forma centralizada la red, los buses, el personal, los pagos y las reglas de operación que hoy se manejan de forma manual o aislada.

El presente documento contiene la propuesta de consultoría para el **Sistema de Control Transmetro**, una aplicación web alojada en la nube que permite a la municipalidad administrar sus líneas y estaciones, asignar y controlar la flota y los parqueos, registrar al personal operativo, monitorear el aforo de cada estación desde la PC de trabajo de su operador, cobrar el pasaje con la Tarjeta Ciudadana o con tarjeta bancaria y generar alertas automáticas cuando una estación se satura o un bus llega lleno.

La propuesta se desarrolló desde la perspectiva del Aseguramiento de la Calidad del Software. Por ello, además de describir el problema y la solución, define requerimientos verificables, aplica las reglas del enunciado directamente en la base de datos, mide el producto con métricas de software y demuestra su funcionamiento con un plan de pruebas automatizadas y criterios de aceptación claros.

El documento se organiza de la siguiente manera:

- **Capítulos 1 a 3:** antecedentes del problema, situación actual y deficiencias de los controles actuales.
- **Capítulo 4:** planificación del proyecto, calendario de entregas y diagrama de Gantt.
- **Capítulo 5:** análisis de requerimientos funcionales y no funcionales, con su matriz de trazabilidad.
- **Capítulos 6 y 7:** estudios de factibilidad y propuesta monetaria.
- **Capítulos 8 a 10:** modelo entidad relación, script de la base de datos y diagramas UML.
- **Capítulos 11 y 12:** stack tecnológico y métricas de software.
- **Capítulos 13 a 16:** implementación de estándares de seguridad, plan de pruebas con sus resultados, criterios de aceptación y carta de aceptación del proyecto.
- **Capítulos 17 y 18:** conclusiones, recomendaciones y enlaces al repositorio.

Carta de Presentación

Guatemala, 28 de agosto de 2026

Señores
Municipalidad de Guatemala
Dirección del Sistema Transmetro
Ciudad de Guatemala
Presente

Asunto: Propuesta de consultoría para el desarrollo del Sistema de Control Transmetro

Estimados señores:

Reciban un cordial saludo. Por este medio me permito presentar a su consideración la propuesta técnica y económica para el diseño, desarrollo e implementación del **Sistema de Control Transmetro**, una solución tecnológica orientada a fortalecer el control de las líneas, estaciones, flota, personal y recaudación del servicio de transporte de la municipalidad.

Como consultora en desarrollo de software y aseguramiento de la calidad, me especializo en el análisis y diseño de sistemas de información, bases de datos relacionales y aplicaciones web en la nube, con un enfoque de calidad que garantiza que cada regla de negocio se cumpla y se verifique antes de su puesta en producción.

La propuesta adjunta responde a las necesidades planteadas por la municipalidad e incluye el análisis de la situación actual, los requerimientos del sistema, los estudios de factibilidad, la inversión requerida, el diseño de la base de datos y de la aplicación, la planificación del proyecto, las métricas de calidad, los estándares de seguridad, el plan de pruebas y los criterios de aceptación. Como respaldo, se presenta un prototipo funcional que demuestra el cumplimiento de las reglas de operación y que fue verificado con 240 pruebas automatizadas y 190 verificaciones de extremo a extremo.

La inversión total del proyecto es de **Q42,912.37** (IVA incluido). El sistema completo será desarrollado en tres meses, entre agosto y octubre de 2026, por un desarrollador, con entregas parciales en cada etapa y la entrega final el 23 de octubre de 2026.

Mi compromiso es trabajar con responsabilidad, transparencia y apego a las buenas prácticas de calidad y seguridad de la información. Quedo a su disposición para ampliar cualquier aspecto de esta propuesta o realizar una demostración del sistema en el momento que lo consideren oportuno.

Atentamente,

_____

Wendy Celeste Nathaly Estrada González
Consultora en Desarrollo de Software y Aseguramiento de la Calidad
Teléfono: _____ Correo: _____

1. Antecedentes del problema

1.1. Contexto Histórico del Transporte Público en el Área Metropolitana

Durante los años, el sistema de transporte público en la Ciudad de Guatemala y sus municipios aledaños se caracterizó por un modelo de operación desorganizado, fragmentado y con un esquema de gestión predominantemente empírico. El sistema tradicional, operado por múltiples asociaciones de transportistas privados, funcionaba bajo un esquema donde la competencia por el pasaje dictaba el comportamiento de las unidades en las calles. Esto generó un desorden vial crónico, donde las unidades se detenían en cualquier punto de la vía pública, obstaculizando el tránsito y poniendo en riesgo la integridad de los peatones y usuarios.

La falta de planificación tecnológica y urbana en este modelo tradicional resultó en una infraestructura deficiente. No existían estaciones controladas, los tiempos de espera eran incalculables debido a la ausencia de rutas estandarizadas y el manejo de los ingresos se realizaba en efectivo y sin ningún tipo de auditoría o control centralizado. Este escenario sentó las bases para una vulnerabilidad estructural que pronto sería aprovechada por estructuras criminales.

1.2. La Crisis de Inseguridad y el Colapso del Modelo Tradicional

El factor más crítico que impulsó la necesidad de un cambio radical en el paradigma de movilidad urbana fue la escalada de violencia y delincuencia que sufren muchos de los municipios de la capital. A partir de la década de los 2000, el transporte público tradicional se convirtió en uno de los sectores más golpeados por la criminalidad.

El manejo de grandes cantidades de dinero en efectivo a bordo de las unidades convirtió a los pilotos y ayudantes en blancos fáciles para el cobro de extorsiones por parte de pandillas y grupos del crimen organizado. Esta situación degeneró en un índice de robos alarmante, asaltos a mano armada a los pasajeros e innumerables atentados contra la vida de los conductores. La percepción de seguridad del usuario se desplomó, obligando a la población a buscar alternativas de movilidad más costosas y saturando aún más el parque vehicular de la ciudad. El colapso operativo y social de este sistema hizo evidente que cualquier nuevo modelo de transporte no solo debía resolver un problema de movilidad, sino principalmente un problema de seguridad pública.

1.3. El Surgimiento del Sistema Transmetro como Medida Alternativa

Ante el inminente colapso del transporte tradicional y la urgencia de salvaguardar la vida de los ciudadanos, las municipalidades optaron como medida alternativa el poder introducir el sistema Transmetro en toda la capital y municipios aledaños. Este modelo se basó en el concepto de Autobús de Tránsito Rápido (BRT, por sus siglas en inglés), el cual requería una reingeniería total de la infraestructura vial y del modelo de gestión.

El Transmetro se diseñó con un enfoque preventivo contra la criminalidad. Para lograr que no exista un índice de robos dentro del sistema, se implementaron cambios radicales:

Infraestructura Cerrada: Se construyeron paradas de bus establecidas y exclusivas, eliminando la posibilidad de que personas ajenas al sistema aborden las unidades en puntos no autorizados.

Eliminación del Efectivo a Bordo: La implementación de métodos de pago electrónicos en las estaciones eliminó el incentivo principal para los asaltos y extorsiones dentro de las unidades.

Profesionalización del Personal: A diferencia del sistema anterior, los pilotos del Transmetro son mejor reclutados, sometidos a pruebas psicológicas, historial de antecedentes y capacitaciones constantes, dignificando la labor del conductor.

1.4. El Desafío de la Expansión Metropolitana y la Descentralización

El éxito inicial del Transmetro en las arterias principales de la capital demostró que un transporte público seguro y ordenado era viable. Esto provocó un efecto dominó en el área metropolitana. Al notar la eficiencia del modelo, diversas municipalidades aledañas comenzaron a buscar la integración o replicación de este sistema para conectar a sus poblaciones pendulares (ciudadanos que residen en municipios periféricos, pero trabajan o estudian en la capital).

Sin embargo, esta expansión trajo consigo un nivel de complejidad operativa sin precedentes. A medida que se sumaron nuevas líneas, cientos de estaciones y una flota vehicular cada vez mayor, la administración manual o mediante sistemas informáticos básicos dejó de ser sostenible. La logística de coordinar flotas intermunicipales requiere rastrear con precisión métricas que antes no se consideraban críticas: distancias exactas entre paradas, asignación y cambio controlado de parqueos, y la gestión masiva de personal de seguridad y operación en múltiples jurisdicciones.

1.5. Brecha Tecnológica y la Necesidad de Aseguramiento de la Calidad (Software)

Es en este punto de inflexión donde se enmarca el presente proyecto. A pesar de los grandes avances en infraestructura física y seguridad perimetral, el sistema enfrenta actualmente una "brecha tecnológica" en su control interno. La recolección de información sobre las líneas de transmetro de la ciudad de Guatemala y sus municipios requiere de una arquitectura de software robusta y altamente escalable.

Actualmente, el manejo de las reglas de negocio (cómo evitar que un bus se asigne a múltiples líneas o asegurar que cada estación cuente con al menos un guardia por acceso) representa un reto enorme si no está respaldado por bases de datos relacionales íntegras. Más aún, la dinámica moderna del transporte exige la toma de decisiones en tiempo real basada en análisis de datos, como la capacidad de generar alertas automáticas cuando una estación supera su capacidad máxima o aplicar penalizaciones de tiempo a unidades vacías.

La falta de una red interconectada entre las computadoras de las diferentes estaciones es un obstáculo crítico que impide la trazabilidad de los métodos de pago y el monitoreo en vivo de la flota. Por lo tanto, el desarrollo de un sistema de control de Aseguramiento de la Calidad del Software (SQA) no es solo una mejora administrativa, sino un pilar fundamental para

garantizar que el índice de delincuencia cero se mantenga, que los recursos municipales se optimicen y que el nivel de servicio al usuario justifique la inversión pública a largo plazo.

2. Definición de la situación actual

2.1. Contexto General y Despliegue Operativo

En la actualidad, las municipalidades de la capital y sus municipios aledaños han optado por introducir el transmetro como una medida alternativa para contrarrestar la delincuencia. Este sistema se encuentra en una fase operativa donde ha logrado consolidar aspectos clave para la seguridad ciudadana: se han establecido paradas de bus físicas, se cuenta con pilotos que son mejor reclutados y se mantiene un escenario donde no hay un índice de robos.

Sin embargo, desde el punto de vista del Aseguramiento de la Calidad del Software (SQA) y la gestión de la información, el escenario actual presenta una alta complejidad logística que demanda un sistema de control centralizado. Se requiere recoger y estructurar de manera sistemática toda la información operativa de las líneas de transmetro que atraviesan la ciudad de Guatemala y sus municipios. En el momento presente, la coordinación de esta información se divide en múltiples frentes (infraestructura, flota, personal y reglas operativas) que deben ser mapeados con precisión para la propuesta del nuevo sistema de control.

2.2. Arquitectura de Rutas, Estaciones y Jurisdicciones

La red física del sistema de transporte se compone actualmente de una estructura jerárquica y relacional que define la movilidad de las unidades. A nivel jurisdiccional, la situación actual estipula que cada línea y estación que existe pertenece a una municipalidad específica. Esto implica que el sistema opera en un entorno intermunicipal que requiere un control detallado de la ubicación de sus activos.

En cuanto a la configuración de las rutas, la situación operativa define que una línea está compuesta por una serie de estaciones dispuestas en un orden determinado. Es un requerimiento absoluto para la operación actual recoger la información de este orden, abarcando tanto el diseño de la ruta como las visitas secuenciales que cada unidad de transporte vaya a realizar durante el momento de su recorrido.

La interacción entre líneas y estaciones presenta una topología de red compleja:

Cada estación pertenece al menos a una línea, pero la operación permite que una estación pueda pertenecer a varias líneas simultáneamente (puntos de transbordo). Para la planificación espacial y de tiempos, actualmente se necesita saber qué distancia existe de una estación a otra. Asimismo, como métrica global de operación, se necesita saber qué distancia total tiene una línea completa.

2.3. Control de Accesos y Seguridad Física en Estaciones

La infraestructura de las estaciones ha sido diseñada para controlar el flujo de pasajeros de forma segura y estructurada, lo cual impacta directamente en la necesidad de control del sistema informático actual. Físicamente, la arquitectura establece que cada estación puede tener varios accesos, mientras que un acceso sólo puede pertenecer a una estación en particular. Desde el punto de vista operativo y de monitoreo, al sistema le interesa conocer todos los accesos específicos de cada línea para mapear los puntos de entrada de los usuarios.

La seguridad, siendo el pilar fundamental que motivó este modelo de transporte, impone restricciones estrictas sobre el personal desplegado. En la situación actual, las normativas exigen que las estaciones cuenten por lo menos con un guardia de seguridad asignado por cada acceso disponible. Esto genera una matriz de asignación de recursos humanos que debe ser controlada continuamente para garantizar la operatividad de cada parada de bus.

2.4. Gestión de los Buses e Infraestructura de Parqueo

La administración de los activos móviles (buses) y estáticos (parqueos) es uno de los componentes más críticos y estrictos de la situación actual. La asignación de la flota está regida por reglas matemáticas y lógicas que buscan optimizar el uso de los vehículos sin saturar las rutas ni dejar estaciones desatendidas:

Cada línea tiene asignados una serie de buses de forma exclusiva.

La regla de negocio actual impide que suceda que un bus esté asignado a más de una línea al mismo tiempo.

Se contempla la existencia de una flota de reserva, por lo que es permitido que un bus no esté asignado a ninguna línea temporalmente.

Existe un balance de carga por ruta: cada línea tiene asignados como mínimo tantos buses como estaciones tenga, y como máximo el doble del número de estaciones.

Por otro lado, la logística nocturna y de resguardo de las unidades exige un control riguroso de la infraestructura de parqueos. En la actualidad, solo algunas estaciones tienen asignados parqueos en sus instalaciones. Sin embargo, la regla operativa obliga a que cada bus tenga asignado un parqueo en todo momento. La flexibilidad del sistema permite que un bus pueda cambiar de parqueo según las necesidades operativas, pero bajo ninguna circunstancia puede quedarse sin él.

2.5. Dinámica de Aforos, Tiempos de Espera y Reglas de Negocio en Tiempo Real

El control del volumen de pasajeros es el mayor desafío operativo en la situación actual. Los buses que conforman la flotilla tienen una capacidad máxima de pasajeros establecida por el fabricante y las normativas de tránsito. Para mantener la calidad del servicio y evitar el hacinamiento en los andenes, el sistema exige mecanismos de respuesta inmediata basados en el flujo de usuarios:

Alerta de Saturación: en el momento en que la cantidad de pasajeros esperando en una estación supera por lo menos un 50 % más de su capacidad, es decir, el 150 % del aforo máximo de la estación, la operación exige generar una alerta para que otro bus llegue a solventar la demanda.

Penalización por Baja Demanda: Por el contrario, para evitar el desperdicio de combustible y optimizar las frecuencias, si un bus no llena el 25% de su capacidad al llegar, la regla estipula que deberá esperar 5 minutos más en cada estación para recolectar más usuarios antes de continuar su recorrido.

Actualmente, procesar estas variables matemáticas de aforo y tiempo de espera en tiempo real representa una necesidad urgente que justifica la implementación de la nueva plataforma de software.

2.6. Estado de la Infraestructura Tecnológica y Telecomunicaciones

A nivel de hardware y redes, la situación actual se encuentra en una fase de diseño y provisión de equipo. Es importante mencionar que la planificación estratégica espera tener una PC de trabajo instalada para cada estación de bus. Esta infraestructura de hardware requiere incluir a un operador designado que pueda tener el control de dichas máquinas en el sitio.

A pesar de contar con la disposición de adquirir el hardware necesario, el proyecto enfrenta un vacío a nivel de arquitectura de redes. Al día de hoy, aún no se ha determinado cuál es la mejor forma tecnológica o de telecomunicaciones para conectar las máquinas de un lugar a otro (LAN, WAN, VPN, conexiones en la nube, etc.). Esta indefinición técnica afecta la forma en que los datos fluyen entre las estaciones locales y el servidor centralizado de la propuesta.

Finalmente, en cuanto al modelo de recaudación financiera y acceso de los usuarios, la situación operativa ha establecido como estándar que los métodos de pago se realicen exclusivamente a través de una tarjeta de acceso. La integración de los lectores de estas tarjetas en las estaciones con las computadoras de los operadores será fundamental para registrar los ingresos y el flujo de pasajeros que dispara las alertas de capacidad mencionadas anteriormente.

2.7. Registro y Control del Capital Humano Central

Más allá del personal de campo y seguridad, el soporte administrativo requiere llevar un control estricto de los conductores que tienen a su cargo las unidades. La situación actual exige que para los pilotos se necesita conocer detalladamente su historial educativo, sus datos de residencia exacta y sus canales de comunicación. Esta recopilación de datos no solo responde a políticas de recursos humanos, sino que es una extensión del estricto filtro de seguridad que caracteriza al reclutamiento del Transmetro, asegurando la trazabilidad total de los perfiles de quienes operan el sistema.

3. Deficiencias de los controles actuales

Carencia de Integración Tecnológica Centralizada

El principal vacío operativo radica en la ausencia de una arquitectura de red definida, ya que aún no se ha determinado cuál es la mejor forma de conectar las máquinas de un lugar con otro. Esto impide la consolidación de datos transaccionales en un Data Warehouse o entorno centralizado, limitando drásticamente el control que el operador local puede ejercer desde la PC asignada en su estación de bus. Sin una red robusta, la comunicación de los sistemas del área metropolitana de Guatemala queda aislada.

3.1. Vulnerabilidades en el Control de Buses y Asignación

La gestión actual del transporte genera riesgos críticos en la integridad referencial de los recursos físicos, requiriendo validaciones estrictas a nivel de base de datos:

Al carecer de bloqueos sistémicos, existe el riesgo de que un bus quede asignado a más de una línea simultáneamente. No existe un mecanismo (como triggers o procedimientos almacenados) que garantice el balance de carga: asignar a cada línea como mínimo tantos buses como estaciones tenga, y como máximo el doble de su número de estaciones.

El control de infraestructura es vulnerable a errores humanos, permitiendo el riesgo de que un bus pueda quedarse sin un parqueo asignado.

3.2. Deficiencias en el Monitoreo de Aforo y Tiempos

La operación carece de inteligencia de negocios en tiempo real para gestionar el flujo de pasajeros y optimizar las frecuencias de las rutas:

Falta un sistema de monitoreo automatizado que genere una alerta inmediata para despachar otro bus en el momento que una estación supera por lo menos un 50% más de su capacidad. No hay controles lógicos para obligar a una unidad a esperar 5 minutos más en cada estación si el bus no llena el 25% de su capacidad.

La ausencia de un sistema que calcule automáticamente qué distancia existe de una estación a otra, o qué distancia total tiene una línea, dificulta la planificación logística.

3.3. Riesgos en la Seguridad de Datos y Personal

El modelo adolece de controles para la auditoría continua del factor humano y las transacciones financieras:

El ecosistema de cobro mediante métodos de pago con tarjeta de acceso requiere un entorno seguro para evitar fraudes en el uso del servicio. Existe un control deficiente sobre los perfiles de los pilotos, dificultando conocer al instante su historial educativo, datos de residencia y medios de comunicación.

El sistema no audita de forma proactiva que se cumpla la restricción de mantener por lo menos un guardia de seguridad por cada acceso de la estación.

4. Planificación

El proyecto se planificó para desarrollarse en tres meses, del 3 de agosto al 23 de octubre de 2026. El sistema completo lo desarrolla un solo desarrollador (la consultora), con un esfuerzo estimado de 502 horas (capítulo 7). El trabajo se organizó de acuerdo con el calendario de entregas establecido, de manera que cada fase del análisis, la propuesta, el diseño, el desarrollo y las pruebas concluye con un entregable revisado en una fecha definida.

4.1. Metodología de trabajo

El proyecto se desarrolla con **Scrum**, un marco de trabajo ágil que organiza el trabajo en ciclos cortos llamados sprints. Cada sprint termina con uno o varios entregables del calendario, que se revisan con el Product Owner (representante de la municipalidad); esto permite incorporar observaciones y cambios en los requerimientos sin rehacer el trabajo terminado. Los roles, artefactos y la definición de terminado se detallan en el capítulo 11.

4.2. Calendario de entregas

Tabla 4.1. Calendario de entregas del proyecto.

No.	Entregable	Fecha	Sprint	Capítulo
1	Carta de presentación personal	28/08/2026	1	Carta de presentación
2	Antecedentes del problema	28/08/2026	1	1
3	Definición de la situación actual	28/08/2026	1	2
4	Deficiencias de los controles actuales	28/08/2026	1	3
5	Planificación	28/08/2026	1	4
6	Análisis de requerimientos	04/09/2026	2	5
7	Estudios de factibilidad (técnica, operativa, económica, legal y ambiental)	11/09/2026	3	6
8	Propuesta monetaria de la solución	11/09/2026	3	7
9	Modelo entidad relación (diagrama)	18/09/2026	4	8
10	Script para la creación de las estructuras de la base de datos	18/09/2026	4	9
11	Diagramas UML completos (CU, componentes, secuencia, actividades, clases y objetos)	18/09/2026	4	10
12	Presentación del stack tecnológico (BD, metodología, lenguaje, framework e IDE)	02/10/2026	5	11
13	Métricas de software	02/10/2026	5	12
14	Demo funcional de la aplicación web	10/10/2026	6	Aplicación
15	Plan de pruebas	17/10/2026	7	14
16	Criterios de aceptación	17/10/2026	7	15
17	Carta de aceptación del proyecto	17/10/2026	7	16
18	Entrega final (documento unificado, presentación ejecutiva y video)	23/10/2026	8	Documento completo

4.3. Diagrama de Gantt

La figura 1 muestra el diagrama de Gantt del proyecto. Cada barra representa el período de trabajo de un entregable y el rombo rojo indica su fecha de entrega. El desarrollo de la aplicación web se realiza en paralelo al diseño y a la documentación técnica, desde mediados de septiembre hasta la demo funcional del 10 de octubre.

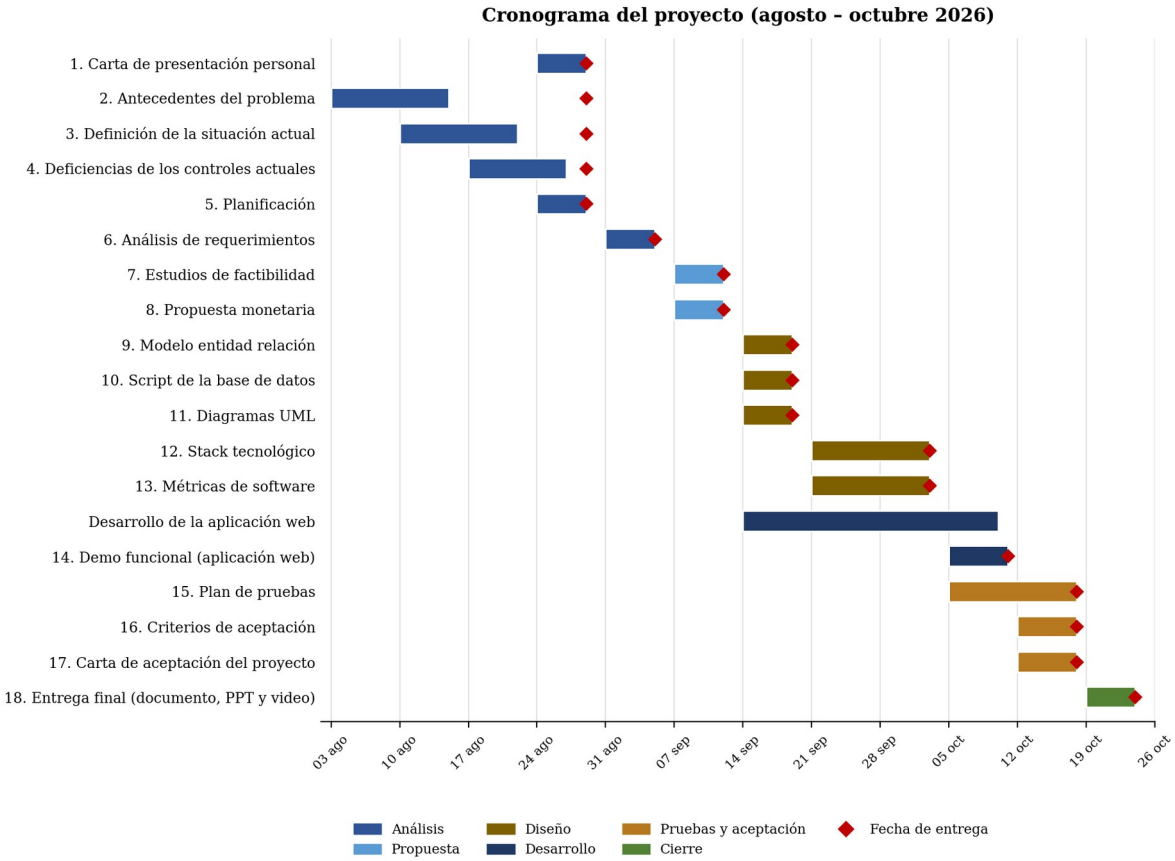


Figura 1. Diagrama de Gantt del proyecto (agosto – octubre 2026).

4.4. Sprints del proyecto

Tabla 4.2. Sprints y objetivos.

Sprint	Fechas	Objetivo	Entregables
1	03/08 – 28/08	Comprender el problema: antecedentes, situación actual, deficiencias y planificación	1 a 5
2	31/08 – 04/09	Definir los requerimientos funcionales y no funcionales	6
3	07/09 – 11/09	Evaluar la factibilidad y estimar el costo de la solución	7 y 8
4	14/09 – 18/09	Diseñar la base de datos y el sistema; iniciar el desarrollo de la aplicación	9 a 11
5	21/09 – 02/10	Definir el stack y las métricas; desarrollar los módulos principales	12 y 13
6	05/10 – 10/10	Completar la aplicación web y presentar la demo funcional	14
7	12/10 – 17/10	Ejecutar y documentar las pruebas y definir la aceptación del proyecto	15 a 17
8	19/10 – 23/10	Unificar el documento, preparar la presentación ejecutiva y el video	18

4.5. Seguimiento del cronograma

- Al cierre de cada sprint se revisan los entregables con el Product Owner y se registran las observaciones.
- El avance se controla con el tablero Scrum y con las métricas de proyecto del capítulo 12 (cumplimiento del sprint, velocidad e índice de desempeño del cronograma).
- Si un entregable presenta atraso, se prioriza dentro del siguiente sprint sin mover las fechas del calendario de entregas.

5. Análisis de Requerimientos

Objetivo del sistema

El objetivo principal del sistema será proporcionar una herramienta tecnológica integral para la administración y supervisión del sistema Transmetro, permitiendo controlar los principales recursos involucrados en la operación y automatizar las reglas de negocio establecidas.
La plataforma permitirá centralizar la información de las diferentes estaciones y líneas, controlar la asignación de buses y parqueos, administrar el personal involucrado en la operación y seguridad, monitorear la demanda de pasajeros y generar alertas cuando las condiciones de operación lo requieran.
De esta manera, la solución pretende convertirse en un punto central de control y consulta para la gestión del transporte, proporcionando información confiable que permita actuar oportunamente ante situaciones que puedan afectar la calidad del servicio

Requerimientos funcionales

RF-01. Gestión de Municipalidades

El sistema deberá mantener centralizada la información de las municipalidades en las que opera el servicio de transporte y relacionarlas con sus líneas y estaciones (punto 11 del enunciado).

El módulo deberá permitir:

- Registrar cada municipalidad con su código, nombre y estado. Los registros se cargan a partir de la información oficial de la Municipalidad de Guatemala y de los municipios donde opera el Transmetro.
- Consultar las municipalidades con la cantidad de líneas y estaciones que les pertenecen.
- Asociar cada línea y cada estación a una sola municipalidad.
- Incorporar una nueva municipalidad mediante carga de datos, sin modificar el código del sistema (RNF-05).

Regla de negocio: cada estación pertenece únicamente a un municipio, aunque un mismo municipio puede contar con múltiples estaciones y líneas. La relación se garantiza con llaves foráneas obligatorias.

Para evitar errores en la red oficial, en la aplicación web los catálogos de municipios, líneas y estaciones son de consulta: no se crean ni eliminan desde la pantalla, sino desde la carga controlada de datos oficiales.

RF-02. Gestión de Líneas y Recorridos

El sistema deberá registrar las líneas de transporte y el recorrido de cada una mediante la asociación ordenada de estaciones (puntos 1, 2, 8, 12 y 13 del enunciado).

El módulo deberá permitir:

- Registrar cada línea con código, nombre, municipalidad, tipo de servicio, tarifa, color oficial, si la ruta es circular y estado (activa, en mantenimiento o inactiva).

- Definir el orden en que se recorren las estaciones y la distancia desde la parada anterior.
- Calcular automáticamente la distancia total de la línea como la suma de sus tramos (trigger trg_linea_estacion_distancia); este dato no se ingresa a mano.
- Consultar el recorrido completo, el horario, los buses asignados y todos los accesos de la línea.
- Cambiar el estado de la línea. Una línea solo puede activarse si tiene entre n y 2n buses, donde n es su número de estaciones.

Una misma estación puede formar parte de varias líneas; por ejemplo, San Sebastián es estación de transbordo de las Líneas 1 y 2, y las Líneas 6 y 18 comparten estaciones. El sistema contiene las 8 rutas oficiales: 1, 2, 6, 7, 12, 13, 18 y Ruta 5.

RF-03. Gestión de Estaciones

El sistema deberá administrar la información de las estaciones que forman parte de la infraestructura del servicio de transporte.

Cada estación deberá contar con la siguiente información:

- Identificador, nombre y dirección.
- Municipalidad a la que pertenece.
- Aforo máximo y aforo actual (personas que esperan).
- Coordenadas geográficas (latitud y longitud).

El sistema deberá permitir:

- Consultar cada estación con sus líneas, accesos, molinetes, guardias, parqueos y operador asignado.
- Calcular el porcentaje de ocupación respecto del aforo máximo.
- Ubicar la estación en el mapa de rutas y buscarla por nombre.
- Cambiar el estado del molinete de cada acceso (activo, inactivo o en mantenimiento).

La situación de la estación se conoce por su aforo y por sus alertas abiertas; no se maneja un campo de estado operativo independiente.

RF-04. Gestión de Accesos y Molinetes

El sistema deberá administrar los accesos físicos de cada estación, los molinetes asociados a esos accesos y los molinetes de entrada y salida de cada bus.

El módulo deberá permitir:

- Registrar los accesos de una estación con su ubicación física (por ejemplo, “Acceso Norte”).
- Asociar un molinete propio a cada acceso, con su dirección IP y su estado operativo (activo, inactivo o en mantenimiento).
- Crear automáticamente un molinete de entrada y uno de salida para cada bus nuevo (trigger trg_buses_molinetes).
- Asignar personal de seguridad a cada acceso.
- Consultar todos los accesos de una estación y de una línea (punto 8).

Reglas de negocio:

- Un acceso pertenece a una sola estación (punto 3).
- Cada estación debe contar con al menos dos accesos físicos; el trigger trg_accesos_min_dos_before_delete impide eliminar el penúltimo acceso.
- Cada acceso dispone de un molinete propio (relación 1 a 1).
- Cada acceso debe contar con al menos un guardia de seguridad (punto 10, trigger trg_seguridad_min_uno).

RF-05. Gestión Integral de la Flota

El sistema deberá administrar las unidades de transporte que conforman la flota y controlar su asignación a las líneas (puntos 4 y 5).

Para cada bus se deberá manejar:

- Identificador y placa con el formato de Guatemala (por ejemplo, C387JGL).
- Tipo de unidad (biarticulado, articulado, convencional o eléctrico) y capacidad máxima.
- Línea asignada, parqueo y piloto.
- Pasajeros a bordo y porcentaje de ocupación.
- Estado: en ruta, en espera, fuera de servicio o en mantenimiento.

El sistema deberá permitir:

- Registrar, editar y eliminar buses; al eliminar un bus se libera su parqueo.
- Asociar un bus a una línea activa o dejarlo sin línea (flota de reserva).
- Consultar el estado de cada unidad, en qué estación va y cuál es su próxima parada.
- Buscar buses por placa y filtrar por línea, sin línea o sin piloto asignado.

Reglas de negocio: un bus solamente puede estar asignado a una línea a la vez y una unidad sin piloto no puede salir a ruta (trigger trg_buses_ruta_con_piloto). Cada línea opera con el tipo de unidad que le corresponde; por ejemplo, las Líneas 6 y 18 con buses articulados.

RF-06. Gestión de Parqueos

El sistema deberá administrar los parqueos con los que cuentan algunas estaciones y garantizar que cada bus tenga siempre un parqueo asignado (puntos 6 y 7 del enunciado).

El sistema deberá permitir:

- Registrar los parqueos (bahías) de una estación con un código único.
- Asignar un parqueo a cada bus al registrarlo. Un parqueo solo puede tener un bus y un bus solo ocupa un parqueo.
- Cambiar el parqueo de un bus mediante el procedimiento almacenado sp_cambiar_parqueo_bus, que libera el parqueo anterior y ocupa el nuevo en una sola transacción.
- Consultar los parqueos libres y ocupados de cada estación.

Regla de negocio: un bus puede cambiar de parqueo, pero nunca quedarse sin él, y no puede moverse a un parqueo ocupado. La asignación es fija hasta que el administrador la cambia.

RF-07. Gestión del Personal Operativo

El sistema deberá administrar el personal que participa en la operación del servicio, diferenciando las funciones de Piloto, Seguridad y Operador (punto 9).

Para todos los empleados se deberá gestionar:

- Nombre completo, fecha de nacimiento y DPI.
- Teléfono y correo electrónico, validados por separado.
- Dirección residencial.
- Estado del empleado (activo o inactivo).

Adicionalmente, el sistema deberá manejar información específica según el puesto:

- **Pilotos:** número y tipo de licencia, años de experiencia, nivel educativo e historial educativo.
- **Personal de seguridad:** permisos vigentes, constancia de antecedentes penales y número de placa.
- **Operadores:** nivel de estudios y años de experiencia en monitoreo y control.

El sistema deberá permitir registrar, editar, consultar y eliminar empleados, buscarlos por nombre y filtrarlos por puesto. Los datos de contacto, la dirección y el historial educativo se guardan cifrados. Al registrar un operador, el sistema le crea su usuario de acceso.

RF-08. Gestión de Asignación de Personal

El sistema deberá relacionar al personal operativo con los recursos que tiene bajo su responsabilidad.

El módulo deberá permitir:

- Asignar un piloto a cada bus.
- Asignar personal de seguridad a los accesos de cada estación.
- Asignar un operador a la PC de trabajo de cada estación (punto 16). Una estación tiene un solo operador.
- Consultar el personal asignado a cada línea: el piloto de cada bus y el operador y los guardias de cada estación.

Reglas de negocio: todo acceso cuenta como mínimo con un guardia de seguridad (punto 10). El control es preventivo: el sistema rechaza eliminar o reasignar el último guardia de un acceso y muestra un mensaje claro. Si un piloto deja su bus mientras va en ruta, el bus termina su viaje y queda en espera.

RF-09. Monitoreo Operativo de la Estación

El sistema deberá proporcionar al operador de cada estación una pantalla para supervisar y controlar su estación desde la PC de trabajo (punto 16).

Desde su estación, el operador deberá poder:

- Consultar el aforo actual, el porcentaje de ocupación y las personas que esperan.
- Consultar los buses detenidos en la estación y los que vienen en camino, con su ocupación y la espera extra que les corresponde.

- Consultar los accesos, el estado de sus molinetes y los guardias asignados, y cambiar el estado de un molinete de su estación.
- Pedir otro bus para su estación y atender las alertas de su estación.
- Registrar a mano un pase cuando la tarjeta no se lee o el validador no responde, y las bajadas cuando falla el contador de salida, solo en un bus detenido en su estación.
- Reportar la falla de un bus de las líneas que pasan por la estación (RF-24).
- Consultar los cobros realizados en la estación y el mapa con las líneas de su estación.

Un operador solo puede actuar sobre su propia estación. La ubicación de los buses por GPS y el control de bahías de abordaje se proponen para una fase 2; en esta versión la posición de los buses se calcula a partir de su recorrido.

RF-10. Gestión de Pasajeros y Ocupación

El sistema deberá registrar y controlar la cantidad de pasajeros en las estaciones y en los buses para conocer el nivel de ocupación y apoyar la toma de decisiones (punto 14).

El sistema deberá:

- Registrar la cantidad de personas que esperan en cada estación (aforo actual) y calcular su porcentaje respecto del aforo máximo.
- Contar a los pasajeros de cada bus con sus molinetes: cada pago en el molinete de entrada suma una persona y cada paso por el molinete de salida la resta (a bordo = subidas − bajadas).
- Calcular automáticamente el porcentaje de ocupación del bus a partir de los pasajeros a bordo.
- Impedir que suban más personas de la capacidad del bus y que bajen más de las que van a bordo.
- Identificar situaciones de alta acumulación de pasajeros.

Regla de negocio (punto 14): cuando una estación tenga por lo menos un 50 % más de su capacidad, es decir, cuando el aforo actual sea mayor o igual al 150 % del aforo máximo, el sistema genera automáticamente una alerta para que otro bus llegue (trigger trg_estaciones_alerta_aforo). Ejemplo: una estación con aforo de 300 personas genera la alerta al llegar a 450. Si un bus llega lleno y hay gente esperando, se genera además una alerta de bus lleno.

RF-11. Gestión de Buses de Refuerzo y Límite de Flota

El sistema deberá incorporar unidades adicionales cuando la demanda lo requiera, respetando el tamaño de flota permitido para cada línea (puntos 5 y 14).

Cuando se genere una alerta de refuerzo o de bus lleno, el sistema deberá:

- Registrar la alerta con la estación, el bus cuando corresponda, la descripción y la fecha y hora.
- Mostrarla en la pantalla del operador de la estación y en el mapa del administrador.
- Permitir que el operador la atienda: el sistema envía un bus disponible de la misma línea y registra la fecha de cierre.

- Enviar el bus de forma automática desde la central si nadie atiende una alerta de bus lleno en 90 segundos.
- Mantener la alerta abierta si no hay buses disponibles, y no enviar refuerzos fuera del horario de la línea.

Regla de negocio (punto 5): cada línea debe tener como mínimo tantos buses como estaciones (n) y como máximo el doble (2n). El trigger trg_buses_max_flota rechaza cualquier asignación que supere 2n, y el reporte de líneas señala las que no cumplen.

RF-12. Gestión de Tiempos de Espera

El sistema deberá calcular y controlar el tiempo que las unidades permanecen en las estaciones de acuerdo con su nivel de ocupación (punto 15).

El sistema deberá:

- Registrar la hora de llegada y de salida de un bus en cada estación.
- Registrar el porcentaje de ocupación del bus al llegar.
- Identificar unidades con baja ocupación y aplicarles la espera adicional.
- Indicar al operador los minutos de espera adicionales de cada bus.

Regla de negocio (punto 15): si un bus no llena el 25 % de su capacidad, deberá esperar 5 minutos más en cada estación, además de su tiempo normal de parada.

RF-13. Gestión de Viajes y Seguimiento de Recorridos

El sistema deberá controlar los viajes realizados por cada unidad dentro de su línea (punto 1).

Cada viaje deberá registrar:

- Bus utilizado y línea correspondiente.
- Fecha y hora de inicio y de finalización.
- Estado del viaje (en curso, finalizado o cancelado).

Durante el recorrido, el sistema registra automáticamente cada visita a una estación con:

- Estación visitada y orden de visita.
- Hora de llegada y de salida.
- Ocupación al llegar y personas que subieron y bajaron.

Esto permite construir el historial operativo de cada unidad, conocer en qué estación va y cuál es la siguiente, y determinar cómo se desarrolló un recorrido.

RF-14. Métodos de Pago y Transacciones

El sistema deberá registrar el pago de cada pasajero al subir al bus, en el molinete de entrada del bus (punto 18). De acuerdo con la operación del Transmetro, el molinete no acepta efectivo.

El sistema deberá permitir procesar:

- Tarjeta Ciudadana (tarjeta de acceso oficial): se descuentan primero los viajes gratis y después el saldo. El número se acepta como va impreso o como lo entrega el código QR.
- Tarjeta bancaria de débito o crédito sin contacto.

Cada transacción deberá almacenar como mínimo:

- Identificador de la transacción, molinete, bus, estación y visita en que se realizó.
- Línea y tarifa aplicada.
- Tarjeta utilizada, cuando es Tarjeta Ciudadana.
- Monto, método de pago, si fue un viaje gratis, y fecha y hora.

Reglas de negocio: se cobra un pago por persona y por subida; no se registra el pase si la tarjeta está bloqueada, dada de baja, no existe o no tiene saldo suficiente; el monto debe ser mayor a cero salvo en los viajes gratis; no se cobra en un molinete fuera de servicio, en un bus sin viaje en curso ni en una línea inactiva. Si el sistema o el validador fallan, el operador puede registrar el pase manualmente (RF-09).

RF-15. Gestión de Alertas Operativas

El sistema deberá generar y administrar alertas relacionadas con situaciones que afecten la continuidad o eficiencia del servicio.

Tipos de alerta contemplados en la base de datos:

- **refuerzo_bus:** la estación llegó al 150 % de su aforo. Se genera automáticamente.
- **bus_lleno:** un bus lleno no puede recoger a las personas que esperan. Se genera automáticamente.
- **espera_minima:** bus con ocupación menor al 25 % (la espera de 5 minutos se aplica directamente).
- **bahia_ocupada:** reservada para el control de bahías de la fase 2.
- **otro:** situaciones relacionadas con la operación de la estación.

Cada alerta deberá registrar el tipo, la estación, el bus cuando corresponda, la descripción, la fecha y hora de creación, el estado (abierta, atendida o cerrada) y la fecha de cierre. El usuario que la atiende queda registrado en la bitácora de auditoría.

La falta de guardia y el exceso de flota no generan alerta porque el sistema los impide directamente (RF-08 y RF-11). Las alertas abiertas se muestran al operador de la estación y en el mapa del administrador; si un operador intenta atender una alerta ya atendida, recibe un aviso en lugar de un error.

RF-16. Gestión de Usuarios, Roles y Permisos

El sistema deberá controlar el acceso a las funcionalidades según las responsabilidades de cada usuario.

Los usuarios deberán contar con:

- Credenciales de acceso (correo y contraseña cifrada).
- Rol dentro del sistema.
- Empleado vinculado, cuando corresponda (por ejemplo, el operador y su estación).

El sistema deberá controlar que cada usuario únicamente pueda ejecutar las operaciones de su rol (middleware VerificarRol en cada ruta), bloquear el inicio de sesión después de 5 intentos fallidos y cerrar la sesión al cerrar el navegador.

Roles implementados:

- **Administrador:** red oficial, flota, personal, tarjetas, reportes, simulación y mapa general.
- **Operador:** monitoreo y operación de la estación asignada.

Roles propuestos para la fase 2:

- **Piloto:** consulta del recorrido y de las paradas asignadas.
- **Seguridad:** registro de incidencias en los accesos.

RF-17. Gestión de Auditoría

El sistema deberá registrar las operaciones relevantes realizadas por los usuarios para garantizar la trazabilidad de las acciones ejecutadas en la plataforma.

La bitácora deberá almacenar:

- Usuario y empleado que realizó la operación (o «Sistema» cuando la realiza la central o la simulación).
- Módulo afectado y registro afectado, con su nombre legible (placa, empleado, tarjeta, línea o estación).
- Tipo de acción: registro (INSERT), cambio (UPDATE) o eliminación (DELETE).
- Solo los datos que cambiaron, con su valor anterior y su valor nuevo.
- Fecha y hora exacta de la operación.

Se registran los cambios de buses (alta, edición, parqueo, molinetes y eliminación), personal (alta, edición y eliminación), estado de líneas y de los molinetes de los accesos, tarjetas, mantenimientos, horas pico y reinicios de la simulación, y todas las acciones del operador (alertas, molinetes, pases, bajadas y fallas).

El administrador consulta la bitácora en el menú Gestión → Bitácora, con cifras del día y filtros por módulo, acción, usuario, rango de fechas y búsqueda por placa, nombre o número de tarjeta.

Reglas: los datos personales (DPI, teléfono, correo, dirección e historial educativo) nunca se copian a la bitácora, solo se indica que fueron actualizados (RNF-12); la bitácora no puede modificarse ni borrarse, porque el trigger trg_bitacora_inmutable rechaza cualquier intento; solo el administrador puede consultarla.

RF-18. Comunicación entre Estaciones y Servicios Centrales

El sistema deberá permitir que el personal autorizado de cada estación acceda a la plataforma mediante Internet desde el equipo asignado a la estación (puntos 16 y 17).

La aplicación funcionará mediante el siguiente flujo:

Computadora de estación (navegador) → HTTPS → Aplicación Laravel en Render → Base de datos PostgreSQL en Supabase

Las acciones realizadas desde una estación son enviadas al servidor para:

- Validar al usuario y verificar sus permisos.
- Validar la información recibida y aplicar las reglas de negocio.

- Registrar o actualizar la información y generar las alertas correspondientes.
- Devolver el resultado de la operación al usuario.

Ningún equipo de estación se conecta directamente con la base de datos central. En caso de pérdida de conexión a Internet, la operación no se registra a medias: el sistema informa al operador que no pudo procesarse y la operación puede repetirse cuando se restablezca la comunicación. Un modo sin conexión con almacenamiento local queda como mejora futura.

RF-19. Gestión de Reportes Operativos

El sistema deberá generar reportes que apoyen la supervisión y administración del servicio. El enunciado pide reportes de estaciones, líneas y buses asignados a ellas, que se muestren en la aplicación web y cuyas consultas queden en este documento (sección 9.2).

Reportes que pide el enunciado:

- **Estaciones:** municipalidad, líneas, accesos, guardias, parqueos y aforo.
- **Líneas:** estaciones, distancia, flota mínima, máxima y asignada, y cumplimiento de la regla de n a 2n.
- **Buses asignados a cada línea:** agrupados por línea, incluidos los buses sin línea.

Reportes de operación:

- **Recorridos:** viajes realizados con la distancia acumulada.
- **Visitas:** estaciones que visitó cada bus con su ocupación.
- **Pasajeros:** personas que subieron y bajaron por línea.
- **Verificación de cobros:** comprueba que cada persona que subió tiene su pago y que lo cobrado coincide con la tarifa.

Los reportes muestran cifras clave, se pueden filtrar por línea y por municipalidad, y tienen un diseño para imprimir o guardar en PDF. Los filtros están protegidos contra inyección SQL.

RF-20. Mapa de Rutas y Monitoreo en Vivo

El sistema deberá mostrar en un mapa las líneas del Transmetro con sus estaciones, utilizando la información oficial de la Municipalidad de Guatemala (puntos 1 y 12).

El sistema deberá permitir:

- Visualizar las 8 líneas con su color oficial, su trazo por las calles y sus estaciones sobre un mapa de OpenStreetMap.
- Cargar el trazo oficial de cada línea desde el archivo KML o KMZ de la Municipalidad, reconociendo los nombres de las estaciones aunque cambien un poco.
- Abrir el panel de una ruta con sus estaciones en orden, buses con su piloto, personal de cada estación, pasajeros del día y alertas abiertas.
- Mostrar la posición estimada de cada bus en ruta, entre su última estación y la siguiente, y actualizarla en vivo.
- Mostrar al operador solo las líneas de su estación, con su estación resaltada.

RF-21. Tarjeta Ciudadana

El sistema deberá administrar la tarjeta de acceso indicada en el punto 18 del enunciado.

El sistema deberá:

- Registrar tarjetas de tipo vecino, adulto mayor o persona con discapacidad. Las dos últimas son personalizadas y requieren el nombre y el DPI de 13 dígitos del titular.
- Aplicar las condiciones de cada tipo: la tarjeta vecino cuesta Q20.00 e incluye 5 viajes gratis; la de adulto mayor tiene 4 viajes de cortesía cada día; la de persona con discapacidad es gratuita.
- Registrar recargas de Q5.00 a Q200.00, que se realizan en el punto de recarga (no en el molinete ni en la pantalla del administrador).
- Consultar el saldo, los viajes gratis, el historial de recargas y pases, y la tarjeta por ambos lados con su número impreso y su código QR.
- Bloquear una tarjeta indicando el motivo, desbloquearla o darla de baja de forma definitiva. Una tarjeta bloqueada no paga ni se recarga.

RF-22. Horarios de Servicio

El sistema deberá registrar el horario oficial de cada línea para tres tipos de día (lunes a viernes, sábado, y domingo y feriados) e indicar si una línea está en servicio en un momento dado. Fuera de horario no salen buses ni refuerzos, los buses que terminan su vuelta se retiran y, pasados 15 minutos del cierre, se retiran los que siguen en la calle. Al abrir la línea salen los buses en reserva hasta completar el mínimo.

RF-23. Simulación de la Operación

Mientras no existan validadores y contadores reales conectados, el sistema deberá simular la operación de la red para demostrar y probar las reglas del enunciado con datos realistas.

El sistema deberá:

- Avanzar los buses de estación en estación según la distancia de cada tramo, con mayor tiempo en las horas pico de lunes a viernes (6:00–8:30 y 17:00–19:30) y una variación por tramo de −10 % a +30 %.
- Simular en cada parada las bajadas por el molinete de salida y las subidas con pago en el molinete de entrada.
- Generar horas pico en las estaciones, que activan la alerta de refuerzo, y alertas de bus lleno.
- Permitir solo al administrador iniciar, pausar y ejecutar ciclos de la simulación, provocar una hora pico y reiniciar los aforos.

RF-24. Fallas y Mantenimiento de Buses

El sistema deberá registrar las fallas de los buses y su paso por el taller.

El sistema deberá permitir:

- Que el operador reporte la falla de un bus de las líneas que pasan por su estación, con su categoría, descripción y prioridad.
- Sacar de servicio automáticamente un bus con una falla de prioridad alta.
- Que el administrador pase la falla al taller, registre mantenimientos preventivos o correctivos y la marque como resuelta, con lo que el bus vuelve a estar disponible.

- Consultar el historial de mantenimiento en la ficha de cada bus.

Matriz de trazabilidad de requerimientos funcionales

La matriz relaciona cada requerimiento con el punto del enunciado que atiende, su estado y la evidencia en el sistema construido. La segunda tabla comprueba que los 18 puntos del enunciado están cubiertos por al menos un requerimiento.

Tabla 5.1. Matriz de trazabilidad de requerimientos funcionales.

RF	Punto del enunciado	Estado	Evidencia en el sistema
RF-01	11	Implementado	Catálogo de municipios; FK en líneas y estaciones; LineasOficialesTest
RF-02	1, 2, 5, 8, 12, 13	Implementado	linea_estacion; trg_linea_estacion_distancia; LineasOficialesTest
RF-03	2, 11, 14	Implementado	Admin/EstacionController; PantallasTest; E2E Estaciones
RF-04	3, 8, 10	Implementado	trg_accesos_min_dos_before_delete; trg_buses_molinetes; ReglasBaseDatosTest
RF-05	4, 5	Implementado	BusController; trg_buses_ruta_con_piloto; FuncionesTest
RF-06	6, 7	Implementado	sp_cambiar_parqueo_bus; UNIQUE en id_bus_asignado; ReglasBaseDatosTest
RF-07	9	Implementado	PersonalController; datos cifrados; PersonalTest
RF-08	10, 16	Implementado	trg_seguridad_min_uno; uq_operadores_estacion; OperadorTest
RF-09	14, 15, 16	Implementado (GPS y bahías en fase 2)	Operador/EstacionController; OperadorTest; E2E Operador
RF-10	14	Implementado	trg_estaciones_alerta_aforo; trg_transacciones_suben; CobroPasajeTest
RF-11	5, 14	Implementado	trg_buses_max_flota; despacharRefuerzo; SimulacionTest
RF-12	15	Implementado	Bus::minutosEsperaExtra(); ReglasModelosTest; SimulacionTest
RF-13	1	Implementado	viajes y visitas_viaje; reporte de recorridos y visitas
RF-14	18	Implementado	Servicio CobroPasaje; CobroPasajeTest
RF-15	14	Implementado	alertas_sistema; OperadorTest; SimulacionTest
RF-16	16	Implementado (piloto y seguridad en fase 2)	Middleware VerificarRol; AutenticacionTest; SesionesTest
RF-17	—	Implementado	BitacoraController; trg_bitacora_inmutable; BitacoraTest; E2E Bitácora
RF-18	16, 17	Implementado (se verifica al publicar)	HTTPS; sin conexión directa a la base
RF-19	15 (entregables)	Implementado	ReporteController; ReportesTest; E2E Reportes
RF-20	1, 12	Implementado	ImportadorKml; MonitoreoBuses; ImportarKmlTest; PantallasTest

RF	Punto del enunciado	Estado	Evidencia en el sistema
RF-21	18	Implementado	TarjetaController; TarjetasTest; E2E Tarjetas
RF-22	—	Implementado	horarios_linea; LineasOficialesTest; SimulacionTest
RF-23	1, 14, 15	Implementado	SimuladorOperacion; SimulacionTest; E2E Simulación
RF-24	16	Implementado	MantenimientoController; mantenimientos_bus; OperadorTest

Tabla 5.2. Cobertura de los puntos del enunciado.

Punto del enunciado	Requerimientos
1. Orden de estaciones y visitas de cada unidad	RF-02, RF-13, RF-20, RF-23
2. Estación en una o varias líneas	RF-02, RF-03
3. Accesos de cada estación	RF-04
4. Bus en una sola línea o en ninguna	RF-05
5. Flota de n a 2n buses por línea	RF-02, RF-05, RF-11
6. Parqueos de estaciones y buses	RF-06
7. Cambio de parqueo sin quedarse sin él	RF-06
8. Accesos de cada línea	RF-02, RF-04
9. Datos de los pilotos	RF-07
10. Un guardia por acceso como mínimo	RF-04, RF-08
11. Líneas y estaciones de una municipalidad	RF-01, RF-03
12. Distancia entre estaciones	RF-02, RF-20
13. Distancia de una línea	RF-02
14. Alerta al 150 % de la capacidad	RF-10, RF-11, RF-15
15. Espera de 5 minutos con menos del 25 %	RF-12
16. PC de trabajo y operador por estación	RF-08, RF-09, RF-16, RF-18
17. Forma de conectar las estaciones	RF-18, RNF-11
18. Métodos de pago, tarjeta de acceso	RF-14, RF-21

Requerimientos No Funcionales

Los requerimientos no funcionales establecen las condiciones de calidad y las características que deberá cumplir el Sistema de Control de Transmetro para garantizar que la solución sea segura, confiable, eficiente y adecuada para las necesidades operativas de las municipalidades.

Estos requerimientos no describen una función específica del sistema, sino las condiciones bajo las cuales dichas funciones deberán operar correctamente.

RNF-01. Seguridad

La aplicación deberá implementar mecanismos de autenticación y autorización que permitan proteger la información almacenada y procesada por el sistema.

Cada usuario deberá contar con credenciales propias para acceder a la plataforma. El sistema deberá identificar al usuario y validar que tenga autorización para realizar la operación solicitada.

El acceso a las funcionalidades deberá establecerse de acuerdo con el rol asignado al usuario. Por ejemplo, un administrador podrá gestionar líneas, estaciones, buses y usuarios, mientras que un operador de estación tendrá acceso únicamente a las funciones necesarias para realizar sus actividades.

Asimismo, las operaciones relacionadas con información sensible, como datos de pilotos, personal de seguridad, usuarios y transacciones de tarjetas de acceso, deberán estar restringidas a usuarios autorizados.

El objetivo es poder reducir el riesgo de accesos no autorizados, modificación indebida de información o exposición de datos del sistema.

El criterio de cumplimiento es que ningún usuario deberá poder acceder a módulos o información que no correspondan a los permisos establecidos para su rol.

Criterio medible: el 100 % de las rutas exige sesión y rol (verificado con AutenticacionTest, PantallasTest y las pruebas de extremo a extremo); contraseñas con bcrypt; protección CSRF en todos los formularios; bloqueo tras 5 intentos fallidos; datos sensibles del personal cifrados; Row Level Security activado en las 23 tablas del sistema.

RNF-02. Integridad de información

La base de datos deberá garantizar que la información relacionada con líneas, estaciones, buses, parqueos, pilotos y demás elementos del sistema mantenga relaciones consistentes.

La aplicación deberá validar las operaciones antes de almacenarlas, evitando registros duplicados, incompletos o contradictorios.

Deberán respetarse las reglas de negocio establecidas para el sistema. Por ejemplo:

- Un bus no podrá estar asignado a dos líneas simultáneamente.
- Un bus deberá mantener un parqueo asignado.
- Una estación deberá pertenecer a una municipalidad.
- Un acceso deberá pertenecer a una estación.
- La cantidad de buses asignados a una línea deberá respetar los límites establecidos.
- Cada acceso deberá contar con el control de seguridad correspondiente.

Las validaciones deberán realizarse tanto desde la aplicación como mediante las restricciones correspondientes de la base de datos cuando sea necesario.

El objetivo es garantizar que la información utilizada para la toma de decisiones sea confiable y coherente.

el sistema deberá rechazar cualquier operación que provoque una inconsistencia o viole una regla de negocio.

Criterio medible: 40 llaves foráneas, 55 restricciones CHECK, 17 triggers y 1 procedimiento almacenado; cada regla del enunciado tiene al menos una prueba automatizada que intenta violarla y comprueba el rechazo.

RNF-03. Disponibilidad

La plataforma deberá estar disponible durante el horario operativo del sistema de transporte, permitiendo que los operadores y administradores puedan consultar y registrar información cuando sea necesario.

Debido a que las estaciones dependerán de la plataforma para realizar diferentes actividades de control, deberán implementarse mecanismos destinados a reducir las interrupciones del servicio.

La solución deberá considerar procedimientos de recuperación ante fallos y mecanismos de respaldo que permitan restablecer la operación en caso de presentarse una interrupción.

La disponibilidad deberá ser especialmente importante para las funcionalidades relacionadas con el monitoreo de pasajeros y la generación de alertas, debido a que una interrupción podría retrasar la respuesta ante situaciones de saturación.

El objetivo es garantizar la continuidad del sistema durante la operación normal del Transmetro.

la plataforma deberá mantenerse operativa durante el horario definido para el servicio, registrando y documentando cualquier interrupción.

Criterio medible: disponibilidad mayor o igual al 99 % durante el horario de servicio (de 04:30 a 21:00, según los horarios oficiales cargados), medida con UptimeRobot. En producción se usan planes de pago, porque en el plan gratuito Render suspende la aplicación tras 15 minutos sin uso y Supabase pausa el proyecto tras 7 días de inactividad.

RNF-04. Rendimiento

Las consultas y operaciones realizadas dentro de la plataforma deberán responder en tiempos adecuados para no afectar las actividades de los operadores de las estaciones.

Las consultas frecuentes, como búsqueda de estaciones, líneas, buses, parqueos y alertas, deberán ejecutarse de manera eficiente.

Las operaciones relacionadas con situaciones críticas deberán recibir especial atención. Por ejemplo, cuando una estación alcance el nivel de pasajeros establecido para generar una alerta, el sistema deberá procesar la información oportunamente para que el operador pueda tomar las acciones correspondientes.

El rendimiento deberá evaluarse mediante pruebas que permitan determinar el comportamiento de la aplicación bajo diferentes cantidades de usuarios y registros.

Poder evitar que los tiempos de espera del sistema afecten las operaciones del personal.

las operaciones de uso frecuente deberán ejecutarse dentro de los tiempos establecidos durante las pruebas de rendimiento.

Criterio medible: el 95 % de las operaciones frecuentes (consultas, registro de pase, atención de alertas) responde en menos de 2 segundos con 50 usuarios simultáneos, medido con k6; la alerta

de refuerzo se crea en la misma transacción que actualiza el aforo; el panel de una ruta se arma con un número reducido de consultas (PantallaPrincipalTest).

RNF-05. Escalabilidad

La solución deberá diseñarse de manera que pueda crecer conforme aumenten las necesidades del sistema de transporte.

La plataforma deberá permitir incorporar nuevas:

- Municipalidades.
- Líneas.
- Estaciones.
- Buses.
- Parqueos.
- Usuarios.
- Operadores.

El crecimiento de la información no deberá requerir una reconstrucción completa de la aplicación.

La arquitectura deberá permitir que posteriormente puedan incorporarse nuevas funcionalidades relacionadas con la operación del transporte sin afectar significativamente los módulos existentes.

Por ejemplo, si posteriormente se incorpora una nueva municipalidad al sistema, deberá ser posible registrarla y administrar sus líneas y estaciones utilizando la infraestructura existente. proteger la inversión realizada y permitir que la solución acompañe el crecimiento futuro del Transmetro.

el sistema deberá permitir agregar nuevos recursos sin modificar de manera significativa la estructura principal de la aplicación.

Criterio medible: agregar una línea, estación o municipalidad no requiere cambios de código; la red actual de 8 líneas, 121 estaciones, 173 buses y 545 empleados se cargó solo con datos.

RNF-06. Usabilidad

La interfaz deberá ser clara, intuitiva y fácil de utilizar por operadores y personal administrativo.

Las funciones deberán organizarse de manera lógica, utilizando menús, botones, formularios y mensajes claramente identificados.

Las operaciones frecuentes deberán requerir la menor cantidad de pasos posibles para evitar errores durante las actividades diarias.

Cuando el usuario realice una operación incorrecta, el sistema deberá mostrar un mensaje comprensible que indique el problema y, cuando sea posible, la forma de solucionarlo.

Por ejemplo, si un operador intenta asignar un bus que ya pertenece a otra línea, el sistema deberá mostrar un mensaje explicando que la operación no puede realizarse debido a que el bus ya se encuentra asignado.

El objetivo es facilitar la adopción del sistema y reducir errores provocados por una interfaz complicada.
un operador previamente capacitado deberá poder realizar las operaciones correspondientes a su puesto sin requerir conocimientos técnicos avanzados.
Criterio medible: un operador capacitado pide otro bus o atiende una alerta con un clic desde su pantalla; los errores se muestran con mensajes en español que indican qué hacer; puntaje de usabilidad SUS de 70 o más con los operadores.

RNF-07. Auditoría

El sistema deberá registrar las operaciones importantes realizadas por los usuarios para permitir conocer quién realizó una acción, cuándo la realizó y qué información modificó.
El registro de auditoría deberá contemplar, como mínimo:

- Usuario que realizó la operación.
- Fecha.
- Hora.
- Tipo de operación.
- Registro afectado.
- Acción realizada.

Deberán considerarse especialmente las operaciones relacionadas con:

- Asignación de buses.
- Cambio de parqueos.
- Modificación de líneas.
- Modificación de estaciones.
- Administración de usuarios.
- Modificación de información de pilotos.
- Gestión de personal de seguridad.
- Operaciones relacionadas con tarjetas de acceso.

La información de auditoría deberá estar disponible únicamente para usuarios que tengan autorización para consultarla.
El objetivo es proporcionar trazabilidad sobre las operaciones realizadas dentro de la plataforma. ante una modificación importante, el sistema deberá permitir identificar al usuario responsable y la fecha y hora en que se realizó.
Criterio medible: el 100 % de las operaciones de los módulos administrativos y del operador queda en bitacora_auditoria con usuario, fecha, datos anteriores y posteriores; la bitácora no puede modificarse ni borrarse (trg_bitacora_inmutable) y solo el administrador la consulta (BitacoraTest).

RNF-08. Respaldo

La información almacenada deberá contar con mecanismos de respaldo que permitan reducir el riesgo de pérdida de información.
Los respaldos deberán incluir información crítica relacionada con:

- Líneas.
- Estaciones.
- Buses.
- Parqueos.
- Pilotos.
- Usuarios.
- Personal de seguridad.
- Tarjetas, recargas y bitácora de auditoría.
- Alertas.
- Transacciones.

Se deberá establecer una política de respaldo que defina la frecuencia con la que se realizarán las copias y el procedimiento para recuperar la información.
Los respaldos deberán mantenerse en una ubicación diferente a la utilizada para la operación principal cuando la infraestructura disponible lo permita.
Además, deberán realizarse pruebas periódicas de restauración para comprobar que los respaldos pueden utilizarse correctamente.
minimizar el impacto de fallos técnicos, errores humanos o incidentes que puedan provocar pérdida de información. deberá existir un respaldo funcional y un procedimiento documentado para recuperar la información.
Criterio medible: respaldo diario automático (Supabase Pro, retención de 7 días) más un respaldo semanal con pg_dump guardado fuera de Supabase; una prueba de restauración al mes.

RNF-09. Mantenibilidad

La aplicación deberá desarrollarse bajo una arquitectura organizada y modular que facilite la corrección de errores, actualización y evolución del sistema.
El código deberá mantener una estructura clara y separada por responsabilidades, evitando que una modificación en un módulo afecte innecesariamente el funcionamiento de otros componentes.
La solución deberá contar con documentación suficiente para que un equipo técnico pueda comprender su funcionamiento y realizar mantenimiento posteriormente.
Las modificaciones deberán realizarse mediante procedimientos controlados que permitan identificar los cambios realizados y verificar que las funcionalidades existentes continúen funcionando correctamente.
reducir el tiempo y costo necesario para corregir errores y realizar actualizaciones. deberá ser posible modificar o actualizar un módulo sin provocar fallos en las funcionalidades independientes del sistema.
Criterio medible: estilo verificado con Laravel Pint y 240 pruebas automatizadas ejecutadas en GitHub Actions en cada cambio; complejidad ciclomática promedio menor a 5 por método en la lógica de la aplicación (actual: 2.29).

RNF-10. Compatibilidad

La aplicación web deberá ser accesible desde navegadores modernos utilizando las computadoras instaladas en las estaciones.

La interfaz deberá mantener un funcionamiento consistente independientemente del navegador compatible utilizado por los operadores.

Se deberán realizar pruebas de compatibilidad para verificar las principales funcionalidades del sistema, incluyendo:

- Inicio de sesión.
- Consulta de información.
- Registro de datos.
- Modificación de registros.
- Generación de reportes.
- Visualización de alertas.

La solución deberá evitar depender de configuraciones particulares del equipo que puedan impedir su utilización en otras estaciones.

garantizar que los operadores puedan utilizar la plataforma desde las computadoras destinadas para cada estación. las funcionalidades principales deberán operar correctamente en los navegadores definidos como compatibles para el proyecto.

Criterio medible: funcionamiento verificado en las dos últimas versiones de Google Chrome, Microsoft Edge y Mozilla Firefox, con resolución mínima de 1366 × 768.

RNF-11. Conectividad

La solución deberá contar con una infraestructura de comunicación que permita conectar las computadoras de las diferentes estaciones con los servicios centrales.

La arquitectura de conectividad deberá permitir que la información registrada en una estación pueda ser procesada y almacenada en los servicios centrales del sistema.

La comunicación deberá diseñarse considerando aspectos como:

- Disponibilidad de conexión.
- Seguridad de la comunicación.
- Capacidad de transmisión.
- Estabilidad.
- Recuperación ante interrupciones.

En caso de una pérdida temporal de conectividad, el sistema deberá informar al operador sobre el estado de la conexión y deberán definirse mecanismos para evitar, en la medida de lo posible, la pérdida de información.

La solución deberá contemplar una arquitectura que permita conectar las diferentes estaciones sin importar que se encuentren ubicadas en distintas zonas o municipalidades.

Este requerimiento adquiere especial importancia debido a que el planteamiento del proyecto establece que todavía no se ha determinado cuál será la mejor alternativa tecnológica para conectar las computadoras de las diferentes estaciones.

garantizar una comunicación confiable entre las estaciones y los servicios centrales de la plataforma. Las estaciones deberán poder comunicarse con los servicios centrales y transmitir información de manera segura y confiable durante la operación normal.
Decisión (punto 17): cada PC de estación se conecta por Internet mediante HTTPS (cifrado TLS) a la aplicación en Render, y solo la aplicación se conecta a la base de datos. No se requiere una red privada (WAN o VPN) entre estaciones. Si se pierde la conexión, la operación no se registra a medias: el operador recibe un mensaje y la repite al volver la conexión. Un modo sin conexión con almacenamiento local queda como mejora futura.

RNF-12. Protección de Datos Personales

El sistema maneja datos personales de los empleados (DPI, teléfono, correo, dirección e historial educativo) y de los titulares de tarjetas personalizadas (nombre y DPI). Estos datos deberán protegerse durante todo su ciclo de vida.

Para ello el sistema deberá:

- Guardar cifrados el contacto, la dirección y el historial educativo del personal, de modo que no puedan leerse directamente en la base de datos.
- Mostrar los datos personales solo a los usuarios con rol de administrador.
- Validar el formato del DPI (13 dígitos) y guardar únicamente los datos necesarios para la operación.
- Transmitir la información siempre por conexiones cifradas (HTTPS y SSL hacia la base de datos).

Criterio medible: el 100 % de los campos de contacto, dirección e historial educativo se almacena cifrado; ningún usuario con rol de operador puede consultar la ficha del personal ni de las tarjetas.

RNF-13. Portabilidad e Instalación

La solución deberá poder instalarse en un entorno nuevo, para desarrollo, pruebas o producción, sin modificar el código fuente.

Para ello el sistema deberá:

- Crear toda la estructura de la base de datos con las migraciones de Laravel o con el script de la sección 9.
- Cargar la red oficial y los datos iniciales con los seeders del proyecto.
- Leer la configuración (base de datos, horarios, tráfico de la simulación) de variables de entorno.
- Ejecutarse tanto en Windows (archivos iniciar.bat y probar.bat) como en Linux (servicio web con Docker en Render).

Criterio medible: un entorno nuevo queda funcionando en menos de 30 minutos siguiendo los pasos documentados, y las mismas 240 pruebas automatizadas se aprueban en el equipo de desarrollo y en GitHub Actions.

Relación con el modelo de calidad ISO/IEC 25010

Para verificar que no falte ninguna característica de calidad, los requerimientos no funcionales se relacionaron con las ocho características del modelo ISO/IEC 25010:

Tabla 5.3. Requerimientos no funcionales por característica de ISO/IEC 25010.

Característica (ISO/IEC 25010)	Requerimientos no funcionales
Adecuación funcional	RNF-02 (integridad) y la matriz de trazabilidad de los RF
Eficiencia de desempeño	RNF-04 (rendimiento), RNF-05 (escalabilidad)
Compatibilidad	RNF-10 (navegadores), RNF-11 (conectividad)
Usabilidad	RNF-06
Fiabilidad	RNF-03 (disponibilidad), RNF-08 (respaldo)
Seguridad	RNF-01, RNF-07 (auditoría), RNF-12 (datos personales)
Mantenibilidad	RNF-09
Portabilidad	RNF-13

6. Estudios de Factibilidad

El estudio de factibilidad evalúa si la propuesta puede desarrollarse, operarse y mantenerse con los recursos disponibles y dentro del marco legal guatemalteco. Se analizan cinco dimensiones: técnica, operativa, económica, legal y ambiental. Los montos se expresan en quetzales; los servicios cobrados en dólares se convierten al tipo de cambio de referencia del Banco de Guatemala del 28 de septiembre de 2026 (Q7.63 por US$1).

6.1. Factibilidad técnica

La factibilidad técnica determina si existen la tecnología, la infraestructura y el conocimiento necesarios para construir y poner en marcha el sistema.

Recurso	Requerido	Disponibilidad	Evaluación
Lenguaje y framework	PHP 8.4 y Laravel 13	Software libre (licencia MIT); el equipo ya lo domina	Factible
Base de datos	PostgreSQL con triggers, procedimientos y RLS	PostgreSQL 16 en desarrollo y Supabase en la nube	Factible
Alojamiento	Servidor web con HTTPS	Render (Web Service con Docker)	Factible
Mapas	Mapa base y trazo de rutas	OpenStreetMap y Leaflet (abiertos); KML oficiales de las líneas	Factible
Control de versiones y pruebas	Repositorio e integración continua	GitHub y GitHub Actions con 240 pruebas automatizadas	Factible
PC de trabajo por estación (punto 16)	Navegador moderno, 4 GB de RAM, Internet	Equipo de oficina estándar disponible en el mercado local	Factible (requiere compra)
Conectividad (punto 17)	Enlace a Internet en cada estación	Proveedores locales de Internet fijo y 4G	Factible
Molinetes	Lector de tarjeta conectado al sistema	En esta versión el pase se registra desde el panel del operador; la conexión directa con el molinete requiere la API del fabricante	Parcial (fase 2)

Respuesta al punto 17 (cómo conectar las máquinas): se recomienda una arquitectura centralizada en la nube. Cada PC de estación se conecta por Internet mediante HTTPS (cifrado TLS) a la aplicación en Render, y solo la aplicación se conecta a la base de datos en Supabase. Así no es necesario montar una red privada (WAN o VPN) entre estaciones, se reduce el costo de infraestructura y una estación nueva funciona con un navegador y conexión a Internet. Como respaldo, cada estación puede contar con un módem 4G.

Prototipo funcional: el sistema ya cuenta con las 8 líneas oficiales (1, 2, 6, 7, 12, 13, 18 y Ruta 5), 121 estaciones, 173 buses, 17 triggers con las reglas de negocio, un procedimiento almacenado, 240 pruebas automatizadas y 190 verificaciones de extremo a extremo aprobadas, lo que demuestra que la solución es técnicamente viable.

Conclusión: el proyecto es técnicamente factible.

6.2. Factibilidad operativa

La factibilidad operativa evalúa si el personal del Transmetro podrá usar el sistema y si este se integra a su forma de trabajo.

- Usuarios: administradores municipales y operadores de estación. Los operadores ya contarán con una PC según el punto 16; el sistema solo requiere conocimientos básicos de navegador.
- Interfaz en español, con menús por rol, mensajes de error comprensibles (por ejemplo, al intentar exceder la flota de una línea) y un panel de estación en una sola pantalla.
- Las reglas de negocio se aplican automáticamente en la base de datos, lo que reduce los controles manuales.
- Capacitación de 24 horas para administradores y operadores, incluida en la propuesta monetaria, con manual de usuario.
- Resistencia al cambio: se mitiga con un piloto en las Líneas 1 y 2 (14 estaciones) antes de extender el sistema a toda la red.
- Soporte: el código está documentado, versionado en GitHub y cubierto por pruebas, lo que facilita el mantenimiento.

Riesgo operativo	Probabilidad	Impacto	Mitigación
Caída de Internet en una estación	Media	Alto	Enlace 4G de respaldo; la operación se repite al volver la conexión
Datos ingresados con errores	Media	Medio	Validaciones en formularios y restricciones en la base de datos
Rotación de operadores	Media	Bajo	Manual de usuario y capacitación corta
Suspensión por inactividad en planes gratuitos	Alta	Alto	Usar en producción Render Starter y Supabase Pro

Conclusión: el proyecto es operativamente factible.

6.3. Factibilidad económica

La factibilidad económica compara la inversión y los costos de operación con los beneficios esperados. El detalle de los costos está en la sección 7 (Propuesta Monetaria).

6.3.1. Costos

Concepto	Monto	Tipo
Desarrollo del software (puntos de función)	Q21,670.25	Inversión única
Integración: análisis, pruebas QA y capacitación	Q13,600.00	Inversión única
IVA (12 %) sobre la inversión	Q4,232.43	Inversión única
Inversión total del software	**Q39,502.68**	
Nube: Render Starter + Supabase Pro + dominio	Q3,044.37	Anual
Mantenimiento correctivo y evolutivo (10 % del desarrollo)	Q2,167.03	Anual
Costo anual de operación	**Q5,211.40**	

Las licencias de software tienen costo cero porque todo el stack es de código abierto. El equipo de las estaciones (PC, UPS e Internet) es una inversión de infraestructura que la municipalidad necesita de todas formas por el punto 16, por lo que se presenta aparte en la sección 7.8.

6.3.2. Beneficios estimados

Los beneficios se calcularon con supuestos conservadores que deben validarse con la municipalidad:

Beneficio	Supuesto	Monto anual
Menos horas de consolidación manual de información (estaciones, buses, personal y reportes)	3 personas × 40 h/mes × Q40/h × 12 meses	Q57,600.00
Menos papel, impresión y formularios de control	Q500 al mes	Q6,000.00
Total de beneficios cuantificables		**Q63,600.00**

Además hay beneficios no cuantificados: alertas de refuerzo en el momento en que se superan los límites, control de la flota mínima y máxima por línea, trazabilidad de pagos y cambios, y reportes inmediatos para la toma de decisiones.

6.3.3. Indicadores financieros (horizonte de 5 años, tasa de descuento del 10 %)

Indicador	Valor	Interpretación
Flujo neto anual	Q58,388.60	Beneficios menos costo anual de operación
Valor actual neto (VAN)	Q181,836.07	Positivo: la inversión se recupera con ganancia
Relación beneficio/costo	4.07	Mayor que 1: cada quetzal invertido devuelve más de un quetzal
Período de recuperación	0.7 años	Dentro del horizonte de evaluación

Conclusión: con los supuestos indicados, el proyecto es económicamente factible. Si los beneficios reales fueran menores, el punto de equilibrio se alcanza con un beneficio anual de Q15,632.10.

6.4. Factibilidad legal

Se revisó el marco legal guatemalteco aplicable al manejo de la información, a la contratación del servicio y al uso de software y datos de terceros.

Norma	Aplicación al proyecto
Constitución Política de la República de Guatemala, artículos 24 y 31	Protege los documentos y registros de las personas y su derecho a conocer la información que de ellas consta en archivos del Estado. Los datos del personal y de los titulares de tarjetas se guardan con acceso restringido.
Ley de Acceso a la Información Pública, Decreto 57-2008	La municipalidad es sujeto obligado. Los datos personales sensibles (DPI, dirección, antecedentes) son información confidencial y solo los consulta el personal autorizado; los reportes públicos no incluyen datos personales.
Ley para el Reconocimiento de las Comunicaciones y Firmas Electrónicas, Decreto 47-2008	Da validez a los registros electrónicos, como transacciones de pago y bitácora de auditoría.
Código Municipal, Decreto 12-2002	El transporte público urbano es competencia de la municipalidad, que es la entidad contratante y dueña de la información.
Ley de Tránsito, Decreto 132-96	Exige licencia de conducir vigente a los pilotos; el sistema registra número y tipo de licencia.
Ley de Derecho de Autor y Derechos Conexos, Decreto 33-98	Protege el software como obra. El contrato debe definir a quién pertenece el código fuente.
Ley de Contrataciones del Estado, Decreto 57-92	Por el monto de la propuesta (Q42,912), la contratación corresponde a la modalidad de compra directa (más de Q25,000 y hasta Q90,000), publicada en Guatecompras.
Licencias del software utilizado	Laravel, Tailwind y Pest (MIT), PHP (PHP License), PostgreSQL (PostgreSQL License), Leaflet (BSD-2). Todas permiten uso sin costo. Los datos de OpenStreetMap (ODbL) exigen mostrar el crédito “© OpenStreetMap”, que ya aparece en el mapa.
Servicios en la nube	Los datos se alojan en servidores de Supabase fuera de Guatemala; el contrato con la municipalidad debe autorizarlo y exigir cifrado y respaldos.

Conclusión: el proyecto es legalmente factible, siempre que el contrato defina la propiedad del código, se publique un aviso de privacidad para los titulares de tarjetas y se mantenga el crédito de OpenStreetMap.

6.5. Factibilidad ambiental

Se evaluó el impacto ambiental del desarrollo y de la operación del sistema, en el marco de la Ley de Protección y Mejoramiento del Medio Ambiente (Decreto 68-86).

Aspecto	Impacto	Medida
Uso de papel	Positivo	Reportes, formularios y bitácoras digitales; los reportes se imprimen solo cuando es necesario.
Manejo de efectivo	Positivo	Tarjeta Ciudadana y pago sin contacto: menos traslado de valores.
Emisiones de los buses	Positivo	Refuerzos solo cuando la estación supera el 150 % y espera extra cuando el bus va casi vacío: viajes más llenos y menos emisiones por pasajero.
Energía de servidores	Bajo	Nube compartida en centros de datos eficientes; no se instala un servidor propio con aire acondicionado.
Energía de las PC de estación	Moderado	Equipos con certificación de eficiencia energética y suspensión fuera del horario de servicio.
Residuos electrónicos	Moderado	Al final de su vida útil, PC, UPS y baterías se entregan a gestores autorizados por el MARN.

Conclusión: el impacto ambiental es bajo y en su mayoría positivo; el proyecto es ambientalmente factible.

6.6. Resumen del estudio de factibilidad

Dimensión	Resultado	Condición principal
Técnica	Factible	Prototipo funcional con pruebas; integración con molinetes en fase 2
Operativa	Factible	Capacitación y piloto en las Líneas 1 y 2
Económica	Factible	VAN de Q181,836 con los supuestos de beneficio indicados
Legal	Factible	Contrato con propiedad del código y aviso de privacidad
Ambiental	Factible	Manejo responsable de los residuos electrónicos

7. Propuesta Monetaria

La propuesta monetaria se calculó con el método de Puntos de Función (FPA-IFPUG), igual que la estimación original, pero ajustada al alcance real del enunciado y del sistema construido. Primero se presenta la revisión de la estimación original y después la estimación corregida.

7.1. Revisión de la estimación original

Las fórmulas de la estimación original están bien aplicadas (PFSA 288 → PFA 362.88 → 2,177.28 H/H → 6.48 meses → Q79,010.00; total Q117,120.00). Las correcciones se deben al contenido:

No.	Hallazgo en la estimación original	Corrección
1	Ítem 14: inyección de bus al llegar al 50 % del aforo	La alerta se genera al 150 % del aforo (50 % más de su capacidad, punto 14)
2	Ítem 15: buses en circulación ≤ paradas de la ruta	La regla es mínimo n y máximo 2n buses por línea (punto 5)
3	Ítems 17 a 21: bahías, notificación push y estacionamiento dinámico por orden de llegada	Contradicen los puntos 6 y 7: cada bus tiene un parqueo fijo que puede cambiar, pero nunca quedarse sin él
4	Ítem 31: cobro en efectivo con monedas de Q1.00	Se elimina: el molinete solo acepta Tarjeta Ciudadana o tarjeta bancaria sin contacto
5	Ítems 34 y 35: autenticación en Vercel y modo sin conexión	La arquitectura es Laravel en Render con PostgreSQL en Supabase; el modo sin conexión queda como mejora futura
6	Ítems 5 y 12: pantalla del piloto y GPS	Se pasan a la fase 2 (no forman parte del enunciado)
7	Faltan funciones construidas	Se agregan Tarjeta Ciudadana y recargas, reportes, mapa e importación KML, horarios y bitácora
8	ILF = 12; EIF = GPS, pasarela y torniquetes	ILF = 14 grupos lógicos de datos; EIF = mapas KML, OpenStreetMap, OSRM y molinete o pasarela
9	“Otros costos” (Q1,250) se sumaron una sola vez	Son gastos mensuales (transporte, alimentación y conectividad); se multiplican por los meses del proyecto
10	“Horas promedio por PF: 6 por día”	Se expresa en horas por punto de función. Se usa 1.2 horas por PF, la productividad medida al construir el sistema con Laravel, que resuelve gran parte del trabajo (ORM, validación, autenticación y migraciones)
11	Hosting en Vercel (Q2,160/año) y Render (Q4,200/año)	Se elimina Vercel; Render Starter US$7/mes y Supabase Pro US$25/mes
12	Hardware solo para 4 accesos, 4 estaciones y 4 buses	El punto 16 exige una PC por estación; se propone un piloto de 14 estaciones (Líneas 1 y 2) como inversión municipal
13	Suma de factores de ajuste = 61	Baja a 57: rendimiento, configuración, tasa de transacción y procesamiento complejo bajan un grado porque no hay GPS ni control de bahías en tiempo real

7.2. Clasificación corregida de las funciones

Estado: I = implementada en el prototipo; P = pendiente para el siguiente sprint.

No.	Módulo	Función	Tipo	Estado
1	1. Infraestructura	Registrar y actualizar municipalidades	EI	I
2	1. Infraestructura	Registrar y actualizar líneas (código, nombre, tipo de servicio, tarifa, color)	EI	I
3	1. Infraestructura	Asignar estaciones a una línea en orden (distancia calculada por trigger)	EI	I
4	1. Infraestructura	Registrar y actualizar estaciones (dirección, aforo, coordenadas)	EI	I
5	1. Infraestructura	Registrar accesos (mínimo 2) con su molinete	EI	I
6	1. Infraestructura	Registrar parqueos por estación	EI	I
7	1. Infraestructura	Registrar horarios de servicio por línea	EI	I
8	1. Infraestructura	Editar el trazo de la línea e importar mapa KML/KMZ oficial	EI	I
9	2. Flota	Registrar buses y asignarlos a una línea (mínimo n, máximo 2n)	EI	I
10	2. Flota	Cambiar el parqueo de un bus sin dejarlo sin parqueo (sp_cambiar_parqueo_bus)	EI	I
11	2. Flota	Registrar viajes y visitas a estaciones (llegada, salida, ocupación)	EI	P
12	3. Personal	Registrar pilotos (historial educativo, residencia, comunicación)	EI	I
13	3. Personal	Registrar seguridad y asignarlo a un acceso (mínimo 1 por acceso)	EI	I
14	3. Personal	Registrar operadores y asignarlos a la PC de su estación	EI	I
15	4. Tarjeta y pagos	Emitir Tarjeta Ciudadana (Q20.00 con 5 viajes gratis)	EI	I
16	4. Tarjeta y pagos	Recargar tarjeta (Q5.00 a Q200.00)	EI	I
17	4. Tarjeta y pagos	Bloquear o reactivar una tarjeta	EI	I
18	4. Tarjeta y pagos	Registrar pase en molinete (Tarjeta Ciudadana o tarjeta bancaria)	EI	I
19	5. Operación de estación	Actualizar el aforo actual de la estación	EI	I
20	5. Operación de estación	Atender alerta de refuerzo (solicitar bus)	EI	I
21	6. Seguridad	Iniciar y cerrar sesión con control de rol	EI	I
22	5. Operación de estación	Alerta automática de refuerzo al superar 150 % del aforo	EO	I
23	2. Flota	Cálculo de espera extra de 5 min cuando la ocupación es menor a 25 %	EO	I
24	7. Reportes	Reporte de estaciones (líneas, accesos, parqueos, aforo)	EO	I
25	7. Reportes	Reporte de líneas (estaciones, distancia, flota mínima y máxima)	EO	I
26	7. Reportes	Reporte de buses asignados por línea	EO	I
27	7. Reportes	Reporte de recorridos (viajes)	EO	I
28	7. Reportes	Reporte de visitas y ocupación por estación	EO	I
29	7. Reportes	Exportación CSV e impresión de reportes	EO	I
30	7. Reportes	Reporte de alertas generadas y atendidas	EO	P
31	7. Reportes	Reporte de recaudación por método de pago	EO	P
32	7. Reportes	Reporte de personal asignado	EO	P
33	1. Infraestructura	Mapa de líneas y estaciones (OpenStreetMap)	EQ	I
34	1. Infraestructura	Consultar línea: estaciones en orden, accesos y horarios	EQ	I
35	1. Infraestructura	Consultar estación: accesos, molinetes, guardias y parqueos	EQ	I

No.	Módulo	Función	Tipo	Estado
36	2. Flota	Consultar buses y parqueos	EQ	I
37	3. Personal	Consultar expedientes del personal	EQ	I
38	4. Tarjeta y pagos	Consultar tarjeta (saldo, viajes gratis, recargas)	EQ	I
39	5. Operación de estación	Panel del operador (aforo, alertas y buses de su estación)	EQ	I
40	6. Seguridad	Panel del administrador con indicadores	EQ	I
41	6. Seguridad	Consultar bitácora de auditoría	EQ	P

Resumen	Cantidad
Entradas externas (EI)	21
Salidas externas (EO)	11
Consultas externas (EQ)	9
Total de transacciones	**41**
Implementadas / pendientes	36 / 5

7.3. Archivos lógicos

Archivos lógicos internos (ILF)	Archivos de interfaz externa (EIF)
1. Municipio	1. Mapas KML/KMZ oficiales de las líneas
2. Línea (con horarios y orden de estaciones)	2. Teselas de OpenStreetMap
3. Estación	3. Servicio de ajuste a calles OSRM
4. Acceso y molinete	4. Lector de molinete / pasarela bancaria (integración futura)
5. Parqueo	
6. Bus	
7. Empleado (piloto, seguridad, operador)	
8. Usuario y rol	
9. Tarjeta	
10. Recarga	
11. Transacción de pago	
12. Alerta del sistema	
13. Viaje y visitas	
14. Bitácora de auditoría	

7.4. Cálculo de puntos de función

Componente	Cantidad	Peso (media)	Subtotal
Entradas (EI)	21	4	84
Salidas (EO)	11	5	55
Consultas (EQ)	9	4	36
Archivos internos (ILF)	14	10	140
Archivos externos (EIF)	4	7	28
Puntos de función sin ajustar (PFSA)			**343**

No.	Factor de ajuste	Grado (0-5)
1	Comunicación de datos	5
2	Proceso distribuido	4
3	Objetivos de rendimiento	4
4	Configuración del equipamiento	3
5	Tasa de transacción	4
6	Entrada de datos en línea	5
7	Interfaz con el usuario	4
8	Actualizaciones en línea	5
9	Procesamiento complejo	4
10	Reusabilidad del código	4
11	Conversión e instalación	3
12	Facilidad de operación	4
13	Instalación múltiple	4
14	Facilidad de cambios	4
	Suma de factores	**57**

Resultado	Valor	Fórmula
Factor de ajuste (FA)	1.22	0.65 + 0.01 × suma de factores
Puntos de función ajustados (PFA)	418.46	PFSA × FA
Horas hombre (H/H)	502.15	PFA × 1.2 horas por PF (productividad con Laravel)
Horas del desarrollador	502.15	H/H ÷ 1 desarrollador
Tiempo (meses)	2.99	Horas ÷ (8 h × 21 días)
Líneas de código estimadas	20,923	PFA × 50 LOC (lenguaje de 4.ª generación)
Sueldos	Q17,934.00	Meses × Q6,000 × 1 desarrollador
Otros costos	Q3,736.25	Meses × Q1,250 (transporte, alimentación y conectividad)
Costo de desarrollo	**Q21,670.25**	Sueldos + otros costos
Precio por hora	Q43.15	Costo ÷ H/H

7.5. Costos del proceso de integración

Actividad	Horas	Costo por hora	Total
Análisis de la operación y arquitectura (Render + Supabase)	55	Q100.00	Q5,500.00
Pruebas de software QA (funcionales, carga, seguridad y SonarQube Cloud)	60	Q85.00	Q5,100.00
Capacitación de administradores y operadores	24	Q125.00	Q3,000.00
Total de integración			**Q13,600.00**

7.6. Costos de operación en la nube (anual)

Servicio	US$ por año	Q por año
Render – Web Service Starter (512 MB, siempre activo)	84.00	Q640.92
Supabase – plan Pro (respaldos diarios, sin pausa)	300.00	Q2,289.00
Dominio .com y certificado TLS (incluido en Render)	15.00	Q114.45
Total anual	**399.00**	**Q3,044.37**

Tipo de cambio: Q7.63 por US$1 (Banguat, 28/09/2026). Licencias de IDE (VS Code), gestor de base de datos (PostgreSQL), framework y análisis de calidad (SonarQube Cloud, plan gratuito para repositorios privados de hasta 50,000 líneas de código): Q0.00. Los planes gratuitos de Render y Supabase sirven para desarrollo, pero no para producción, porque suspenden el servicio por inactividad y no incluyen respaldos.

7.7. Resumen de la propuesta

Concepto	Monto
Desarrollo del software	Q21,670.25
Proceso de integración (análisis, pruebas y capacitación)	Q13,600.00
Operación en la nube, primer año	Q3,044.37
Subtotal	**Q38,314.62**
IVA (12 %)	Q4,597.75
Total de la propuesta	**Q42,912.37**
Costo recurrente desde el segundo año (nube + mantenimiento, sin IVA)	Q5,211.40

El sistema completo se desarrollará en 3 meses con 1 desarrollador, de agosto a octubre de 2026, de acuerdo con el calendario de entregas del capítulo 4. Como referencia, las funciones ya implementadas en el prototipo suman 320 PFSA (390.40 PFA), el 93.3 % del total estimado.

7.8. Inversión en equipo de estaciones (a cargo de la municipalidad)

El punto 16 requiere una PC de trabajo en cada estación. Este equipo no forma parte del precio del software; se presenta como referencia para la municipalidad. Los precios son referenciales y deben cotizarse.

Elemento	Precio unitario	Piloto Líneas 1 y 2 (14 estaciones)	Red completa (121 estaciones)
PC de trabajo	Q4,500.00	Q63,000.00	Q544,500.00
UPS	Q750.00	Q10,500.00	Q90,750.00
Inversión en equipo		**Q73,500.00**	**Q635,250.00**
Internet por estación (anual)	Q3,600.00	Q50,400.00	Q435,600.00

Fase 2 opcional, por cotizar: tablets para la consola del piloto (referencia Q1,200 por bus) e integración directa con los lectores de los molinetes, que depende de la API del fabricante.

7.9. Forma de pago

Hito	Porcentaje	Monto
Firma del contrato	30 %	Q12,873.71
Entrega del piloto en las Líneas 1 y 2	40 %	Q17,164.95
Carta de aceptación del proyecto	30 %	Q12,873.71

8. Modelo Entidad Relación

El modelo entidad relación se generó directamente desde la base de datos final, después de aplicar todas las migraciones del proyecto, por lo que coincide con el script de la sección 9 y con la aplicación en funcionamiento. Reemplaza la versión anterior elaborada en DBeaver, a la que le faltaban las tablas y columnas agregadas durante la programación.

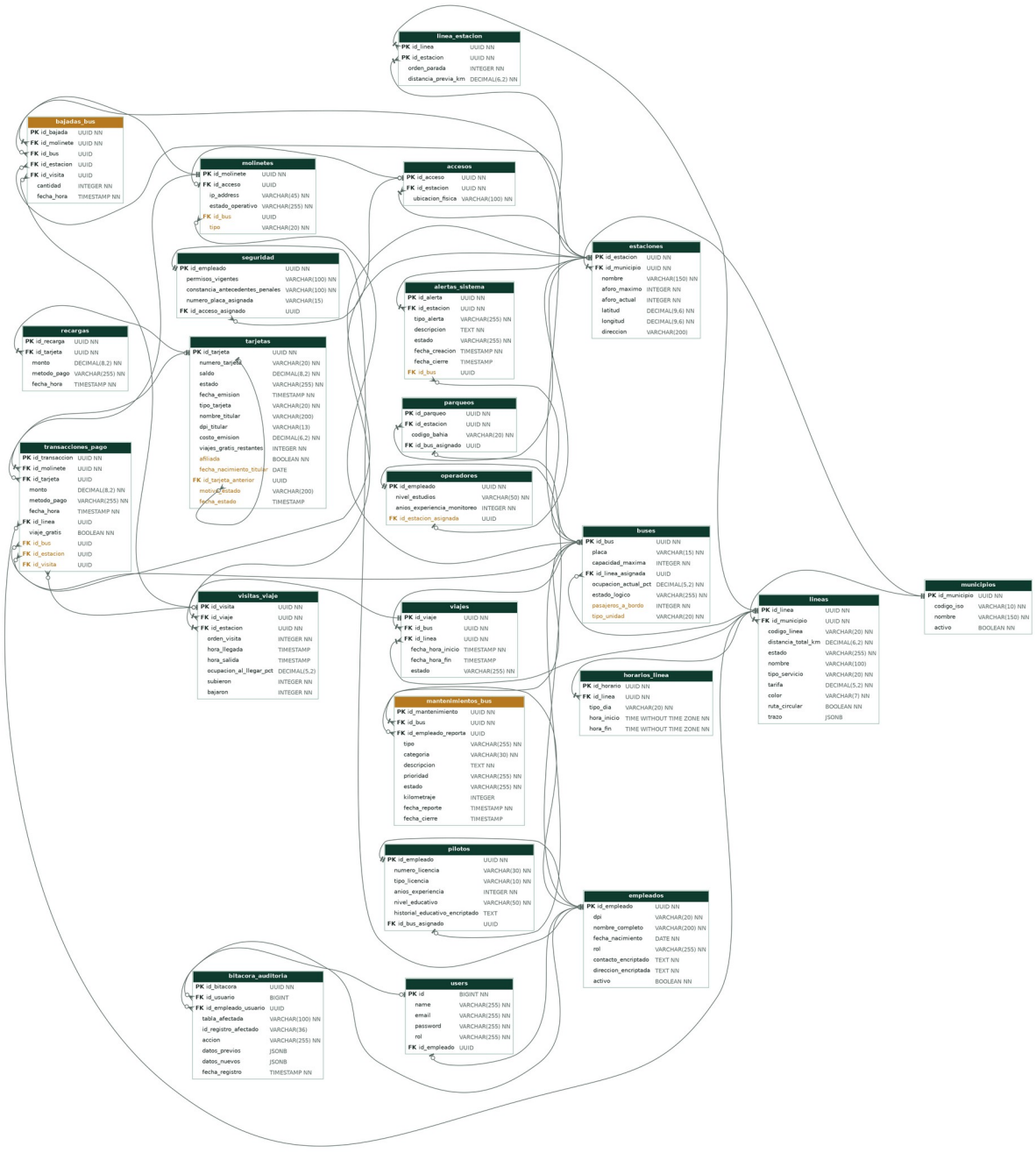


Figura 2. Modelo entidad relación del Sistema de Control Transmetro (PostgreSQL).

Convenciones: PK = llave primaria, FK = llave foránea, NN = no nulo. Las tablas con encabezado dorado (bajadas_bus y mantenimientos_bus) y los atributos en dorado son los que se agregaron en la última etapa del desarrollo. Las relaciones usan notación de pata de gallo: la

línea simple indica “uno” y la pata de gallo indica “muchos”; el círculo indica que la relación es opcional.

8.1. Cambios respecto de la versión anterior

Tabla	Cambio	Motivo
horarios_linea	Tabla nueva	Horario oficial por tipo de día (RF-22)
lineas	nombre, tipo_servicio, tarifa, color, ruta_circular, trazo	Datos oficiales de las líneas, tarifa del pase y trazo del mapa (RF-02, RF-20)
estaciones	direccion; coordenadas en latitud y longitud	Ubicación en el mapa (RF-03)
tarjetas	tipo_tarjeta, nombre_titular, dpi_titular, costo_emision, viajes_gratis_restantes	Tarjeta Ciudadana: Q20 con 5 viajes gratis (RF-21)
transacciones_pago	id_linea, viaje_gratis, id_bus, id_estacion, id_visita	Cobro en el molinete de entrada del bus, por parada (RF-14)
molinetes	tipo, id_bus	Molinetes de entrada y salida de cada bus (RF-04, RF-10)
buses	tipo_unidad, pasajeros_a_bordo	Tipo de unidad por línea y conteo de pasajeros (RF-05, RF-10)
operadores	id_estacion_asignada (antes id_bus_asignado)	Un operador por estación (punto 16, RF-08)
alertas_sistema	id_bus; tipo bus_lleno	Alerta de bus lleno (RF-11, RF-15)
tarjetas	afiliada, motivo_estado, fecha_estado, id_tarjeta_anterior	Bloqueo con motivo, baja y reposición (RF-21)
bajadas_bus	Tabla nueva	Personas que bajan por el molinete de salida (RF-10)
mantenimientos_bus	Tabla nueva	Fallas y mantenimientos de los buses (RF-24)

8.2. Relaciones y reglas del enunciado

Relación	Cardinalidad	Regla (punto del enunciado)
municipios – lineas / estaciones	1 : N	Cada línea y estación pertenece a una municipalidad (11)
lineas – estaciones (linea_estacion)	N : M	Orden de las estaciones y distancia entre ellas (1, 2, 12, 13)
estaciones – accesos	1 : N (mínimo 2)	Varios accesos por estación; un acceso, una estación (3)
accesos – molinetes	1 : 1	Un molinete propio por acceso
accesos – seguridad	1 : N (mínimo 1)	Al menos un guardia por acceso (10)
lineas – buses	1 : N (de n a 2n)	Un bus en una sola línea o en ninguna (4, 5)
estaciones – parqueos – buses	1 : N y 1 : 1	Cada bus tiene un parqueo y puede cambiarlo (6, 7)
empleados – pilotos / seguridad / operadores	1 : 1	Especialización del personal (9, 10, 16)
buses – viajes – visitas_viaje	1 : N	Recorrido real y visitas a estaciones (1, 15)

Relación	Cardinalidad	Regla (punto del enunciado)
estaciones – alertas_sistema	1 : N	Alerta de refuerzo al 150 % del aforo (14)
tarjetas – recargas / transacciones_pago	1 : N	Métodos de pago y tarjeta de acceso (18)
buses – molinetes	1 : 2	Molinete de entrada (cobro) y de salida (bajadas) de cada bus (14, 18)
molinetes – transacciones_pago / bajadas_bus	1 : N	Pase y bajadas registrados en el molinete (18)
operadores – estaciones	1 : 1	Un operador a cargo de la PC de cada estación (16)
buses – mantenimientos_bus	1 : N	Fallas reportadas y mantenimientos del bus
users – bitacora_auditoria	1 : N	Trazabilidad de los cambios

9. Script de la estructura de base de datos

El script crea todas las estructuras de la base de datos en PostgreSQL 15 o superior. Se generó desde la base de datos resultante de las migraciones de Laravel y se validó ejecutándolo en una base vacía, sin errores. Equivale a ejecutar php artisan migrate.

Objeto	Cantidad	Detalle
Tablas del dominio	22	Más la tabla users de Laravel, a la que se agregan rol e id_empleado
Llaves foráneas	40	Con ON UPDATE CASCADE y la acción de borrado adecuada para cada caso
Restricciones CHECK	55	Valores permitidos, rangos (recargas de Q5 a Q200, ocupación de 0 a 100 %) y reglas de pago
Triggers	17	Mínimo 2 accesos, mínimo 1 guardia, máximo 2n buses, alerta al 150 %, distancia total de la línea, molinetes del bus, conteo de subidas y bajadas, ocupación del bus, bus sin piloto, tarjeta activa, estado final de la tarjeta y bitácora inalterable
Procedimiento almacenado	1	sp_cambiar_parqueo_bus: cambio de parqueo sin dejar al bus sin parqueo
Row Level Security	23 tablas	Bloquea el acceso directo desde la API pública de Supabase

Forma de ejecución: en DBeaver, abrir el archivo Script_Creacion_Estructuras_BD_PostgreSQL.sql sobre la conexión de Supabase y usar “Ejecutar script” (Alt + X); o desde la terminal: psql "cadena de conexión" -f Script_Creacion_Estructuras_BD_PostgreSQL.sql. La tabla users debe existir antes (se crea con la migración inicial de Laravel).

9.1. Script de creación de estructuras (DDL)

```
-- ======================================================================
-- PROYECTO ACS - SISTEMA DE CONTROL TRANSMETRO
-- Script de creacion de estructuras de base de datos (DDL)
-- Motor: PostgreSQL 15+ (Supabase)
-- Autora: Wendy Celeste Nathaly Estrada Gonzalez
-- Fecha: 2026-10-13
-- Generado desde la base de datos resultante de las migraciones de Laravel.
-- Requiere la tabla users de Laravel (se crea con php artisan migrate).
-- ======================================================================

-- ----------------------------------------------------------------------
-- 1. MUNICIPIOS
--    Punto 11: cada linea y estacion pertenece a una municipalidad.
-- ----------------------------------------------------------------------
CREATE TABLE municipios (
    id_municipio  UUID NOT NULL DEFAULT gen_random_uuid(),
    codigo_iso    VARCHAR(10) NOT NULL,
    nombre        VARCHAR(150) NOT NULL,
    activo        BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT municipios_pkey PRIMARY KEY (id_municipio),
    CONSTRAINT uq_municipios_codigo_iso UNIQUE (codigo_iso)
);

-- ----------------------------------------------------------------------
-- 2. LINEAS
--    Puntos 11 y 13. distancia_total_km la calcula trg_linea_estacion_distancia. Tarifa, color, ruta circular y
trazo (JSONB) del mapa oficial.
-- ----------------------------------------------------------------------
CREATE TABLE lineas (
    id_linea            UUID NOT NULL DEFAULT gen_random_uuid(),
    id_municipio        UUID NOT NULL,
    codigo_linea        VARCHAR(20) NOT NULL,
    distancia_total_km  DECIMAL(6,2) NOT NULL DEFAULT '0'::numeric,
    estado              VARCHAR(255) NOT NULL DEFAULT 'activa',
    nombre              VARCHAR(100),
    tipo_servicio       VARCHAR(20) NOT NULL DEFAULT 'paradas_continuas',
    tarifa              DECIMAL(5,2) NOT NULL DEFAULT 1.00,
    color               VARCHAR(7) NOT NULL DEFAULT '#155240',
    ruta_circular       BOOLEAN NOT NULL DEFAULT false,
    trazo               JSONB,
```

```
    CONSTRAINT lineas_pkey PRIMARY KEY (id_linea),
    CONSTRAINT uq_lineas_codigo UNIQUE (codigo_linea),
    CONSTRAINT fk_lineas_municipio FOREIGN KEY (id_municipio) REFERENCES municipios(id_municipio) ON UPDATE CASCADE
ON DELETE RESTRICT,
    CONSTRAINT chk_lineas_color CHECK (((color) ~ '^#[0-9A-Fa-f]{6}$')),
    CONSTRAINT chk_lineas_distancia CHECK ((distancia_total_km >= (0))),
    CONSTRAINT chk_lineas_tarifa CHECK ((tarifa >= (0))),
    CONSTRAINT chk_lineas_tipo_servicio CHECK (tipo_servicio IN ('paradas_continuas', 'directo')),
    CONSTRAINT chk_lineas_trazo CHECK (((trazo IS NULL) OR (jsonb_typeof(trazo) = 'array'))),
    CONSTRAINT lineas_estado_check CHECK (estado IN ('activa', 'inactiva', 'mantenimiento'))
);

-- ------------------------------------------------------------------------
-- 3. HORARIOS_LINEA
--    RF-22: horario oficial por tipo de dia (lunes a viernes, sabado, domingo y feriados).
-- ------------------------------------------------------------------------
CREATE TABLE horarios_linea (
    id_horario   UUID NOT NULL DEFAULT gen_random_uuid(),
    id_linea     UUID NOT NULL,
    tipo_dia     VARCHAR(20) NOT NULL,
    hora_inicio  TIME WITHOUT TIME ZONE NOT NULL,
    hora_fin     TIME WITHOUT TIME ZONE NOT NULL,
    CONSTRAINT horarios_linea_pkey PRIMARY KEY (id_horario),
    CONSTRAINT uq_horarios_linea_dia UNIQUE (id_linea, tipo_dia),
    CONSTRAINT fk_horarios_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT chk_horarios_rango CHECK ((hora_fin > hora_inicio)),
    CONSTRAINT chk_horarios_tipo_dia CHECK (tipo_dia IN ('lunes_viernes', 'sabado', 'domingo_feriado'))
);

-- ------------------------------------------------------------------------
-- 4. ESTACIONES
--    Puntos 11 y 14: aforo maximo y actual; direccion y coordenadas para el mapa.
-- ------------------------------------------------------------------------
CREATE TABLE estaciones (
    id_estacion   UUID NOT NULL DEFAULT gen_random_uuid(),
    id_municipio  UUID NOT NULL,
    nombre        VARCHAR(150) NOT NULL,
    aforo_maximo  INTEGER NOT NULL,
    aforo_actual  INTEGER NOT NULL DEFAULT 0,
    latitud       DECIMAL(9,6) NOT NULL,
    longitud      DECIMAL(9,6) NOT NULL,
    direccion     VARCHAR(200),
    CONSTRAINT estaciones_pkey PRIMARY KEY (id_estacion),
    CONSTRAINT fk_estaciones_municipio FOREIGN KEY (id_municipio) REFERENCES municipios(id_municipio) ON UPDATE
CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_estaciones_aforo CHECK (((aforo_maximo > 0) AND (aforo_actual >= 0))),
    CONSTRAINT chk_estaciones_coordenadas CHECK ((((latitud >= ('-90'::integer)) AND (latitud <= (90))) AND
((longitud >= ('-180'::integer)) AND (longitud <= (180)))))
);

-- ------------------------------------------------------------------------
-- 5. LINEA_ESTACION
--    Puntos 1, 2 y 12: orden de las estaciones de cada linea y distancia desde la parada anterior.
-- ------------------------------------------------------------------------
CREATE TABLE linea_estacion (
    id_linea             UUID NOT NULL,
    id_estacion          UUID NOT NULL,
    orden_parada         INTEGER NOT NULL,
    distancia_previa_km  DECIMAL(6,2) NOT NULL DEFAULT '0'::numeric,
    CONSTRAINT linea_estacion_pkey PRIMARY KEY (id_linea, id_estacion),
    CONSTRAINT uq_linea_orden UNIQUE (id_linea, orden_parada),
    CONSTRAINT fk_linea_estacion_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE
CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_linea_estacion_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT chk_linea_estacion_distancia CHECK ((distancia_previa_km >= (0))),
    CONSTRAINT chk_linea_estacion_orden CHECK ((orden_parada >= 1))
);
CREATE INDEX idx_linea_estacion_estacion ON linea_estacion (id_estacion);

-- ------------------------------------------------------------------------
-- 6. BUSES
--    Puntos 4, 14 y 15: un bus en una sola linea o ninguna (de n a 2n por linea), capacidad, pasajeros a bordo y
tipo de unidad.
-- ------------------------------------------------------------------------
CREATE TABLE buses (
    id_bus                UUID NOT NULL DEFAULT gen_random_uuid(),
    placa                 VARCHAR(15) NOT NULL,
    capacidad_maxima      INTEGER NOT NULL,
    id_linea_asignada     UUID,
    ocupacion_actual_pct  DECIMAL(5,2) NOT NULL DEFAULT '0'::numeric,
    estado_logico         VARCHAR(255) NOT NULL DEFAULT 'fuera_de_servicio',
    pasajeros_a_bordo     INTEGER NOT NULL DEFAULT 0,
    tipo_unidad           VARCHAR(20) NOT NULL DEFAULT 'articulado',
    CONSTRAINT buses_pkey PRIMARY KEY (id_bus),
    CONSTRAINT uq_buses_placa UNIQUE (placa),
    CONSTRAINT fk_buses_linea FOREIGN KEY (id_linea_asignada) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON
DELETE SET NULL,
    CONSTRAINT buses_estado_logico_check CHECK (estado_logico IN ('en_ruta', 'en_espera', 'fuera_de_servicio',
'en_mantenimiento')),
    CONSTRAINT chk_buses_a_bordo CHECK (((pasajeros_a_bordo >= 0) AND (pasajeros_a_bordo <= capacidad_maxima))),
    CONSTRAINT chk_buses_capacidad CHECK ((capacidad_maxima > 0)),
    CONSTRAINT chk_buses_ocupacion CHECK (((ocupacion_actual_pct >= (0)) AND (ocupacion_actual_pct <= (100)))),
    CONSTRAINT chk_buses_placa CHECK (((placa) ~ '^(P|A|C|U|O|M|TC|TE)[0-9]{3}[A-Z]{3}$')),
    CONSTRAINT chk_buses_tipo_unidad CHECK (tipo_unidad IN ('biarticulado', 'articulado', 'convencional',
'electrico'))
```

```
);
CREATE INDEX idx_buses_linea ON buses (id_linea_asignada);

-- ----------------------------------------------------------------------
-- 7. ACCESOS
--    Punto 3: varios accesos por estacion (minimo 2); un acceso pertenece a una sola estacion.
-- ----------------------------------------------------------------------
CREATE TABLE accesos (
    id_acceso         UUID NOT NULL DEFAULT gen_random_uuid(),
    id_estacion       UUID NOT NULL,
    ubicacion_fisica  VARCHAR(100) NOT NULL,
    CONSTRAINT accesos_pkey PRIMARY KEY (id_acceso),
    CONSTRAINT fk_accesos_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE RESTRICT
);
CREATE INDEX idx_accesos_estacion ON accesos (id_estacion);

-- ----------------------------------------------------------------------
-- 8. MOLINETES
--    Molinete de cada acceso de la estacion y molinetes de entrada y salida de cada bus (cobro y conteo).
-- ----------------------------------------------------------------------
CREATE TABLE molinetes (
    id_molinete       UUID NOT NULL DEFAULT gen_random_uuid(),
    id_acceso         UUID,
    ip_address        VARCHAR(45) NOT NULL,
    estado_operativo  VARCHAR(255) NOT NULL DEFAULT 'activo',
    id_bus            UUID,
    tipo              VARCHAR(20) NOT NULL DEFAULT 'estacion',
    CONSTRAINT molinetes_pkey PRIMARY KEY (id_molinete),
    CONSTRAINT uq_molinetes_acceso UNIQUE (id_acceso),
    CONSTRAINT uq_molinetes_bus_tipo UNIQUE (id_bus, tipo),
    CONSTRAINT fk_molinetes_acceso FOREIGN KEY (id_acceso) REFERENCES accesos(id_acceso) ON UPDATE CASCADE ON
DELETE RESTRICT,
    CONSTRAINT fk_molinetes_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT chk_molinetes_tipo CHECK (tipo IN ('estacion', 'entrada_bus', 'salida_bus')),
    CONSTRAINT chk_molinetes_ubicacion CHECK (((((tipo) = 'estacion') AND (id_acceso IS NOT NULL) AND (id_bus IS
NULL)) OR (((tipo) = ANY ((ARRAY['entrada_bus', 'salida_bus']))) AND (id_bus IS NOT NULL) AND (id_acceso IS
NULL)))),
    CONSTRAINT molinetes_estado_operativo_check CHECK (estado_operativo IN ('activo', 'inactivo', 'mantenimiento'))
);

-- ----------------------------------------------------------------------
-- 9. PARQUEOS
--    Puntos 6 y 7: cada bus ocupa un parqueo; lo cambia con sp_cambiar_parqueo_bus sin quedarse sin el.
-- ----------------------------------------------------------------------
CREATE TABLE parqueos (
    id_parqueo       UUID NOT NULL DEFAULT gen_random_uuid(),
    id_estacion      UUID NOT NULL,
    codigo_bahia     VARCHAR(20) NOT NULL,
    id_bus_asignado  UUID,
    CONSTRAINT parqueos_pkey PRIMARY KEY (id_parqueo),
    CONSTRAINT uq_parqueos_bus UNIQUE (id_bus_asignado),
    CONSTRAINT uq_parqueos_estacion_bahia UNIQUE (id_estacion, codigo_bahia),
    CONSTRAINT fk_parqueos_bus FOREIGN KEY (id_bus_asignado) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE
SET NULL,
    CONSTRAINT fk_parqueos_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE RESTRICT
);

-- ----------------------------------------------------------------------
-- 10. EMPLEADOS
--    Punto 9: datos generales del personal (contacto y direccion cifrados por la aplicacion).
-- ----------------------------------------------------------------------
CREATE TABLE empleados (
    id_empleado           UUID NOT NULL DEFAULT gen_random_uuid(),
    dpi                   VARCHAR(20) NOT NULL,
    nombre_completo       VARCHAR(200) NOT NULL,
    fecha_nacimiento      DATE NOT NULL,
    rol                   VARCHAR(255) NOT NULL,
    contacto_encriptado   TEXT NOT NULL,
    direccion_encriptada  TEXT NOT NULL,
    activo                BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT empleados_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT uq_empleados_dpi UNIQUE (dpi),
    CONSTRAINT chk_empleados_dpi CHECK (((dpi) ~ '^[0-9]{13}$')),
    CONSTRAINT empleados_rol_check CHECK (rol IN ('piloto', 'seguridad', 'operador'))
);

-- ----------------------------------------------------------------------
-- 11. PILOTOS
--    Punto 9: licencia, experiencia, historial educativo (cifrado) y bus asignado.
-- ----------------------------------------------------------------------
CREATE TABLE pilotos (
    id_empleado                    UUID NOT NULL,
    numero_licencia                VARCHAR(30) NOT NULL,
    tipo_licencia                  VARCHAR(10) NOT NULL,
    anios_experiencia              INTEGER NOT NULL DEFAULT 0,
    nivel_educativo                VARCHAR(50) NOT NULL,
    historial_educativo_encriptado  TEXT,
    id_bus_asignado                UUID,
    CONSTRAINT pilotos_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT uq_pilotos_bus UNIQUE (id_bus_asignado),
    CONSTRAINT uq_pilotos_licencia UNIQUE (numero_licencia),
    CONSTRAINT fk_pilotos_bus FOREIGN KEY (id_bus_asignado) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE
SET NULL,
    CONSTRAINT fk_pilotos_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE ON
DELETE CASCADE,
```

```
    CONSTRAINT chk_pilotos_experiencia CHECK ((anios_experiencia >= 0))
);

-- ----------------------------------------------------------------------
-- 12. SEGURIDAD
--    Punto 10: guardia asignado a un acceso; minimo uno por acceso (trg_seguridad_min_uno).
-- ----------------------------------------------------------------------
CREATE TABLE seguridad (
    id_empleado                     UUID NOT NULL,
    permisos_vigentes               VARCHAR(100) NOT NULL,
    constancia_antecedentes_penales  VARCHAR(100) NOT NULL,
    numero_placa_asignada           VARCHAR(15),
    id_acceso_asignado              UUID,
    CONSTRAINT seguridad_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT fk_seguridad_acceso FOREIGN KEY (id_acceso_asignado) REFERENCES accesos(id_acceso) ON UPDATE CASCADE
ON DELETE SET NULL,
    CONSTRAINT fk_seguridad_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE
ON DELETE CASCADE
);
CREATE INDEX idx_seguridad_acceso ON seguridad (id_acceso_asignado);

-- ----------------------------------------------------------------------
-- 13. OPERADORES
--    Punto 16: un operador por estacion, a cargo de la PC de trabajo.
-- ----------------------------------------------------------------------
CREATE TABLE operadores (
    id_empleado                 UUID NOT NULL,
    nivel_estudios              VARCHAR(50) NOT NULL,
    anios_experiencia_monitoreo  INTEGER NOT NULL DEFAULT 0,
    id_estacion_asignada        UUID,
    CONSTRAINT operadores_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT uq_operadores_estacion UNIQUE (id_estacion_asignada),
    CONSTRAINT fk_operadores_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE
ON DELETE CASCADE,
    CONSTRAINT fk_operadores_estacion FOREIGN KEY (id_estacion_asignada) REFERENCES estaciones(id_estacion) ON
UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT chk_operadores_experiencia CHECK ((anios_experiencia_monitoreo >= 0))
);

-- ----------------------------------------------------------------------
-- 14. TARJETAS
--    Punto 18: Tarjeta Ciudadana (vecino, adulto mayor, discapacidad), saldo y viajes gratis.
-- ----------------------------------------------------------------------
CREATE TABLE tarjetas (
    id_tarjeta                UUID NOT NULL DEFAULT gen_random_uuid(),
    numero_tarjeta            VARCHAR(20) NOT NULL,
    saldo                     DECIMAL(8,2) NOT NULL DEFAULT '0'::numeric,
    estado                    VARCHAR(255) NOT NULL DEFAULT 'activa',
    fecha_emision             TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tipo_tarjeta              VARCHAR(20) NOT NULL DEFAULT 'vecino',
    nombre_titular            VARCHAR(200),
    dpi_titular               VARCHAR(13),
    costo_emision             DECIMAL(6,2) NOT NULL DEFAULT 20.00,
    viajes_gratis_restantes   INTEGER NOT NULL DEFAULT 0,
    afiliada                  BOOLEAN NOT NULL DEFAULT false,
    fecha_nacimiento_titular  DATE,
    id_tarjeta_anterior       UUID,
    motivo_estado             VARCHAR(200),
    fecha_estado              TIMESTAMP(0),
    CONSTRAINT tarjetas_pkey PRIMARY KEY (id_tarjeta),
    CONSTRAINT uq_tarjetas_numero UNIQUE (numero_tarjeta),
    CONSTRAINT fk_tarjetas_anterior FOREIGN KEY (id_tarjeta_anterior) REFERENCES tarjetas(id_tarjeta) ON UPDATE
CASCADE ON DELETE SET NULL,
    CONSTRAINT chk_tarjetas_afiliada CHECK ((((tipo_tarjeta) = 'vecino') OR afiliada)),
    CONSTRAINT chk_tarjetas_costo CHECK ((costo_emision >= (0))),
    CONSTRAINT chk_tarjetas_dpi CHECK (((dpi_titular IS NULL) OR ((dpi_titular) ~ '^[0-9]{13}$'))),
    CONSTRAINT chk_tarjetas_numero CHECK (((numero_tarjeta) ~ '^[0-9]{10}$')),
    CONSTRAINT chk_tarjetas_saldo CHECK ((saldo >= (0))),
    CONSTRAINT chk_tarjetas_tipo CHECK (tipo_tarjeta IN ('vecino', 'adulto_mayor', 'discapacidad')),
    CONSTRAINT chk_tarjetas_titular CHECK (((((tipo_tarjeta) = 'vecino') AND (NOT afiliada)) OR ((nombre_titular IS
NOT NULL) AND (dpi_titular IS NOT NULL)))),
    CONSTRAINT chk_tarjetas_viajes_gratis CHECK ((viajes_gratis_restantes >= 0)),
    CONSTRAINT tarjetas_estado_check CHECK (estado IN ('activa', 'bloqueada', 'vencida', 'reemplazada', 'baja'))
);

-- ----------------------------------------------------------------------
-- 15. RECARGAS
--    Punto 18: recargas de Q5 a Q200 en el punto de recarga.
-- ----------------------------------------------------------------------
CREATE TABLE recargas (
    id_recarga   UUID NOT NULL DEFAULT gen_random_uuid(),
    id_tarjeta   UUID NOT NULL,
    monto        DECIMAL(8,2) NOT NULL,
    metodo_pago  VARCHAR(255) NOT NULL,
    fecha_hora   TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT recargas_pkey PRIMARY KEY (id_recarga),
    CONSTRAINT fk_recargas_tarjeta FOREIGN KEY (id_tarjeta) REFERENCES tarjetas(id_tarjeta) ON UPDATE CASCADE ON
DELETE RESTRICT,
    CONSTRAINT chk_recargas_monto CHECK (((monto >= (5)) AND (monto <= (200)))),
    CONSTRAINT recargas_metodo_pago_check CHECK (metodo_pago IN ('efectivo', 'debito', 'credito', 'en_linea'))
);
CREATE INDEX idx_recargas_tarjeta ON recargas (id_tarjeta);

-- ----------------------------------------------------------------------
-- 16. VIAJES
--    Punto 1: recorridos reales de cada bus en su linea.
-- ----------------------------------------------------------------------
```

```
CREATE TABLE viajes (
    id_viaje           UUID NOT NULL DEFAULT gen_random_uuid(),
    id_bus             UUID NOT NULL,
    id_linea           UUID NOT NULL,
    fecha_hora_inicio  TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_hora_fin     TIMESTAMP(0),
    estado             VARCHAR(255) NOT NULL DEFAULT 'en_curso',
    CONSTRAINT viajes_pkey PRIMARY KEY (id_viaje),
    CONSTRAINT fk_viajes_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_viajes_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON DELETE
RESTRICT,
    CONSTRAINT chk_viajes_fechas CHECK (((fecha_hora_fin IS NULL) OR (fecha_hora_fin >= fecha_hora_inicio))),
    CONSTRAINT viajes_estado_check CHECK (estado IN ('en_curso', 'finalizado', 'cancelado'))
);
CREATE INDEX idx_viajes_bus ON viajes (id_bus);
CREATE INDEX idx_viajes_linea ON viajes (id_linea);

-- ---------------------------------------------------------------------
-- 17. VISITAS_VIAJE
--    Puntos 1 y 15: paradas visitadas con hora de llegada y salida, ocupacion y espera extra.
-- ---------------------------------------------------------------------
CREATE TABLE visitas_viaje (
    id_visita               UUID NOT NULL DEFAULT gen_random_uuid(),
    id_viaje                UUID NOT NULL,
    id_estacion             UUID NOT NULL,
    orden_visita            INTEGER NOT NULL,
    hora_llegada            TIMESTAMP(0),
    hora_salida             TIMESTAMP(0),
    ocupacion_al_llegar_pct  DECIMAL(5,2),
    subieron                INTEGER NOT NULL DEFAULT 0,
    bajaron                 INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT visitas_viaje_pkey PRIMARY KEY (id_visita),
    CONSTRAINT uq_visitas_viaje_orden UNIQUE (id_viaje, orden_visita),
    CONSTRAINT fk_visitas_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE RESTRICT,
    CONSTRAINT fk_visitas_viaje FOREIGN KEY (id_viaje) REFERENCES viajes(id_viaje) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT chk_visitas_horas CHECK (((hora_salida IS NULL) OR (hora_llegada IS NULL) OR (hora_salida >=
hora_llegada))),
    CONSTRAINT chk_visitas_ocupacion CHECK (((ocupacion_al_llegar_pct IS NULL) OR ((ocupacion_al_llegar_pct >= (0))
AND (ocupacion_al_llegar_pct <= (100))))),
    CONSTRAINT chk_visitas_orden CHECK ((orden_visita >= 1)),
    CONSTRAINT chk_visitas_pasajeros CHECK (((subieron >= 0) AND (bajaron >= 0)))
);
CREATE INDEX idx_visitas_estacion ON visitas_viaje (id_estacion, hora_llegada);

-- ---------------------------------------------------------------------
-- 18. ALERTAS_SISTEMA
--    Punto 14: alertas de refuerzo (150 % del aforo) y de bus lleno.
-- ---------------------------------------------------------------------
CREATE TABLE alertas_sistema (
    id_alerta       UUID NOT NULL DEFAULT gen_random_uuid(),
    id_estacion     UUID NOT NULL,
    tipo_alerta     VARCHAR(255) NOT NULL,
    descripcion     TEXT NOT NULL,
    estado          VARCHAR(255) NOT NULL DEFAULT 'abierta',
    fecha_creacion  TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_cierre    TIMESTAMP(0),
    id_bus          UUID,
    CONSTRAINT alertas_sistema_pkey PRIMARY KEY (id_alerta),
    CONSTRAINT fk_alertas_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_alertas_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE CASCADE,
    CONSTRAINT alertas_sistema_estado_check CHECK (estado IN ('abierta', 'atendida', 'cerrada')),
    CONSTRAINT alertas_sistema_tipo_alerta_check CHECK (tipo_alerta IN ('refuerzo_bus', 'bus_lleno',
'bahia_ocupada', 'espera_minima', 'otro'))
);
CREATE INDEX idx_alertas_estado ON alertas_sistema (estado);

-- ---------------------------------------------------------------------
-- 19. TRANSACCIONES_PAGO
--    Punto 18: pago de cada pasajero en el molinete de entrada del bus (sin efectivo).
-- ---------------------------------------------------------------------
CREATE TABLE transacciones_pago (
    id_transaccion  UUID NOT NULL DEFAULT gen_random_uuid(),
    id_molinete     UUID NOT NULL,
    id_tarjeta      UUID,
    monto           DECIMAL(8,2) NOT NULL,
    metodo_pago     VARCHAR(255) NOT NULL,
    fecha_hora      TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_linea        UUID,
    viaje_gratis    BOOLEAN NOT NULL DEFAULT false,
    id_bus          UUID,
    id_estacion     UUID,
    id_visita       UUID,
    CONSTRAINT transacciones_pago_pkey PRIMARY KEY (id_transaccion),
    CONSTRAINT fk_transacciones_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE SET
NULL,
    CONSTRAINT fk_transacciones_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE
CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_transacciones_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON
DELETE SET NULL,
    CONSTRAINT fk_transacciones_molinete FOREIGN KEY (id_molinete) REFERENCES molinetes(id_molinete) ON UPDATE
CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_transacciones_tarjeta FOREIGN KEY (id_tarjeta) REFERENCES tarjetas(id_tarjeta) ON UPDATE CASCADE
ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_transacciones_visita FOREIGN KEY (id_visita) REFERENCES visitas_viaje(id_visita) ON UPDATE
CASCADE ON DELETE SET NULL,
    CONSTRAINT chk_transacciones_monto CHECK (((monto >= (0)) AND (viaje_gratis OR (monto > (0))))),
    CONSTRAINT chk_transacciones_tarjeta CHECK ((((metodo_pago) <> 'tarjeta_ciudadana') OR (id_tarjeta IS NOT
NULL))),
    CONSTRAINT transacciones_pago_metodo_pago_check CHECK (metodo_pago IN ('tarjeta_ciudadana',
'tarjeta_bancaria'))
);
CREATE INDEX idx_transacciones_bus ON transacciones_pago (id_bus, fecha_hora);
CREATE INDEX idx_transacciones_estacion ON transacciones_pago (id_estacion, fecha_hora);
CREATE INDEX idx_transacciones_fecha ON transacciones_pago (fecha_hora);
CREATE INDEX idx_transacciones_linea ON transacciones_pago (id_linea);
CREATE INDEX idx_transacciones_linea_fecha ON transacciones_pago (id_linea, fecha_hora);
CREATE INDEX idx_transacciones_tarjeta ON transacciones_pago (id_tarjeta);
CREATE INDEX idx_transacciones_visita ON transacciones_pago (id_visita);

-- ----------------------------------------------------------------------
-- 20. BAJADAS_BUS
--    Conteo de las personas que bajan por el molinete de salida del bus.
-- ----------------------------------------------------------------------
CREATE TABLE bajadas_bus (
    id_bajada    UUID NOT NULL DEFAULT gen_random_uuid(),
    id_molinete  UUID NOT NULL,
    id_bus       UUID,
    id_estacion  UUID,
    id_visita    UUID,
    cantidad     INTEGER NOT NULL DEFAULT 1,
    fecha_hora   TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT bajadas_bus_pkey PRIMARY KEY (id_bajada),
    CONSTRAINT fk_bajadas_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_bajadas_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE SET NULL,
    CONSTRAINT fk_bajadas_molinete FOREIGN KEY (id_molinete) REFERENCES molinetes(id_molinete) ON UPDATE CASCADE ON
DELETE RESTRICT,
    CONSTRAINT fk_bajadas_visita FOREIGN KEY (id_visita) REFERENCES visitas_viaje(id_visita) ON UPDATE CASCADE ON
DELETE SET NULL,
    CONSTRAINT chk_bajadas_cantidad CHECK ((cantidad >= 1))
);
CREATE INDEX idx_bajadas_bus ON bajadas_bus (id_bus, fecha_hora);
CREATE INDEX idx_bajadas_estacion ON bajadas_bus (id_estacion, fecha_hora);
CREATE INDEX idx_bajadas_visita ON bajadas_bus (id_visita);

-- ----------------------------------------------------------------------
-- 21. MANTENIMIENTOS_BUS
--    Fallas reportadas por el operador y mantenimientos del taller.
-- ----------------------------------------------------------------------
CREATE TABLE mantenimientos_bus (
    id_mantenimiento     UUID NOT NULL DEFAULT gen_random_uuid(),
    id_bus               UUID NOT NULL,
    id_empleado_reporta  UUID,
    tipo                 VARCHAR(255) NOT NULL,
    categoria            VARCHAR(30) NOT NULL,
    descripcion          TEXT NOT NULL,
    prioridad            VARCHAR(255) NOT NULL DEFAULT 'media',
    estado               VARCHAR(255) NOT NULL DEFAULT 'reportado',
    kilometraje          INTEGER,
    fecha_reporte        TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_cierre         TIMESTAMP(0),
    CONSTRAINT mantenimientos_bus_pkey PRIMARY KEY (id_mantenimiento),
    CONSTRAINT fk_mantenimientos_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_mantenimientos_empleado FOREIGN KEY (id_empleado_reporta) REFERENCES empleados(id_empleado) ON
UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT chk_mantenimientos_cierre CHECK ((((estado) = 'resuelto') = (fecha_cierre IS NOT NULL))),
    CONSTRAINT chk_mantenimientos_km CHECK (((kilometraje IS NULL) OR (kilometraje >= 0))),
    CONSTRAINT mantenimientos_bus_estado_check CHECK (estado IN ('reportado', 'en_proceso', 'resuelto')),
    CONSTRAINT mantenimientos_bus_prioridad_check CHECK (prioridad IN ('baja', 'media', 'alta')),
    CONSTRAINT mantenimientos_bus_tipo_check CHECK (tipo IN ('falla', 'preventivo', 'correctivo'))
);
CREATE INDEX idx_mantenimientos_bus ON mantenimientos_bus (id_bus, fecha_reporte);

-- ----------------------------------------------------------------------
-- 22. BITACORA_AUDITORIA
--    Auditoria: usuario, tabla, registro, accion, datos anteriores y posteriores.
-- ----------------------------------------------------------------------
CREATE TABLE bitacora_auditoria (
    id_bitacora           UUID NOT NULL DEFAULT gen_random_uuid(),
    id_usuario            BIGINT,
    id_empleado_usuario   UUID,
    tabla_afectada        VARCHAR(100) NOT NULL,
    id_registro_afectado  VARCHAR(36),
    accion                VARCHAR(255) NOT NULL,
    datos_previos         JSONB,
    datos_nuevos          JSONB,
    fecha_registro        TIMESTAMP(6) WITHOUT TIME ZONE NOT NULL DEFAULT clock_timestamp(),
    CONSTRAINT bitacora_auditoria_pkey PRIMARY KEY (id_bitacora),
    CONSTRAINT fk_bitacora_empleado FOREIGN KEY (id_empleado_usuario) REFERENCES empleados(id_empleado) ON UPDATE
CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_bitacora_usuario FOREIGN KEY (id_usuario) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET
NULL,
    CONSTRAINT bitacora_auditoria_accion_check CHECK (accion IN ('INSERT', 'UPDATE', 'DELETE'))
);
CREATE INDEX idx_bitacora_fecha ON bitacora_auditoria (fecha_registro DESC);
CREATE INDEX idx_bitacora_tabla ON bitacora_auditoria (tabla_afectada, id_registro_afectado);

-- ======================================================================
-- USUARIOS DEL SISTEMA (tabla users de Laravel): rol y empleado vinculado
```

```
-- ========================================================================
ALTER TABLE users
    ADD COLUMN rol VARCHAR(255) NOT NULL DEFAULT 'operador',
    ADD COLUMN id_empleado UUID NULL,
    ADD CONSTRAINT fk_users_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE
ON DELETE SET NULL,
    ADD CONSTRAINT uq_users_empleado UNIQUE (id_empleado),
    ADD CONSTRAINT users_rol_check CHECK (rol IN ('administrador', 'operador'));

-- ========================================================================
-- REGLAS DE NEGOCIO: funciones, triggers y procedimiento almacenado
-- ========================================================================
CREATE OR REPLACE PROCEDURE sp_cambiar_parqueo_bus(IN p_id_bus uuid, IN p_id_parqueo_nuevo uuid)
 LANGUAGE plpgsql
AS $procedure$
DECLARE
    v_parqueo_actual UUID;
    v_ocupado UUID;
BEGIN
    SELECT id_bus_asignado INTO v_ocupado
      FROM parqueos WHERE id_parqueo = p_id_parqueo_nuevo
       FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El parqueo destino no existe.';
    END IF;
    IF v_ocupado = p_id_bus THEN
        RETURN;
    END IF;
    IF v_ocupado IS NOT NULL THEN
        RAISE EXCEPTION 'El parqueo destino ya esta ocupado por otro bus.';
    END IF;

    SELECT id_parqueo INTO v_parqueo_actual
      FROM parqueos WHERE id_bus_asignado = p_id_bus
       FOR UPDATE;
    IF v_parqueo_actual IS NOT NULL THEN
        UPDATE parqueos SET id_bus_asignado = NULL WHERE id_parqueo = v_parqueo_actual;
    END IF;
    UPDATE parqueos SET id_bus_asignado = p_id_bus WHERE id_parqueo = p_id_parqueo_nuevo;
END;
$procedure$;

CREATE OR REPLACE FUNCTION fn_accesos_min_dos()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_conteo INT;
BEGIN
    SELECT COUNT(*) INTO v_conteo FROM accesos WHERE id_estacion = OLD.id_estacion;
    IF v_conteo <= 2 THEN
        RAISE EXCEPTION 'No se puede eliminar: la estacion debe conservar al menos 2 accesos.';
    END IF;
    RETURN OLD;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_bajadas_en_bus()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_tipo VARCHAR;
    v_bus UUID;
    v_actual RECORD;
BEGIN
    SELECT tipo, id_bus INTO v_tipo, v_bus FROM molinetes WHERE id_molinete = NEW.id_molinete;
    IF v_tipo IS DISTINCT FROM 'salida_bus' THEN
        RAISE EXCEPTION 'Las bajadas se cuentan en el molinete de salida del bus.';
    END IF;
    NEW.id_bus := v_bus;
    -- Si no se indica la estación (lo normal en el molinete real), es donde está detenido el bus.
    IF NEW.id_visita IS NULL AND NEW.id_estacion IS NULL THEN
        SELECT * INTO v_actual FROM fn_visita_actual_bus(v_bus);
        IF v_actual.id_visita IS NULL THEN
            RAISE EXCEPTION 'El bus no esta detenido en una estacion: no tiene un viaje en curso.';
        END IF;
        NEW.id_visita := v_actual.id_visita;
        NEW.id_estacion := COALESCE(NEW.id_estacion, v_actual.id_estacion);
    END IF;
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_bajadas_restan()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_placa VARCHAR;
BEGIN
    SELECT b.placa INTO v_placa
      FROM (SELECT id_bus, SUM(cantidad) AS n FROM nuevas GROUP BY id_bus) s
      JOIN buses b ON b.id_bus = s.id_bus
     WHERE s.n > b.pasajeros_a_bordo
     LIMIT 1;
    IF v_placa IS NOT NULL THEN
```

```
        RAISE EXCEPTION 'En el bus % el contador de salida marca mas bajadas que pasajeros a bordo.', v_placa;
    END IF;

    UPDATE buses b SET pasajeros_a_bordo = b.pasajeros_a_bordo - s.n
      FROM (SELECT id_bus, SUM(cantidad) AS n FROM nuevas GROUP BY id_bus) s
     WHERE b.id_bus = s.id_bus;
    UPDATE visitas_viaje vv SET bajaron = vv.bajaron + s.n
      FROM (SELECT id_visita, SUM(cantidad) AS n FROM nuevas WHERE id_visita IS NOT NULL GROUP BY id_visita) s
     WHERE vv.id_visita = s.id_visita;
    RETURN NULL;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_bitacora_inmutable()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
    BEGIN
        IF TG_OP = 'DELETE' THEN
            RAISE EXCEPTION 'La bitácora de auditoría no se puede eliminar.' USING ERRCODE = 'P0001';
        END IF;
        IF NEW.id_bitacora IS DISTINCT FROM OLD.id_bitacora
           OR NEW.tabla_afectada IS DISTINCT FROM OLD.tabla_afectada
           OR NEW.id_registro_afectado IS DISTINCT FROM OLD.id_registro_afectado
           OR NEW.accion IS DISTINCT FROM OLD.accion
           OR NEW.datos_previos IS DISTINCT FROM OLD.datos_previos
           OR NEW.datos_nuevos IS DISTINCT FROM OLD.datos_nuevos
           OR NEW.fecha_registro IS DISTINCT FROM OLD.fecha_registro
           OR (NEW.id_usuario IS DISTINCT FROM OLD.id_usuario AND NEW.id_usuario IS NOT NULL)
           OR (NEW.id_empleado_usuario IS DISTINCT FROM OLD.id_empleado_usuario AND NEW.id_empleado_usuario IS NOT
NULL) THEN
            RAISE EXCEPTION 'La bitácora de auditoría no se puede modificar.' USING ERRCODE = 'P0001';
        END IF;
        RETURN NEW;
    END;
    $function$;

CREATE OR REPLACE FUNCTION fn_buses_max_flota()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_buses_actuales INT;
    v_paradas INT;
BEGIN
    IF NEW.id_linea_asignada IS NOT NULL
       AND (TG_OP = 'INSERT' OR NEW.id_linea_asignada IS DISTINCT FROM OLD.id_linea_asignada) THEN
        SELECT COUNT(*) INTO v_buses_actuales FROM buses WHERE id_linea_asignada = NEW.id_linea_asignada;
        SELECT COUNT(*) INTO v_paradas FROM linea_estacion WHERE id_linea = NEW.id_linea_asignada;
        IF v_buses_actuales >= 2 * v_paradas THEN
            RAISE EXCEPTION 'No se puede asignar el bus: se alcanzo el maximo permitido (2x el numero de paradas)
para esta linea.';
        END IF;
    END IF;
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_buses_molinetes()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_n INT;
BEGIN
    SELECT COUNT(*) INTO v_n FROM buses;
    INSERT INTO molinetes (id_bus, tipo, ip_address, estado_operativo) VALUES
        (NEW.id_bus, 'entrada_bus', '10.20.' || v_n || '.11', 'activo'),
        (NEW.id_bus, 'salida_bus',  '10.20.' || v_n || '.12', 'activo')
    ON CONFLICT (id_bus, tipo) DO NOTHING;
    RETURN NULL;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_buses_ocupacion()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
BEGIN
    NEW.ocupacion_actual_pct := ROUND(NEW.pasajeros_a_bordo * 100.0 / GREATEST(NEW.capacidad_maxima, 1), 2);
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_buses_ruta_con_piloto()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
BEGIN
    IF NEW.estado_logico = 'en_ruta'
       AND (TG_OP = 'INSERT' OR OLD.estado_logico IS DISTINCT FROM 'en_ruta')
       AND NOT EXISTS (SELECT 1 FROM pilotos WHERE id_bus_asignado = NEW.id_bus) THEN
        RAISE EXCEPTION 'El bus % no tiene piloto asignado: no puede salir a ruta.', NEW.placa;
    END IF;
    RETURN NEW;
END;
$function$;
```

```
CREATE OR REPLACE FUNCTION fn_estaciones_alerta_aforo()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
BEGIN
    IF NEW.aforo_actual >= 1.5 * NEW.aforo_maximo
       AND OLD.aforo_actual < 1.5 * OLD.aforo_maximo THEN
        INSERT INTO alertas_sistema (id_estacion, tipo_alerta, descripcion, estado)
        VALUES (
            NEW.id_estacion,
            'refuerzo_bus',
            'La estacion supero el 150% de su aforo maximo (' || NEW.aforo_actual || '/' || NEW.aforo_maximo
                || '). Se requiere unidad de refuerzo.',
            'abierta'
        );
    END IF;
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_linea_estacion_distancia()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
BEGIN
    IF TG_OP IN ('UPDATE', 'DELETE') THEN
        UPDATE lineas
           SET distancia_total_km = COALESCE((SELECT SUM(distancia_previa_km) FROM linea_estacion WHERE id_linea =
OLD.id_linea), 0)
         WHERE id_linea = OLD.id_linea;
    END IF;
    IF TG_OP IN ('INSERT', 'UPDATE') THEN
        UPDATE lineas
           SET distancia_total_km = COALESCE((SELECT SUM(distancia_previa_km) FROM linea_estacion WHERE id_linea =
NEW.id_linea), 0)
         WHERE id_linea = NEW.id_linea;
    END IF;
    RETURN NULL;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_pilotos_deja_bus()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
BEGIN
    IF OLD.id_bus_asignado IS NOT NULL
       AND (TG_OP = 'DELETE' OR NEW.id_bus_asignado IS DISTINCT FROM OLD.id_bus_asignado)
       AND NOT EXISTS (SELECT 1 FROM pilotos WHERE id_bus_asignado = OLD.id_bus_asignado) THEN
        UPDATE viajes SET estado = 'finalizado', fecha_hora_fin = (now() AT TIME ZONE
'America/Guatemala')::timestamp(0)
         WHERE id_bus = OLD.id_bus_asignado AND estado = 'en_curso';
        UPDATE buses SET estado_logico = 'en_espera', pasajeros_a_bordo = 0
         WHERE id_bus = OLD.id_bus_asignado AND estado_logico = 'en_ruta';
    END IF;
    RETURN NULL;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_seguridad_min_uno()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_conteo INT;
BEGIN
    IF OLD.id_acceso_asignado IS NOT NULL
       AND (TG_OP = 'DELETE' OR NEW.id_acceso_asignado IS DISTINCT FROM OLD.id_acceso_asignado)
       AND EXISTS (SELECT 1 FROM accesos WHERE id_acceso = OLD.id_acceso_asignado) THEN
        SELECT COUNT(*) INTO v_conteo FROM seguridad WHERE id_acceso_asignado = OLD.id_acceso_asignado;
        IF v_conteo <= 1 THEN
            RAISE EXCEPTION 'No se puede reasignar ni dar de baja: el acceso quedaria sin personal de seguridad.';
        END IF;
    END IF;
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_tarjeta_activa()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_estado VARCHAR;
    v_numero VARCHAR;
BEGIN
    IF NEW.id_tarjeta IS NULL THEN
        RETURN NEW;
    END IF;
    SELECT estado, numero_tarjeta INTO v_estado, v_numero FROM tarjetas WHERE id_tarjeta = NEW.id_tarjeta;
    IF v_estado IS DISTINCT FROM 'activa' THEN
        RAISE EXCEPTION 'La tarjeta % no está activa (%): no se puede usar.', v_numero, v_estado;
    END IF;
```

```
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_tarjetas_estado_final()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
BEGIN
    IF NEW.estado IS DISTINCT FROM OLD.estado THEN
        IF OLD.estado IN ('baja', 'reemplazada') THEN
            RAISE EXCEPTION 'La tarjeta % está %: su estado ya no se puede cambiar.', OLD.numero_tarjeta,
                CASE OLD.estado WHEN 'baja' THEN 'dada de baja' ELSE 'reemplazada' END;
        END IF;
        IF NEW.fecha_estado IS NOT DISTINCT FROM OLD.fecha_estado THEN
            NEW.fecha_estado := now()::timestamp(0);
        END IF;
    END IF;
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_transacciones_en_bus()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_tipo VARCHAR;
    v_bus UUID;
    v_actual RECORD;
BEGIN
    SELECT tipo, id_bus INTO v_tipo, v_bus FROM molinetes WHERE id_molinete = NEW.id_molinete;
    IF v_tipo IS DISTINCT FROM 'entrada_bus' THEN
        RAISE EXCEPTION 'El pasaje se cobra en el molinete de entrada del bus.';
    END IF;
    NEW.id_bus := v_bus;
    -- Si no se indica la estación (lo normal en el molinete real), es donde está detenido el bus.
    IF NEW.id_visita IS NULL AND NEW.id_estacion IS NULL THEN
        SELECT * INTO v_actual FROM fn_visita_actual_bus(v_bus);
        IF v_actual.id_visita IS NULL THEN
            RAISE EXCEPTION 'El bus no esta detenido en una estacion: no tiene un viaje en curso.';
        END IF;
        NEW.id_visita := v_actual.id_visita;
        NEW.id_estacion := COALESCE(NEW.id_estacion, v_actual.id_estacion);
        NEW.id_linea := COALESCE(NEW.id_linea, v_actual.id_linea);
    END IF;
    RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_transacciones_suben()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_placa VARCHAR;
BEGIN
    SELECT b.placa INTO v_placa
      FROM (SELECT id_bus, COUNT(*) AS n FROM nuevos WHERE id_bus IS NOT NULL GROUP BY id_bus) s
      JOIN buses b ON b.id_bus = s.id_bus
     WHERE b.pasajeros_a_bordo + s.n > b.capacidad_maxima
     LIMIT 1;
    IF v_placa IS NOT NULL THEN
        RAISE EXCEPTION 'El bus % esta lleno: el molinete de entrada no deja subir mas pasajeros.', v_placa;
    END IF;

    UPDATE estaciones e SET aforo_actual = GREATEST(0, e.aforo_actual - s.n)
      FROM (SELECT id_estacion, COUNT(*) AS n FROM nuevos WHERE id_bus IS NOT NULL AND id_estacion IS NOT NULL
GROUP BY id_estacion) s
     WHERE e.id_estacion = s.id_estacion;
    UPDATE buses b SET pasajeros_a_bordo = b.pasajeros_a_bordo + s.n
      FROM (SELECT id_bus, COUNT(*) AS n FROM nuevos WHERE id_bus IS NOT NULL GROUP BY id_bus) s
     WHERE b.id_bus = s.id_bus;
    UPDATE visitas_viaje vv SET subieron = vv.subieron + s.n
      FROM (SELECT id_visita, COUNT(*) AS n FROM nuevos WHERE id_visita IS NOT NULL GROUP BY id_visita) s
     WHERE vv.id_visita = s.id_visita;
    RETURN NULL;
END;
$function$;

CREATE OR REPLACE FUNCTION fn_visita_actual_bus(p_id_bus uuid, OUT id_visita uuid, OUT id_estacion uuid, OUT
id_linea uuid)
 RETURNS record
 LANGUAGE sql
 STABLE
AS $function$
    SELECT vv.id_visita, vv.id_estacion, v.id_linea
      FROM viajes v
      JOIN visitas_viaje vv ON vv.id_viaje = v.id_viaje
     WHERE v.id_bus = p_id_bus AND v.estado = 'en_curso'
     ORDER BY vv.orden_visita DESC
     LIMIT 1
$function$;

CREATE TRIGGER trg_accesos_min_dos_before_delete BEFORE DELETE ON accesos FOR EACH ROW EXECUTE FUNCTION
fn_accesos_min_dos();
CREATE TRIGGER trg_bajadas_en_bus BEFORE INSERT ON bajadas_bus FOR EACH ROW EXECUTE FUNCTION fn_bajadas_en_bus();
```

```
CREATE TRIGGER trg_bajadas_restan AFTER INSERT ON bajadas_bus REFERENCING NEW TABLE AS nuevas FOR EACH STATEMENT
EXECUTE FUNCTION fn_bajadas_restan();
CREATE TRIGGER trg_bitacora_inmutable BEFORE DELETE OR UPDATE ON bitacora_auditoria FOR EACH ROW EXECUTE FUNCTION
fn_bitacora_inmutable();
CREATE TRIGGER trg_buses_max_flota BEFORE INSERT OR UPDATE OF id_linea_asignada ON buses FOR EACH ROW EXECUTE
FUNCTION fn_buses_max_flota();
CREATE TRIGGER trg_buses_molinetes AFTER INSERT ON buses FOR EACH ROW EXECUTE FUNCTION fn_buses_molinetes();
CREATE TRIGGER trg_buses_ocupacion BEFORE INSERT OR UPDATE ON buses FOR EACH ROW EXECUTE FUNCTION
fn_buses_ocupacion();
CREATE TRIGGER trg_buses_ruta_con_piloto BEFORE INSERT OR UPDATE OF estado_logico ON buses FOR EACH ROW EXECUTE
FUNCTION fn_buses_ruta_con_piloto();
CREATE TRIGGER trg_estaciones_alerta_aforo AFTER UPDATE OF aforo_actual, aforo_maximo ON estaciones FOR EACH ROW
EXECUTE FUNCTION fn_estaciones_alerta_aforo();
CREATE TRIGGER trg_linea_estacion_distancia AFTER INSERT OR DELETE OR UPDATE ON linea_estacion FOR EACH ROW EXECUTE
FUNCTION fn_linea_estacion_distancia();
CREATE TRIGGER trg_pilotos_deja_bus AFTER DELETE OR UPDATE OF id_bus_asignado ON pilotos FOR EACH ROW EXECUTE
FUNCTION fn_pilotos_deja_bus();
CREATE TRIGGER trg_recargas_tarjeta_activa BEFORE INSERT ON recargas FOR EACH ROW EXECUTE FUNCTION
fn_tarjeta_activa();
CREATE TRIGGER trg_seguridad_min_uno BEFORE DELETE OR UPDATE ON seguridad FOR EACH ROW EXECUTE FUNCTION
fn_seguridad_min_uno();
CREATE TRIGGER trg_tarjetas_estado_final BEFORE UPDATE OF estado ON tarjetas FOR EACH ROW EXECUTE FUNCTION
fn_tarjetas_estado_final();
CREATE TRIGGER trg_transacciones_en_bus BEFORE INSERT ON transacciones_pago FOR EACH ROW EXECUTE FUNCTION
fn_transacciones_en_bus();
CREATE TRIGGER trg_transacciones_suben AFTER INSERT ON transacciones_pago REFERENCING NEW TABLE AS nuevos FOR EACH
STATEMENT EXECUTE FUNCTION fn_transacciones_suben();
CREATE TRIGGER trg_transacciones_tarjeta_activa BEFORE INSERT ON transacciones_pago FOR EACH ROW EXECUTE FUNCTION
fn_tarjeta_activa();

-- ====================================================================
-- SEGURIDAD: Row Level Security en todas las tablas del sistema
-- ====================================================================
ALTER TABLE municipios ENABLE ROW LEVEL SECURITY;
ALTER TABLE lineas ENABLE ROW LEVEL SECURITY;
ALTER TABLE horarios_linea ENABLE ROW LEVEL SECURITY;
ALTER TABLE estaciones ENABLE ROW LEVEL SECURITY;
ALTER TABLE linea_estacion ENABLE ROW LEVEL SECURITY;
ALTER TABLE buses ENABLE ROW LEVEL SECURITY;
ALTER TABLE accesos ENABLE ROW LEVEL SECURITY;
ALTER TABLE molinetes ENABLE ROW LEVEL SECURITY;
ALTER TABLE parqueos ENABLE ROW LEVEL SECURITY;
ALTER TABLE empleados ENABLE ROW LEVEL SECURITY;
ALTER TABLE pilotos ENABLE ROW LEVEL SECURITY;
ALTER TABLE seguridad ENABLE ROW LEVEL SECURITY;
ALTER TABLE operadores ENABLE ROW LEVEL SECURITY;
ALTER TABLE tarjetas ENABLE ROW LEVEL SECURITY;
ALTER TABLE recargas ENABLE ROW LEVEL SECURITY;
ALTER TABLE viajes ENABLE ROW LEVEL SECURITY;
ALTER TABLE visitas_viaje ENABLE ROW LEVEL SECURITY;
ALTER TABLE alertas_sistema ENABLE ROW LEVEL SECURITY;
ALTER TABLE transacciones_pago ENABLE ROW LEVEL SECURITY;
ALTER TABLE bajadas_bus ENABLE ROW LEVEL SECURITY;
ALTER TABLE mantenimientos_bus ENABLE ROW LEVEL SECURITY;
ALTER TABLE bitacora_auditoria ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
```

9.2. Consultas SQL de los reportes

El enunciado pide dejar constancia de las consultas de los reportes. Estas son las consultas que ejecuta la aplicación (punto 15); los parámetros :linea y :municipio son filtros opcionales.

```
-- Sistema de Control Transmetro — Consultas SQL de los reportes (punto 15)
-- PostgreSQL. Los parámetros :linea y :municipio son filtros opcionales (NULL = todos).
-- Este archivo se genera con: php artisan transmetro:consultas-reportes

-- ====================================================================
-- Estaciones (Puntos 2, 3, 6, 10, 11 y 14)
-- Estaciones con municipio, líneas, aforo, accesos, agentes de seguridad y parqueos.
-- ====================================================================
SELECT e.nombre AS estacion,
       m.nombre AS municipio,
       e.direccion,
       string_agg(l.codigo_linea, ', ' ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'),
coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '', 'g'), '')::int, 0)) AS lineas,
       e.aforo_maximo,
       e.aforo_actual,
       round(e.aforo_actual * 100.0 / e.aforo_maximo, 1) AS ocupacion_pct,
       CASE WHEN e.aforo_actual >= 1.5 * e.aforo_maximo THEN 'Requiere refuerzo' ELSE 'Normal' END AS estado_aforo,
       (SELECT count(*) FROM accesos a WHERE a.id_estacion = e.id_estacion) AS accesos,
       (SELECT count(*) FROM seguridad s
          JOIN accesos a ON a.id_acceso = s.id_acceso_asignado
         WHERE a.id_estacion = e.id_estacion) AS guardias,
       (SELECT count(*) FROM parqueos p WHERE p.id_estacion = e.id_estacion) AS parqueos
  FROM estaciones e
  JOIN municipios m ON m.id_municipio = e.id_municipio
  LEFT JOIN linea_estacion le ON le.id_estacion = e.id_estacion
  LEFT JOIN lineas l ON l.id_linea = le.id_linea
 WHERE (CAST(:linea AS varchar) IS NULL
        OR e.id_estacion IN (SELECT x.id_estacion
                             FROM linea_estacion x
                             JOIN lineas y ON y.id_linea = x.id_linea
```

```
                              WHERE y.codigo_linea = :linea))
   AND (CAST(:municipio AS varchar) IS NULL OR m.nombre = :municipio)
 GROUP BY e.id_estacion, m.nombre
 ORDER BY e.nombre;

-- ========================================================================
-- Líneas (Puntos 4, 5, 8, 11 y 13)
-- Líneas con municipio, estaciones, distancia, accesos, flota asignada y cumplimiento de la flota autorizada.
-- ========================================================================
SELECT l.codigo_linea AS codigo,
       l.nombre,
       m.nombre AS municipio,
       count(le.id_estacion) AS estaciones,
       l.distancia_total_km,
       (SELECT count(*) FROM accesos a
          JOIN linea_estacion x ON x.id_estacion = a.id_estacion
         WHERE x.id_linea = l.id_linea) AS accesos,
       (SELECT count(*) FROM buses b WHERE b.id_linea_asignada = l.id_linea) AS buses_asignados,
       count(le.id_estacion) AS minimo_buses,
       2 * count(le.id_estacion) AS maximo_buses,
       CASE WHEN (SELECT count(*) FROM buses b WHERE b.id_linea_asignada = l.id_linea)
                 BETWEEN count(le.id_estacion) AND 2 * count(le.id_estacion)
            THEN 'Cumple' ELSE 'No cumple' END AS regla_buses,
       l.tarifa,
       l.estado
  FROM lineas l
  JOIN municipios m ON m.id_municipio = l.id_municipio
  LEFT JOIN linea_estacion le ON le.id_linea = l.id_linea
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
   AND (CAST(:municipio AS varchar) IS NULL OR m.nombre = :municipio)
 GROUP BY l.id_linea, m.nombre
 ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'), coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '',
'g'), '')::int, 0);

-- ========================================================================
-- Buses asignados a cada línea (Puntos 4, 6, 7, 9 y 15)
-- Unidades de cada línea con su tipo, capacidad, pasajeros a bordo, ocupación, estado, parqueo y piloto.
-- ========================================================================
SELECT coalesce(l.codigo_linea, 'Sin línea') AS linea,
       b.placa,
       CASE b.tipo_unidad WHEN 'biarticulado' THEN 'Biarticulado' WHEN 'articulado' THEN 'Articulado'
            WHEN 'convencional' THEN 'Convencional' ELSE 'Eléctrico TuBus' END AS tipo_unidad,
       b.capacidad_maxima,
       b.pasajeros_a_bordo,
       b.ocupacion_actual_pct,
       CASE WHEN l.id_linea IS NOT NULL AND b.ocupacion_actual_pct < 25
            THEN '+5 min por estación' ELSE '—' END AS espera_extra,
       b.estado_logico,
       e.nombre || ' · ' || p.codigo_bahia AS parqueo,
       emp.nombre_completo AS piloto,
       pi.tipo_licencia || ' ' || pi.numero_licencia AS licencia
  FROM buses b
  LEFT JOIN lineas l ON l.id_linea = b.id_linea_asignada
  LEFT JOIN parqueos p ON p.id_bus_asignado = b.id_bus
  LEFT JOIN estaciones e ON e.id_estacion = p.id_estacion
  LEFT JOIN pilotos pi ON pi.id_bus_asignado = b.id_bus
  LEFT JOIN empleados emp ON emp.id_empleado = pi.id_empleado
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY (l.id_linea IS NULL), substring(l.codigo_linea from '^[A-Za-z]*'),
coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '', 'g'), '')::int, 0), b.placa;

-- ========================================================================
-- Recorridos y distancias (Puntos 1, 12 y 13)
-- Estaciones de cada línea en orden de recorrido, con distancia entre paradas y acumulada.
-- ========================================================================
SELECT l.codigo_linea AS linea,
       le.orden_parada,
       e.nombre AS estacion,
       e.direccion,
       le.distancia_previa_km,
       sum(le.distancia_previa_km) OVER (PARTITION BY l.id_linea ORDER BY le.orden_parada) AS km_acumulado
  FROM linea_estacion le
  JOIN lineas l ON l.id_linea = le.id_linea
  JOIN estaciones e ON e.id_estacion = le.id_estacion
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'), coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '',
'g'), '')::int, 0), le.orden_parada;

-- ========================================================================
-- Viajes de las unidades (Puntos 1 y 15)
-- Paradas de cada viaje con hora de llegada y salida, tiempo detenido, ocupación, descensos y abordajes.
-- ========================================================================
SELECT l.codigo_linea AS linea,
       b.placa AS bus,
       v.fecha_hora_inicio AS inicio_viaje,
       v.estado AS estado_viaje,
       vv.orden_visita,
       e.nombre AS estacion,
       vv.hora_llegada,
       vv.hora_salida,
       round(extract(epoch FROM (vv.hora_salida - vv.hora_llegada)) / 60, 1) AS minutos_detenido,
       vv.ocupacion_al_llegar_pct,
       vv.bajaron,
       vv.subieron
  FROM visitas_viaje vv
  JOIN viajes v ON v.id_viaje = vv.id_viaje
  JOIN buses b ON b.id_bus = v.id_bus
```

```
  JOIN lineas l ON l.id_linea = v.id_linea
  JOIN estaciones e ON e.id_estacion = vv.id_estacion
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY v.fecha_hora_inicio DESC, b.placa, vv.orden_visita
 LIMIT 500;

-- ========================================================================
-- Abordajes y descensos por estación (Puntos 1, 14, 15 y 18)
-- Abordajes, descensos y recaudación del día por línea y estación, y abordajes de los últimos 7 días.
-- ========================================================================
WITH hoy AS (SELECT (now() AT TIME ZONE 'America/Guatemala')::date AS dia)
SELECT l.codigo_linea AS linea,
       le.orden_parada,
       e.nombre AS estacion,
       (SELECT count(*) FROM transacciones_pago t
         WHERE t.id_estacion = e.id_estacion AND t.id_linea = l.id_linea
           AND t.fecha_hora >= (SELECT dia FROM hoy)) AS subieron_hoy,
       (SELECT coalesce(sum(bj.cantidad), 0) FROM bajadas_bus bj
          JOIN visitas_viaje vv ON vv.id_visita = bj.id_visita
          JOIN viajes v ON v.id_viaje = vv.id_viaje
         WHERE bj.id_estacion = e.id_estacion AND v.id_linea = l.id_linea
           AND bj.fecha_hora >= (SELECT dia FROM hoy)) AS bajaron_hoy,
       (SELECT coalesce(sum(t.monto), 0) FROM transacciones_pago t
         WHERE t.id_estacion = e.id_estacion AND t.id_linea = l.id_linea
           AND t.fecha_hora >= (SELECT dia FROM hoy)) AS recaudado_hoy,
       (SELECT count(*) FROM transacciones_pago t
         WHERE t.id_estacion = e.id_estacion AND t.id_linea = l.id_linea
           AND t.fecha_hora >= (SELECT dia FROM hoy) - 6) AS subieron_7_dias
  FROM linea_estacion le
  JOIN lineas l ON l.id_linea = le.id_linea
  JOIN estaciones e ON e.id_estacion = le.id_estacion
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'), coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '',
'g'), '')::int, 0), le.orden_parada;

-- ========================================================================
-- Verificación de cobros (Puntos 15 y 18)
-- Comprueba que cada persona que subió hoy a un bus pagó su pasaje: personas que subieron contra pagos registrados
en el molinete de entrada, y lo cobrado contra la tarifa.
-- ========================================================================
WITH hoy AS (SELECT (now() AT TIME ZONE 'America/Guatemala')::date AS dia),
-- Personas que subieron en cada parada de hoy (lo cuenta la visita del bus a la estación)
visitas AS (
    SELECT vj.id_linea, vv.id_visita, vv.subieron
      FROM visitas_viaje vv
      JOIN viajes vj ON vj.id_viaje = vv.id_viaje
     WHERE vv.hora_llegada::date = (SELECT dia FROM hoy)
),
-- Pagos del molinete de entrada hechos en esas paradas
pagos AS (
    SELECT v.id_linea,
           count(t.*) AS pagos,
           count(t.*) FILTER (WHERE t.metodo_pago = 'tarjeta_ciudadana') AS tarjeta_ciudadana,
           count(t.*) FILTER (WHERE t.metodo_pago = 'tarjeta_bancaria') AS tarjeta_bancaria,
           count(t.*) FILTER (WHERE t.viaje_gratis) AS gratis,
           coalesce(sum(t.monto), 0) AS cobrado
      FROM visitas v
      JOIN transacciones_pago t ON t.id_visita = v.id_visita
     GROUP BY v.id_linea
)
SELECT l.codigo_linea AS linea,
       coalesce(s.subieron, 0) AS subieron,
       coalesce(p.pagos, 0) AS pagos,
       coalesce(s.subieron, 0) - coalesce(p.pagos, 0) AS sin_pago,
       coalesce(p.tarjeta_ciudadana, 0) AS tarjeta_ciudadana,
       coalesce(p.tarjeta_bancaria, 0) AS tarjeta_bancaria,
       coalesce(p.gratis, 0) AS gratis,
       coalesce(p.cobrado, 0) AS cobrado,
       (coalesce(p.pagos, 0) - coalesce(p.gratis, 0)) * l.tarifa AS esperado,
       CASE WHEN coalesce(s.subieron, 0) = coalesce(p.pagos, 0)
             AND coalesce(p.cobrado, 0) = (coalesce(p.pagos, 0) - coalesce(p.gratis, 0)) * l.tarifa
            THEN 'Cuadra' ELSE 'No cuadra' END AS verificacion
  FROM lineas l
  LEFT JOIN (SELECT id_linea, sum(subieron) AS subieron FROM visitas GROUP BY id_linea) s ON s.id_linea =
l.id_linea
  LEFT JOIN pagos p ON p.id_linea = l.id_linea
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'), coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '',
'g'), '')::int, 0);
```

10. Diagramas UML

10.1. Diagrama de Casos de Uso

Representa las interacciones entre los actores del sistema (Administrador/Municipalidad, Operador de Estación, Pasajero y el propio Sistema de forma automática) y las funcionalidades que cada uno puede ejecutar, agrupadas por módulo. Los casos de uso del piloto y del personal de seguridad se muestran como fase 2.

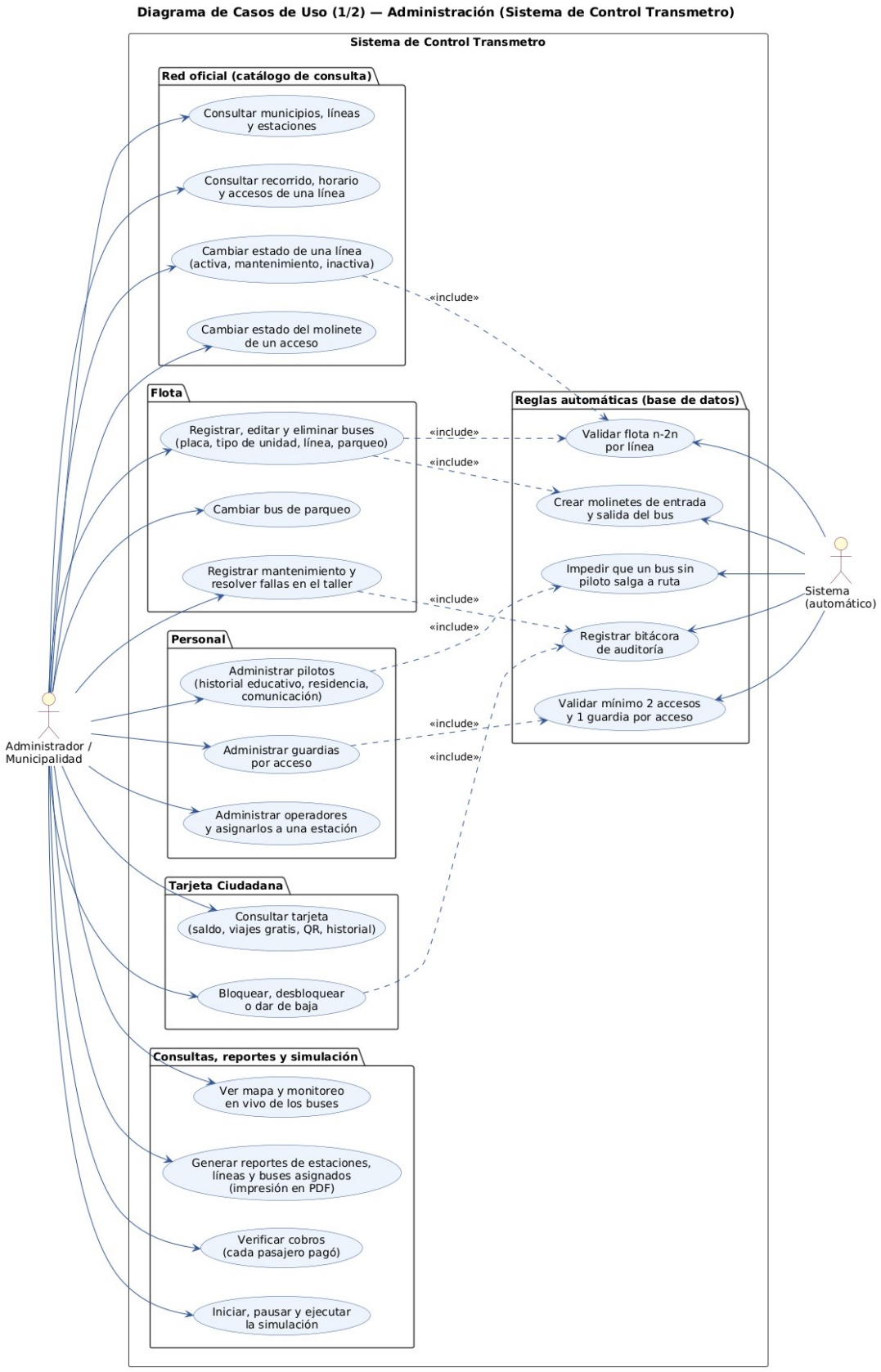


Figura 3. Casos de uso (1/2) — Administración de la red, flota, personal, tarjetas, reportes y simulación.

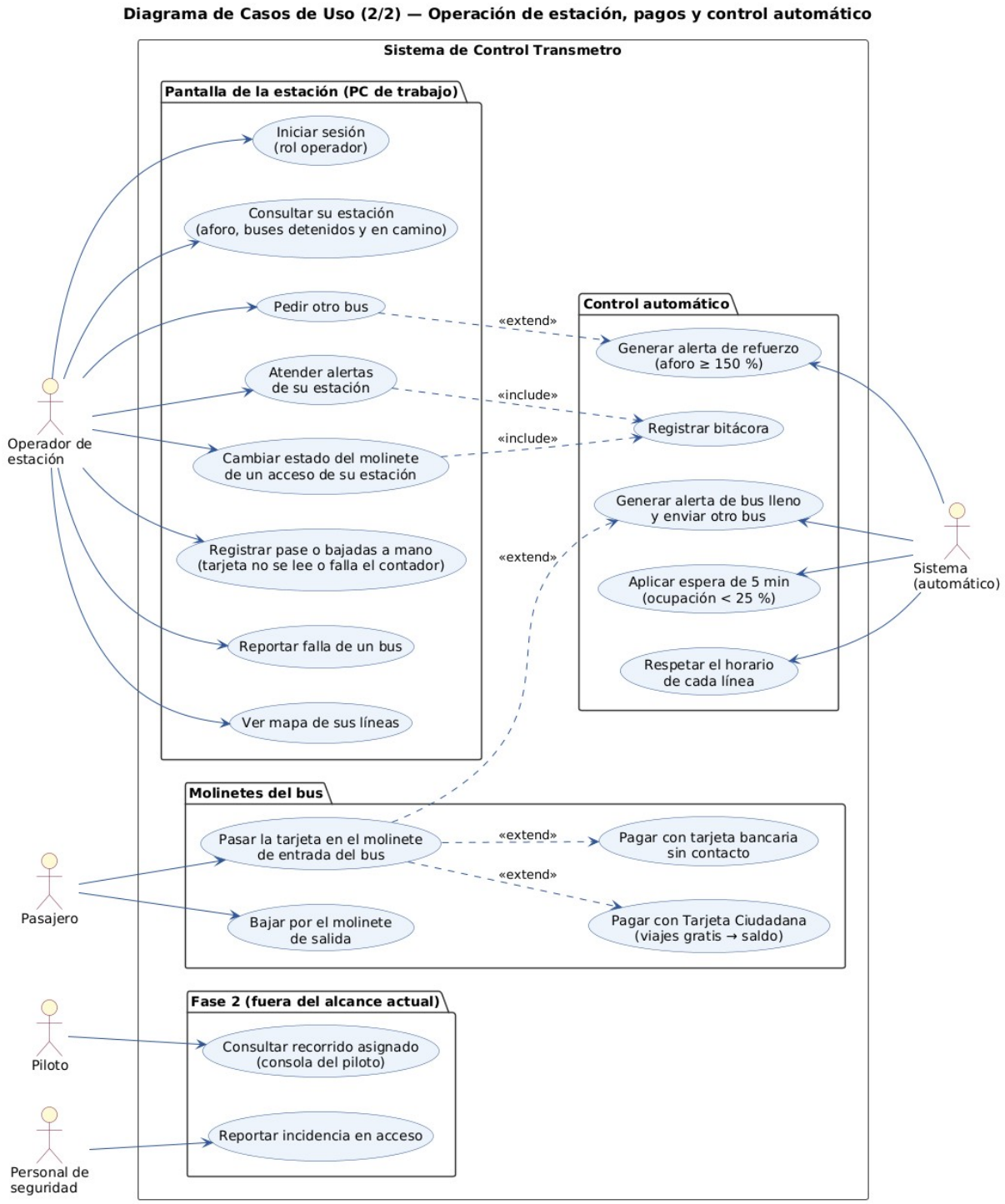


Figura 4. Casos de uso (2/2) — Operación de estación, pagos y control automático.

10.2. Diagrama de Componentes

Muestra la organización física y lógica de la solución: el navegador de la PC de cada estación se comunica por HTTPS con la aplicación Laravel alojada en Render, y solo la aplicación se conecta a la base de datos PostgreSQL en Supabase. Dentro de Laravel se distinguen la autenticación por roles, el módulo administrativo, la pantalla del operador, la simulación, el mapa con el monitoreo en vivo, los reportes y el servicio de cobro. El mapa usa teselas de OpenStreetMap y GitHub Actions ejecuta las pruebas en cada cambio (ver RNF-11 y la sección 6.1).

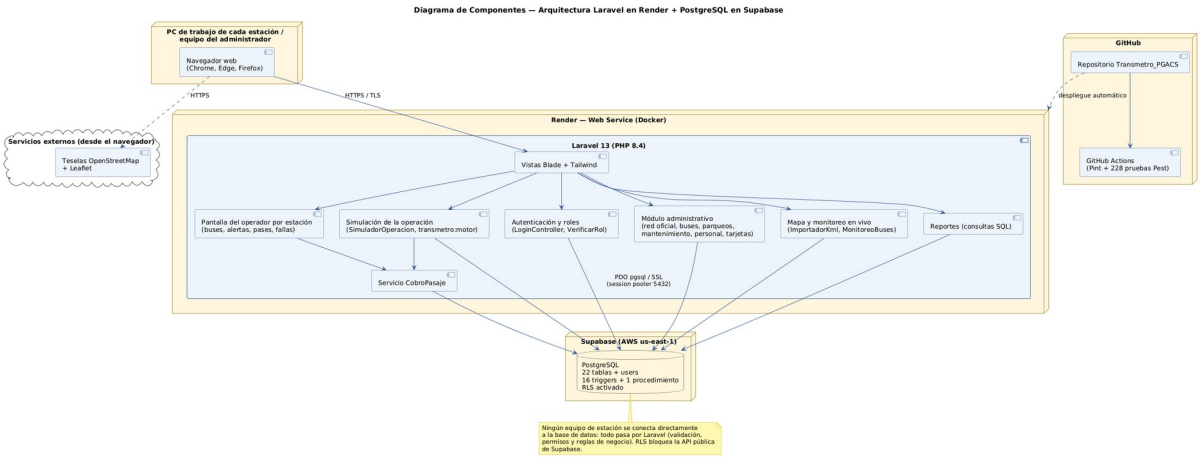


Figura 5. Diagrama de componentes de la arquitectura del sistema.

10.3. Diagrama de Clases

Traduce a nivel orientado a objetos el Modelo Entidad Relación, incluyendo la generalización de Empleado en Piloto, Seguridad y Operador, las clases agregadas durante el desarrollo (HorarioLinea, Tarjeta, Recarga, Usuario, MantenimientoBus y BajadaBus), los molinetes de entrada y salida de cada bus, la relación de un operador por estación, los servicios CobroPasaje y SimuladorOperacion y las multiplicidades tal como están implementadas en las llaves foráneas y restricciones del esquema.

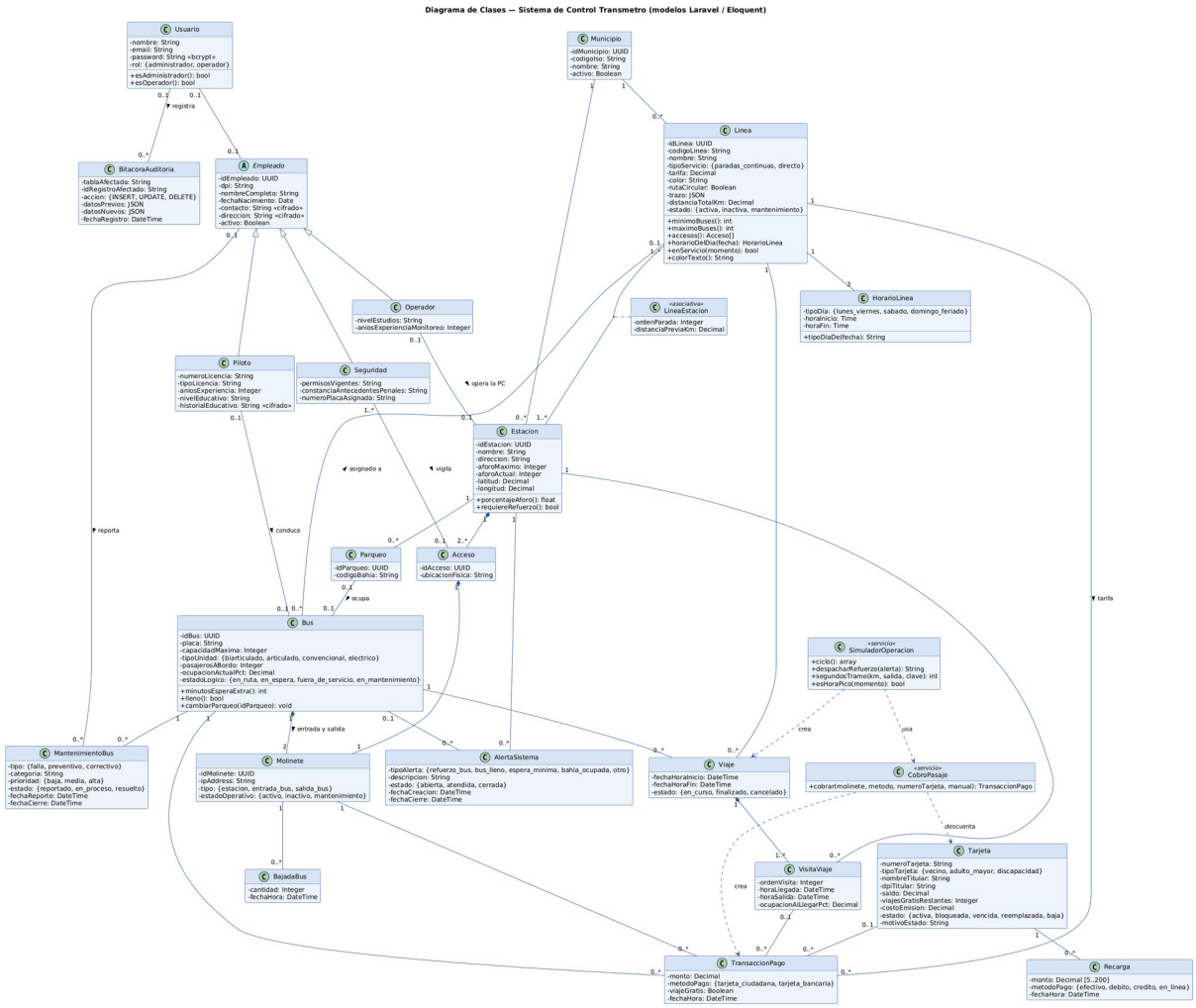


Figura 6. Diagrama de clases del Sistema de Control Transmetro.

10.4. Diagrama de Objetos

Presenta una instancia concreta del diagrama de clases con datos reales cargados en el sistema: el transbordo en la estación San Sebastián entre las Líneas 1 y 2, el bus C100JVX con baja ocupación, un acceso con su guardia y molinete, el operador de la estación, un pase con viaje gratis de la Tarjeta Ciudadana registrado en el molinete de entrada del bus y una alerta de refuerzo abierta porque la estación supera el 150 % de su aforo.

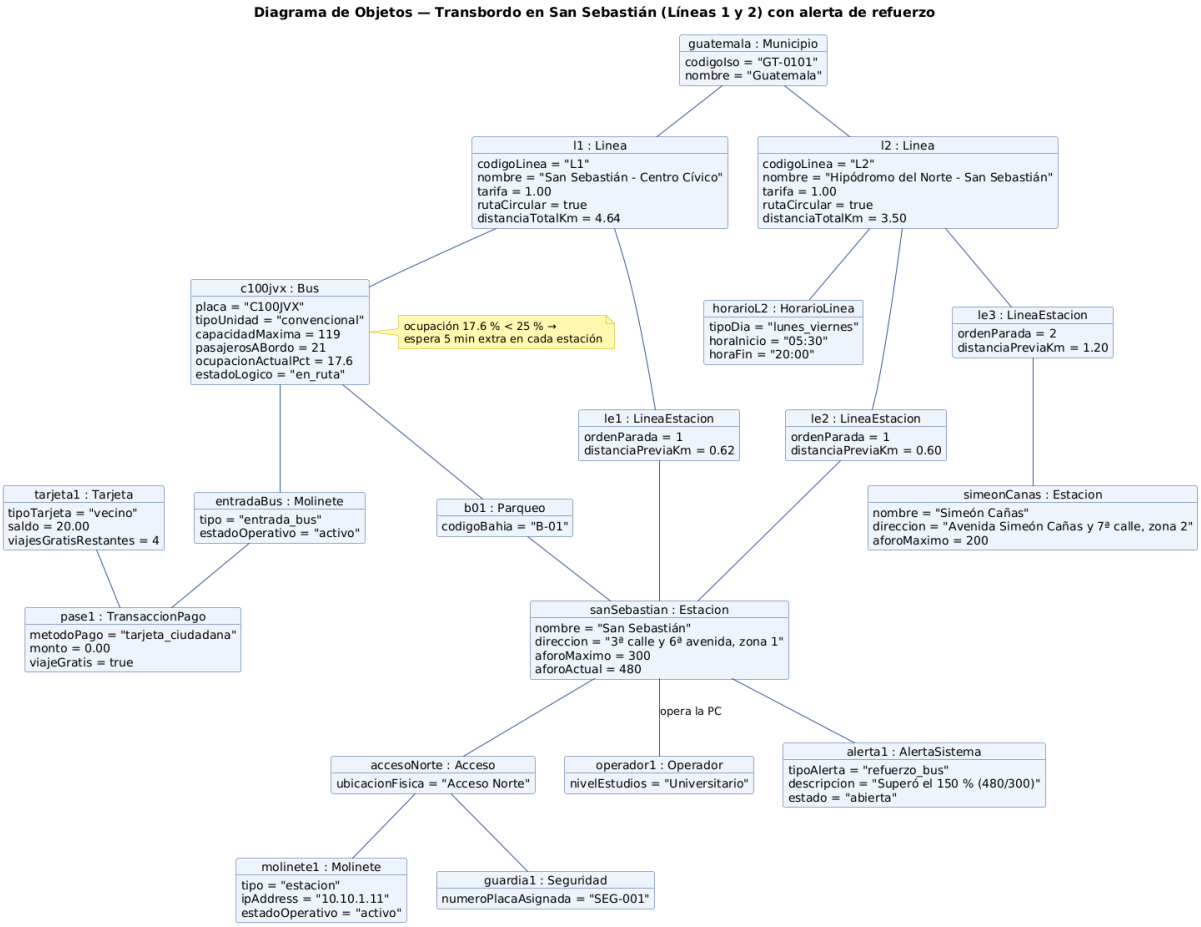


Figura 7. Diagrama de objetos — transbordo en San Sebastián (Líneas 1 y 2).

10.5. Diagrama de Secuencia

Detalla la interacción entre el pasajero, el molinete de entrada del bus, la aplicación Laravel con el servicio CobroPasaje, la base de datos y el operador de estación durante el pago de un pasaje con Tarjeta Ciudadana (primero los viajes gratis y después el saldo), el rechazo cuando el bus va lleno, y la alerta de refuerzo que se genera cuando la estación se satura y que el operador atiende enviando otro bus.

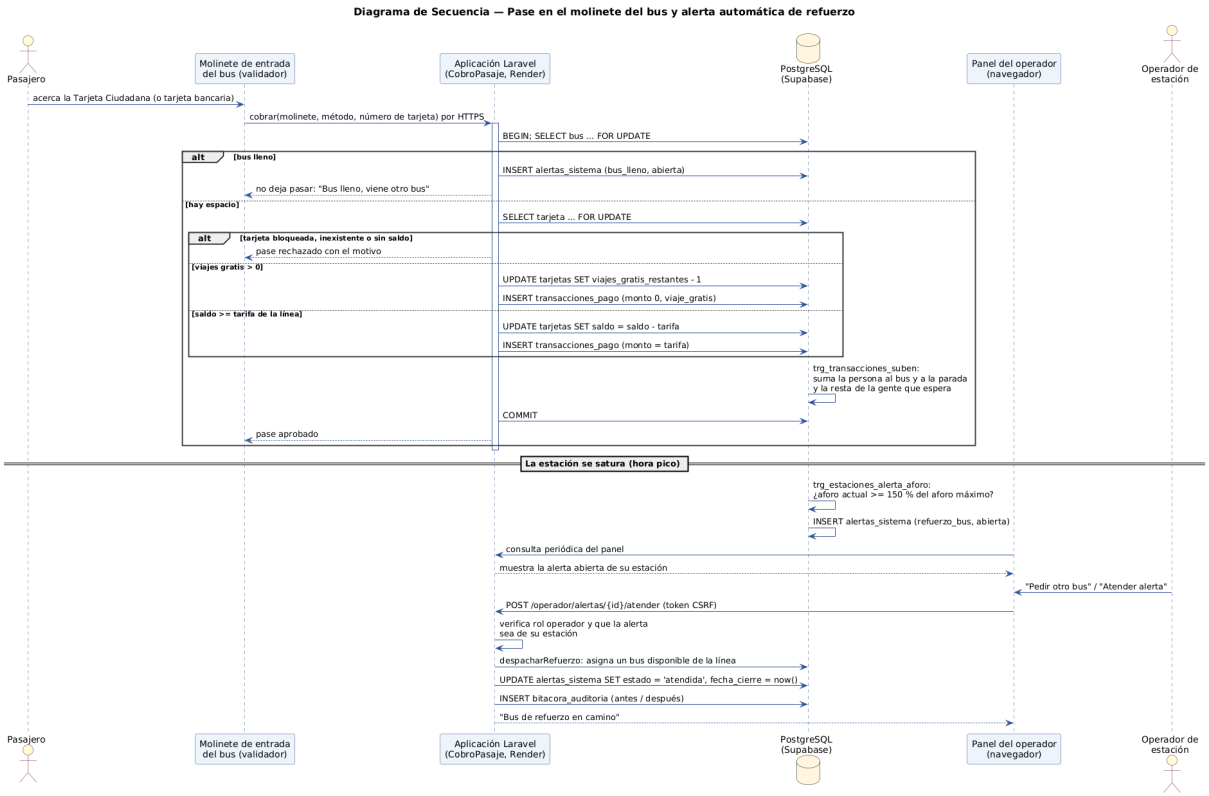


Figura 8. Secuencia del pago en el molinete del bus y de la alerta automática de refuerzo.

10.6. Diagrama de Actividades

Describe, mediante carriles (swimlanes) de Administrador y Sistema (Backend), el flujo completo de asignación de un bus a una línea: el rechazo de asignaciones duplicadas, la validación del límite de flota que realiza el trigger trg_buses_max_flota, la verificación del parqueo obligatorio con sp_cambiar_parqueo_bus y la presentación del bus en el listado y en el reporte de buses asignados.

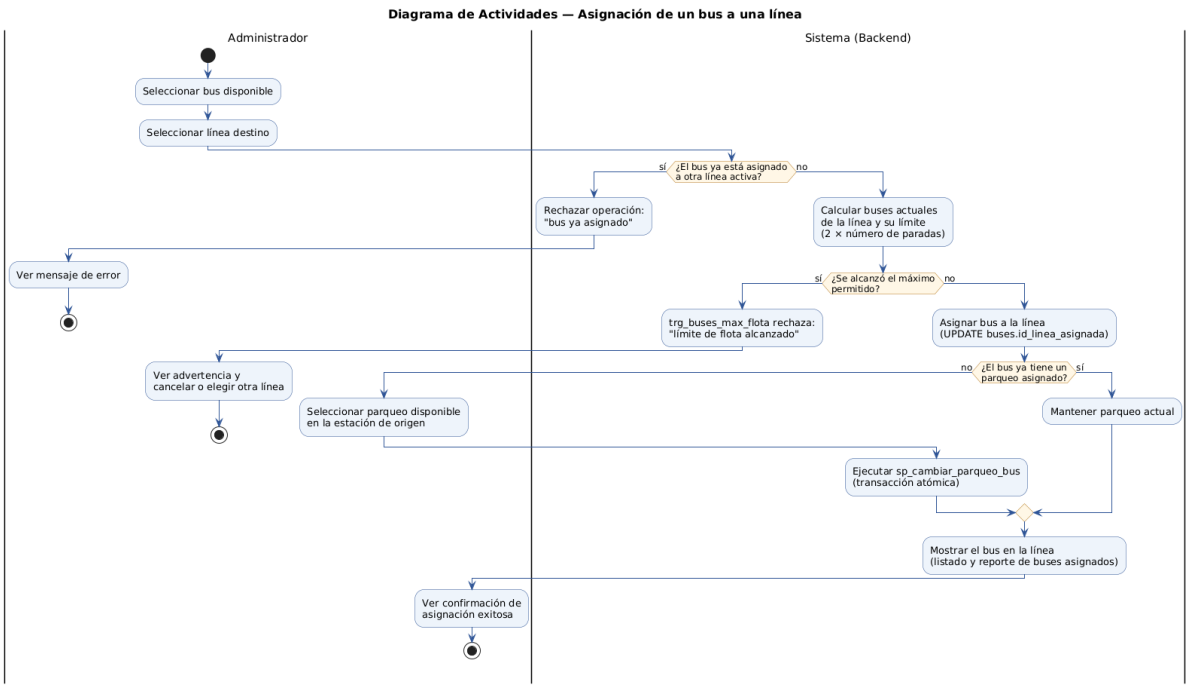


Figura 9. Actividades del proceso de asignación de un bus a una línea.

11. Presentación de Stack Tecnológico

Para desarrollar el Sistema de Control Transmetro se seleccionó un conjunto de tecnologías que permite construir una aplicación web segura, organizada, escalable y fácil de mantener, capaz de operar desde la PC de trabajo de cada estación y de crecer conforme se incorporen nuevas líneas y municipalidades. El stack tecnológico comprende la base de datos, la metodología de trabajo, el lenguaje de programación, el framework, el entorno de desarrollo integrado, la plataforma de despliegue y las herramientas de calidad que acompañan todo el ciclo de vida del proyecto, desde el análisis hasta la operación.

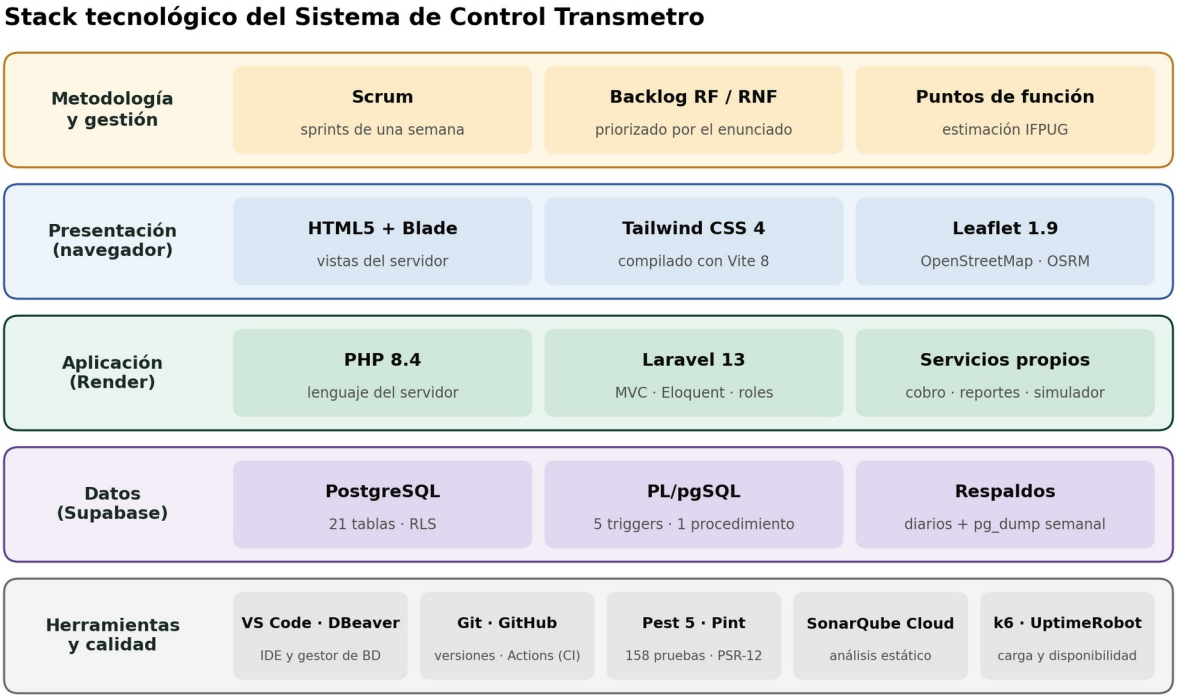


Figura 10. Stack tecnológico del Sistema de Control Transmetro.

11.1. Base de datos: PostgreSQL

Como sistema gestor de base de datos se utilizará **PostgreSQL**, un motor relacional de código abierto reconocido por su cumplimiento del estándar SQL, su manejo de transacciones y su capacidad para aplicar reglas de integridad dentro de la propia base de datos. PostgreSQL almacenará y administrará toda la información del sistema: municipalidades, líneas, horarios, estaciones, accesos, molinetes, parqueos, buses, viajes, personal, tarjetas, recargas, pagos, alertas, usuarios y bitácora de auditoría.

La base de datos está compuesta por 23 tablas relacionadas mediante llaves primarias de tipo UUID y llaves foráneas. Las tablas se organizan en seis grupos según su función:

- **Estructura territorial y rutas:** municipios, lineas, horarios_linea, estaciones y linea_estacion (orden de las paradas y distancia entre ellas).
- **Infraestructura de estaciones:** accesos, molinetes (de estación y de cada bus) y parqueos.

- **Flota y operación:** buses, viajes, visitas_viaje, bajadas_bus, mantenimientos_bus y alertas_sistema.
- **Personal:** empleados, pilotos, seguridad y operadores.
- **Pagos:** tarjetas, recargas y transacciones_pago.
- **Seguridad y auditoría:** users y bitacora_auditoria.

Las reglas del enunciado se aplicarán directamente en la base de datos, de modo que ninguna operación pueda dejar la información en un estado inválido, aunque se intente modificarla fuera de la aplicación:

- Restricciones CHECK para valores válidos: recargas de Q5.00 a Q200.00, ocupación de 0 a 100 %, estados permitidos y métodos de pago sin efectivo.
- Trigger que impide que una estación quede con menos de dos accesos y otro que impide que un acceso quede sin guardia de seguridad.
- Trigger que rechaza asignar a una línea más del doble de buses que estaciones tiene.
- Trigger que genera automáticamente la alerta de refuerzo cuando una estación supera el 150 % de su aforo.
- Trigger que recalcula la distancia total de una línea a partir de la distancia entre sus estaciones.
- Triggers que crean los molinetes de entrada y salida de cada bus, cuentan las subidas y bajadas, calculan la ocupación e impiden que un bus sin piloto salga a ruta o que una tarjeta bloqueada pague o se recargue.
- Procedimiento almacenado sp_cambiar_parqueo_bus, que cambia el parqueo de un bus en una sola transacción sin que quede sin parqueo.

Se eligió PostgreSQL frente a MySQL y SQL Server porque ofrece triggers y procedimientos en PL/pgSQL, tipos JSONB para guardar el trazo de las rutas, bloqueo de filas para evitar cobros duplicados y seguridad por fila (Row Level Security), sin costo de licencia. La base de datos se alojará en **Supabase**, un servicio que administra PostgreSQL en la nube e incluye conexión cifrada por SSL, respaldos diarios con siete días de retención y monitoreo. Como medida adicional se realizará un respaldo semanal con pg_dump almacenado fuera de Supabase.

11.2. Metodología de desarrollo: Scrum

El proyecto se desarrollará con **Scrum**, un marco de trabajo ágil que organiza el trabajo en ciclos cortos llamados sprints. Al final de cada sprint se entrega una parte funcional del sistema que se revisa con el Product Owner, lo que permite incorporar cambios en los requerimientos, como la información oficial de líneas y horarios o las reglas de la Tarjeta Ciudadana, sin rehacer el trabajo terminado.

Los roles de Scrum se asignarán de la siguiente manera:

- **Product Owner:** representante de la municipalidad, encargado de definir y priorizar los requerimientos a partir de los puntos del enunciado.
- **Scrum Master:** la consultora, responsable de organizar los sprints, eliminar impedimentos y velar por la calidad del proceso.
- **Equipo de desarrollo:** la consultora, encargada de analizar, programar, probar y documentar el sistema.

Los artefactos serán el **Product Backlog**, formado por los requerimientos funcionales y no funcionales; el **Sprint Backlog**, con las tareas seleccionadas para cada sprint; y el **Incremento**, que es la versión del sistema con todas sus pruebas aprobadas. En cada sprint se realizarán la planificación, el seguimiento diario, la revisión con demostración del sistema y la retrospectiva.

Una tarea se considerará terminada cuando su código cumpla el estándar de formato, tenga pruebas automatizadas aprobadas, esté relacionada con un requerimiento en la matriz de trazabilidad y su documentación esté actualizada. El proyecto se organizará en ocho sprints alineados con el calendario de entregas, del 3 de agosto al 23 de octubre de 2026:

Tabla 11.1. Planificación de sprints del proyecto.

Sprint	Fechas	Objetivo del sprint
1	3 – 28 ago	Carta de presentación, antecedentes, situación actual, deficiencias y planificación
2	31 ago – 4 sep	Análisis de requerimientos funcionales y no funcionales
3	7 – 11 sep	Estudios de factibilidad y propuesta monetaria
4	14 – 18 sep	Modelo entidad relación, script de la base de datos y diagramas UML; inicio del desarrollo
5	21 sep – 2 oct	Stack tecnológico y métricas de software; desarrollo de los módulos principales
6	5 – 10 oct	Aplicación web completa y demo funcional
7	12 – 17 oct	Plan de pruebas, criterios de aceptación y carta de aceptación
8	19 – 23 oct	Documento final, presentación ejecutiva, video y entrega

11.3. Lenguaje de programación: PHP 8.4

El lenguaje principal será **PHP 8.4**, un lenguaje del lado del servidor diseñado para aplicaciones web, con tipado estricto, enumeraciones y un amplio ecosistema de librerías. Esta versión tiene soporte activo hasta el 31 de diciembre de 2026 y actualizaciones de seguridad hasta el 31 de diciembre de 2028, lo que cubre el tiempo de desarrollo y los primeros años de operación del sistema.

PHP se complementará con otros lenguajes, cada uno en su capa:

- **SQL y PL/pgSQL:** estructura de la base de datos, triggers, procedimiento almacenado y consultas de los reportes.
- **JavaScript:** mapa de las líneas, editor del trazo de las rutas y actualización periódica del panel de la estación.
- **HTML5 y CSS:** estructura y diseño de las pantallas, con plantillas Blade y estilos de Tailwind CSS.

11.4. Framework: Laravel 13

Para construir la aplicación se utilizará **Laravel 13**, el framework de PHP más utilizado para aplicaciones web. Laravel 13 se publicó el 17 de marzo de 2026, recibe correcciones hasta el tercer trimestre de 2027 y actualizaciones de seguridad hasta el 17 de marzo de 2028. Laravel organiza la aplicación con el patrón **Modelo – Vista – Controlador (MVC)**:

- **Modelo:** representa las entidades del sistema (Línea, Estación, Bus, Tarjeta, Alerta, entre otras) y sus relaciones.
- **Vista:** presenta la información al usuario mediante plantillas Blade.
- **Controlador:** recibe las solicitudes del usuario, valida los datos y coordina la respuesta.

Cuando un operador registra información, la ruta verifica que haya iniciado sesión y que tenga el rol correcto, el controlador valida los datos, la capa de servicios aplica las reglas de negocio (por ejemplo, el cobro del pasaje con viajes gratis y saldo) y el modelo guarda el registro, momento en que la base de datos verifica sus propias reglas.

También se utilizará **Eloquent ORM**, el componente de persistencia de Laravel. Su función será comunicar la aplicación con PostgreSQL mediante clases y objetos de PHP, reduciendo la necesidad de escribir consultas SQL manualmente para las operaciones comunes; los reportes usarán consultas SQL preparadas para documentarlas en este informe. Otros componentes de Laravel que se aprovecharán son:

- Migraciones para crear y versionar la estructura de la base de datos.
- Middleware de roles para separar las funciones del administrador y del operador.
- Validación de formularios con mensajes en español.
- Cifrado de contraseñas con bcrypt, protección CSRF y cifrado AES-256 de los datos personales sensibles.
- Comandos Artisan propios para tareas como la simulación de la operación y la exportación de las consultas de los reportes.

En la capa de presentación se usará **Tailwind CSS 4** compilado con **Vite**, y **Leaflet** con mapas de **OpenStreetMap** para mostrar las líneas y estaciones del Transmetro.

11.5. IDE: Visual Studio Code

El entorno de desarrollo será **Visual Studio Code**, un editor gratuito y liviano en el que se crearán los modelos, vistas, controladores, servicios, migraciones y pruebas del proyecto. Visual Studio Code proporciona funciones como:

- Editor de código con resaltado de sintaxis para PHP, Blade, SQL y JavaScript.
- Autocompletado y detección de errores mediante extensiones (PHP Intelephense, Laravel y Tailwind CSS IntelliSense).
- Terminal integrada para ejecutar Composer, Artisan, npm y las pruebas.
- Integración con Git y GitHub para el control de versiones.
- Conexión a PostgreSQL mediante extensiones de bases de datos.

Como herramientas complementarias se utilizarán **DBeaver** para diseñar y consultar la base de datos y generar el diagrama entidad relación, **Git y GitHub** para el control de versiones y el repositorio del proyecto, y **PlantUML** para los diagramas UML.

11.6. Plataforma de despliegue: Render y Supabase

La aplicación se publicará en **Render**, una plataforma en la nube que ejecutará el sistema dentro de un contenedor Docker con certificado HTTPS. Se utilizará el plan Starter, que mantiene la

aplicación activa de forma permanente. La base de datos se alojará en **Supabase** con el plan Pro, que evita las pausas por inactividad e incluye respaldos diarios.

Cada PC de estación se conectará al sistema por Internet mediante HTTPS, por lo que no será necesario construir una red privada entre estaciones; una estación nueva solo necesitará un navegador y conexión a Internet. Esta arquitectura responde al punto 17 del enunciado, que indica que aún no se ha definido la mejor forma de conectar las máquinas.

11.7. Herramientas de calidad

Para asegurar la calidad del software durante todo el proyecto se utilizarán las siguientes herramientas:

- **Pest:** pruebas automatizadas unitarias y de funcionalidad que verifican cada regla del enunciado.
- **Laravel Pint:** revisión del formato del código según el estándar PSR-12.
- **GitHub Actions:** integración continua; en cada cambio revisa el formato y ejecuta todas las pruebas sobre una base PostgreSQL limpia antes de publicar.
- **SonarQube Cloud:** análisis estático para detectar errores, vulnerabilidades, código duplicado y deuda técnica.
- **k6:** pruebas de carga para medir el tiempo de respuesta con varios usuarios simultáneos.
- **UptimeRobot:** monitoreo de la disponibilidad del sistema en producción.

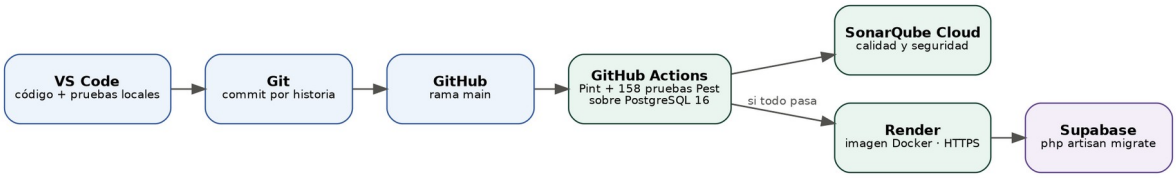


Figura 11. Flujo de integración y despliegue continuo.

11.8. Integración del stack tecnológico

El usuario accederá al sistema desde un navegador web en la PC de su estación o de la municipalidad. La solicitud viajará cifrada por HTTPS hasta la aplicación desarrollada con Laravel y PHP en Render. Las rutas y el middleware verificarán la sesión y el rol, los controladores validarán los datos y los servicios aplicarán las reglas de negocio. Posteriormente, Eloquent realizará las consultas o modificaciones en PostgreSQL, donde los triggers y restricciones aplicarán las reglas de integridad y generarán las alertas automáticas. La respuesta regresará al navegador como una página construida con Blade y Tailwind CSS.

El desarrollo se realizará en Visual Studio Code, las actividades se organizarán mediante Scrum, cada cambio será verificado por GitHub Actions y, finalmente, la aplicación se publicará en Render con la base de datos en Supabase.

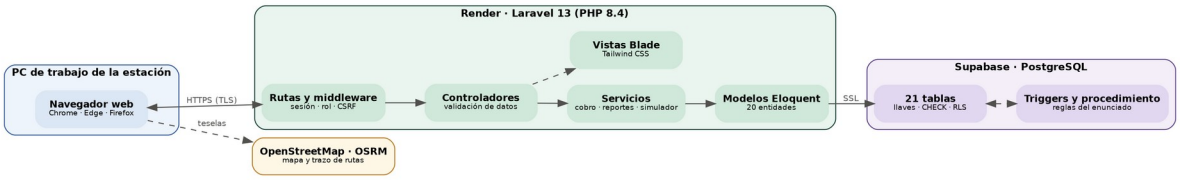


Figura 12. Integración de los componentes del stack durante una solicitud.

Tabla 11.2. Resumen del stack tecnológico.

Componente	Tecnología	Versión	Función en el proyecto	Costo
Base de datos	PostgreSQL en Supabase Pro	15 o superior	Almacenar la información y aplicar las reglas de integridad	US$25 al mes
Metodología	Scrum	—	Organizar el trabajo en sprints alineados con las entregas	Sin costo
Lenguaje	PHP	8.4	Lógica del servidor	Sin costo
Framework	Laravel	13	Estructura MVC, seguridad y acceso a datos	Sin costo
IDE	Visual Studio Code	Estable	Desarrollo del código	Sin costo
Gestor de BD	DBeaver Community	26.2	Diseño y consulta de la base de datos	Sin costo
Interfaz	Blade, Tailwind CSS, Leaflet	— / 4 / 1.9	Pantallas y mapa de rutas	Sin costo
Control de versiones	Git y GitHub	—	Repositorio e integración continua	Sin costo
Calidad	Pest, Pint, SonarQube Cloud, k6	—	Pruebas, formato, análisis y carga	Sin costo
Despliegue	Render Starter	—	Publicación de la aplicación con HTTPS	US$7 al mes

Tabla 11.3. Alternativas evaluadas y motivo de la selección.

Componente	Seleccionado	Alternativas evaluadas	Motivo de la selección
Base de datos	PostgreSQL	MySQL, SQL Server	Reglas en la base de datos con PL/pgSQL, JSONB y seguridad por fila, sin licencia y con hospedaje administrado accesible
Metodología	Scrum	Kanban, Cascada	Entregas frecuentes para revisión y adaptación a cambios en los requerimientos
Lenguaje y framework	PHP con Laravel	Python con Django, Java con Spring Boot	Mayor productividad para aplicaciones web, seguridad incluida y dominio previo del equipo
IDE	Visual Studio Code	PhpStorm, NetBeans	Gratuito, liviano y con extensiones para todo el stack
Despliegue	Render y Supabase	Servidor propio, Azure, AWS	Sin administración de servidores, costo mensual bajo y HTTPS incluido

12. Métricas de Software

En el Sistema de Control Transmetro se utilizarán métricas de proceso, de proyecto y de producto, evaluadas en cuatro dimensiones: eficacia, velocidad, calidad e impacto. Estas métricas se aplicarán durante todo el ciclo de vida del sistema: en la planificación, en cada sprint, en las pruebas finales y durante la operación. Permiten conocer el avance del proyecto, detectar problemas a tiempo, comprobar el cumplimiento de los requerimientos y demostrar el beneficio que el sistema genera para la municipalidad.

12.1. Objetivo general

Definir y evaluar indicadores que permitan medir la eficacia, la velocidad, la calidad y el impacto del Sistema de Control Transmetro, desde su desarrollo hasta su operación.

12.2. Objetivos específicos

- Medir la eficacia del proceso de desarrollo y el grado de cumplimiento de los requerimientos del enunciado.
- Controlar la velocidad de entrega del equipo y el cumplimiento del cronograma y del presupuesto.
- Evaluar la calidad del producto mediante su tamaño, complejidad, defectos, rendimiento, confiabilidad, seguridad y usabilidad.
- Determinar el impacto del sistema en la operación del Transmetro y en la gestión de la municipalidad.
- Establecer metas, herramientas y responsables para la recolección de cada métrica.

12.3. Marco conceptual

Una **medida** es un valor cuantitativo de un atributo (por ejemplo, el número de defectos); una **métrica** relaciona medidas para obtener información (defectos por cada mil líneas de código); y un **indicador** es una métrica con una meta que permite tomar decisiones. Según Pressman, las métricas de software se agrupan en tres tipos según lo que miden:

- **Métricas de proceso:** miden la forma en que se desarrolla el software para mejorar el proceso en este y en futuros proyectos. Son estratégicas y de largo plazo; por ejemplo, la eficiencia de remoción de defectos o el tiempo de ciclo de una tarea.
- **Métricas de proyecto:** permiten al equipo controlar el proyecto en curso: avance, esfuerzo, costo, cronograma y riesgos. Son tácticas; por ejemplo, la velocidad del sprint o el índice de desempeño del cronograma.
- **Métricas de producto:** miden las características del software entregado: tamaño, complejidad, calidad, rendimiento, confiabilidad, seguridad y facilidad de uso, alineadas con el estándar ISO/IEC 25010.

Cada métrica se evalúa además en una de cuatro dimensiones, que responden a las preguntas que la municipalidad y el equipo necesitan contestar:

Tabla 12.1. Dimensiones de evaluación de las métricas.

Dimensión	Pregunta que responde	Ejemplos de métricas
Eficacia	¿Se logra lo que se planificó y lo que pide el enunciado?	Requerimientos cumplidos, avance funcional, cumplimiento del sprint, eficiencia de remoción de defectos
Velocidad	¿Qué tan rápido se entrega y qué tan rápido responde el sistema?	Velocidad del equipo, tiempo de ciclo, frecuencia de despliegue, tiempo de respuesta
Calidad	¿El software es correcto, mantenible, seguro y confiable?	Densidad de defectos, cobertura de pruebas, complejidad, vulnerabilidades, disponibilidad
Impacto	¿Qué beneficio genera el sistema para la municipalidad y los usuarios?	Tiempo de atención de alertas, horas administrativas ahorradas, satisfacción de usuarios, VAN

Para seleccionar las métricas se usó el enfoque **GQM (Objetivo – Pregunta – Métrica)**: cada métrica responde a una pregunta derivada de un objetivo del proyecto. Las metas se clasifican con un semáforo: **verde** cuando se cumple la meta, **amarillo** cuando el valor está a menos del 10 % de la meta y requiere una acción de mejora en el siguiente sprint, y **rojo** cuando no se cumple y debe corregirse antes de cerrar el sprint.

12.4. Métricas de proceso

Evalúan cómo trabaja el equipo con Scrum y con el flujo de integración continua. Se basan en las prácticas de medición de entrega de software de DORA (frecuencia de despliegue, tiempo de entrega de cambios, tasa de fallos y tiempo de recuperación) y en las métricas de calidad del proceso de Pressman.

Tabla 12.2. Métricas de proceso.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Eficiencia de remoción de defectos (DRE)	Calidad	DRE = E ÷ (E + D), donde E son los defectos encontrados antes de la entrega y D los encontrados después	≥ 95 %	Issues de GitHub
Eficacia de las pruebas automatizadas	Eficacia	Defectos detectados por pruebas ÷ defectos totales × 100	≥ 70 %	Issues de GitHub, Pest
Cumplimiento de la definición de terminado	Eficacia	Tareas que cumplen formato, pruebas, trazabilidad y documentación ÷ tareas terminadas × 100	100 %	GitHub Actions, tablero Scrum
Tiempo de ciclo	Velocidad	Días desde que una tarea inicia hasta que está terminada	≤ 3 días	Tablero Scrum
Tiempo de entrega de cambios	Velocidad	Horas desde el commit hasta que el cambio está en producción	≤ 1 día	GitHub, Render
Frecuencia de despliegue	Velocidad	Publicaciones en producción por semana	≥ 1 por sprint	Render
Tasa de fallo de cambios	Calidad	Despliegues que causan fallas ÷ despliegues totales × 100	≤ 15 %	Render, Issues
Tiempo de recuperación de un despliegue fallido	Velocidad	Tiempo hasta restaurar el servicio tras una falla	< 1 hora	Render, UptimeRobot
Porcentaje de retrabajo	Calidad	Horas dedicadas a corregir defectos ÷ horas totales × 100	≤ 10 %	Registro de horas
Productividad	Velocidad	Puntos de función terminados ÷ horas hombre	≥ 0.17 PF por hora (6 h por PF)	Registro de horas

12.5. Métricas de proyecto

Permiten controlar el proyecto frente a la línea base de la propuesta monetaria: 418.46 puntos de función ajustados, 502.15 horas hombre, 3 meses con un desarrollador y un costo de desarrollo de Q21,670.25. El avance se controlará con la técnica de **valor ganado**, usando como unidad los puntos de función terminados.

Tabla 12.3. Métricas de proyecto.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Avance funcional	Eficacia	PF terminados ÷ PF totales (418.46) × 100	100 % al cierre	Matriz de trazabilidad
Cumplimiento de requerimientos	Eficacia	Requerimientos RF y RNF aceptados ÷ requerimientos totales × 100	100 %	Matriz de trazabilidad
Cumplimiento del sprint	Eficacia	Tareas terminadas ÷ tareas comprometidas en el sprint × 100	≥ 90 %	Tablero Scrum
Velocidad del equipo	Velocidad	Puntos de función terminados por sprint	≈ 52 PF por sprint (418.46 ÷ 8)	Tablero Scrum
Gráfico de trabajo pendiente (burndown)	Velocidad	PF pendientes al final de cada día del sprint	Llegar a 0 al cerrar el sprint	Tablero Scrum
Índice de desempeño del cronograma (SPI)	Velocidad	SPI = valor ganado ÷ valor planificado	≥ 0.95	Hoja de control
Índice de desempeño del costo (CPI)	Eficacia	CPI = valor ganado ÷ costo real	≥ 0.95	Hoja de control
Desviación del esfuerzo	Eficacia	(Horas reales − horas estimadas) ÷ horas estimadas × 100	≤ ±10 %	Registro de horas
Volatilidad de requerimientos	Calidad	Requerimientos agregados o modificados ÷ requerimientos totales × 100	≤ 15 %	Product Backlog
Riesgos mitigados	Eficacia	Riesgos con plan de mitigación aplicado ÷ riesgos identificados × 100	100 % de riesgos altos	Matriz de riesgos
Rentabilidad del proyecto	Impacto	Valor actual neto a 5 años al 10 % y relación beneficio/costo	VAN > 0; B/C > 1	Estudio de factibilidad

12.6. Métricas de producto

Evalúan el software que se entregará a la municipalidad. Incluyen las métricas de calidad propuestas por Pressman (corrección, facilidad de mantenimiento, integridad y facilidad de uso) y las características del estándar ISO/IEC 25010.

Tamaño y complejidad

Tabla 12.4. Métricas de tamaño y complejidad.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Puntos de función ajustados	Eficacia	PFA = PFSA × (0.65 + 0.01 × Σ factores de ajuste)	418.46 PFA	Hoja de estimación FPA
Líneas de código	Calidad	Líneas de código fuente sin comentarios ni líneas en blanco, por capa	Registrar por sprint	SonarQube Cloud
Complejidad ciclomática	Calidad	CC = 1 + número de decisiones de un método	Promedio ≤ 5; ningún método > 10	SonarQube Cloud
Complejidad cognitiva	Calidad	Dificultad para entender un método según anidamiento y saltos	≤ 15 por método	SonarQube Cloud
Código duplicado	Calidad	Líneas duplicadas ÷ líneas totales × 100	< 3 %	SonarQube Cloud
Deuda técnica	Calidad	Tiempo para corregir problemas de calidad ÷ tiempo de desarrollo × 100	Calificación A (< 5 %)	SonarQube Cloud

Calidad del producto

Tabla 12.5. Métricas de calidad del producto.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Corrección	Calidad	Defectos encontrados ÷ miles de líneas de código (KLOC)	≤ 2 por KLOC	Issues de GitHub
Cobertura de pruebas	Calidad	Líneas ejecutadas por las pruebas ÷ líneas totales × 100	≥ 80 % en servicios y modelos	Pest con PCOV
Pruebas aprobadas	Eficacia	Pruebas aprobadas ÷ pruebas ejecutadas × 100	100 % en cada publicación	Pest, GitHub Actions
Reglas del enunciado verificadas	Eficacia	Reglas con prueba automatizada ÷ reglas del enunciado × 100	100 %	Pest
Facilidad de mantenimiento (MTTC)	Velocidad	Tiempo medio para analizar, implementar, probar y publicar un cambio	≤ 2 días	Issues de GitHub
Integridad	Calidad	Integridad = Σ [1 − (amenaza × (1 − seguridad))] por tipo de ataque	≥ 0.95	Análisis de riesgos de seguridad
Vulnerabilidades	Calidad	Vulnerabilidades del código y de las dependencias por severidad	0 críticas y 0 altas	SonarQube Cloud, composer audit
Protección de datos	Calidad	Tablas con Row Level Security y campos sensibles cifrados ÷ totales × 100	100 %	Catálogo de PostgreSQL

Rendimiento, confiabilidad y facilidad de uso

Tabla 12.6. Métricas de rendimiento, confiabilidad y usabilidad.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Tiempo de respuesta (percentil 95)	Velocidad	Tiempo bajo el que se atiende el 95 % de las solicitudes	< 2 s con 50 usuarios simultáneos	k6
Capacidad de procesamiento	Velocidad	Solicitudes atendidas por segundo	≥ 20 por segundo	k6
Tiempo de generación de alertas	Velocidad	Tiempo entre la actualización del aforo y la creación de la alerta	Misma transacción (< 1 s)	Pruebas automatizadas
Disponibilidad	Calidad	Tiempo en servicio ÷ horario de servicio (04:30 a 21:00) × 100	≥ 99 %	UptimeRobot
Tiempo medio entre fallas (MTBF)	Calidad	Horas de operación ÷ número de fallas	≥ 720 horas	UptimeRobot
Tiempo medio de recuperación (MTTR)	Velocidad	Tiempo de interrupción ÷ número de fallas	< 1 hora	UptimeRobot, Render
Tasa de errores del servidor	Calidad	Respuestas con error 5xx ÷ solicitudes totales × 100	< 1 %	Registros de Render
Facilidad de uso (SUS)	Impacto	Cuestionario de usabilidad de 10 preguntas, puntaje de 0 a 100	≥ 70 puntos	Cuestionario SUS
Tasa de éxito de tareas	Eficacia	Tareas completadas sin ayuda ÷ tareas intentadas × 100	≥ 90 %	Prueba con operadores
Tiempo por tarea	Velocidad	Tiempo para registrar un pase o actualizar el aforo	< 30 segundos	Prueba con operadores

12.7. Métricas de impacto

Miden el beneficio que el sistema aporta a la operación del Transmetro y a la gestión de la municipalidad. Se obtendrán de los reportes del sistema y se compararán con la situación anterior a su implementación.

Tabla 12.7. Métricas de impacto.

Métrica	Dimensión	Fórmula o forma de medición	Meta	Herramienta
Tiempo de atención de alertas de refuerzo	Impacto	Promedio entre la creación y el cierre de la alerta	< 10 minutos	Reporte de alertas
Horas de saturación por estación	Impacto	Horas al mes con aforo ≥ 150 %	Reducción mensual	Reporte de estaciones
Buses con espera extra	Impacto	Visitas con ocupación < 25 % ÷ visitas totales × 100	Reducción mensual	Reporte de visitas
Cumplimiento de flota por línea	Impacto	Líneas con buses entre n y 2n ÷ líneas totales × 100	100 %	Reporte de líneas
Horas administrativas ahorradas	Impacto	Horas de consolidación manual antes − después	≥ 1,440 horas al año	Encuesta a la municipalidad
Recaudación conciliada	Impacto	Monto conciliado ÷ monto registrado en molinetes × 100	100 %	Reporte de recaudación
Adopción del sistema	Impacto	Operadores que usan el sistema a diario ÷ operadores capacitados × 100	≥ 90 %	Bitácora de auditoría
Valor actual neto (VAN)	Impacto	Σ flujos netos descontados al 10 % − inversión	> 0 (estimado Q181,836 a 5 años)	Estudio de factibilidad
Relación beneficio/costo	Impacto	Beneficios descontados ÷ costos descontados	> 1 (estimado 4.07)	Estudio de factibilidad
Período de recuperación	Impacto	Inversión ÷ flujo neto anual	< 3 años (estimado 0.7 años)	Estudio de factibilidad

12.8. Evaluación de las métricas

La siguiente tabla resume los indicadores clave de cada dimensión, su meta para el proyecto completo y su evaluación con la medición realizada sobre el sistema construido. Los indicadores de operación e impacto se evaluarán una vez que el sistema esté en producción.

Tabla 12.8. Evaluación de los indicadores clave por dimensión.

Dimensión	Indicador	Meta	Resultado medido	Evaluación
Eficacia	Reglas del enunciado verificadas	100 %	100 % (9 de 9 reglas con prueba)	Verde
Eficacia	Pruebas aprobadas	100 %	100 % (240 de 240 y 190 de 190 de extremo a extremo)	Verde
Eficacia	Requerimientos funcionales implementados	100 % al cierre	24 de 24 completos	Verde
Eficacia	Eficiencia de remoción de defectos	≥ 95 %	100 % (17 defectos corregidos antes de la entrega)	Verde
Velocidad	Velocidad del equipo	≈ 52 PF por sprint	≈ 130 PF por sprint en los sprints de desarrollo (4 a 6)	Verde
Velocidad	Ejecución de pruebas automatizadas	< 60 segundos	11.4 segundos (240 pruebas)	Verde
Velocidad	Tiempo de respuesta (p95)	< 2 s	Se medirá con k6 en el sprint 8	Pendiente
Calidad	Corrección (densidad de defectos)	≤ 2 por KLOC	1.85 por KLOC (17 en 9.2 KLOC)	Verde
Calidad	Complejidad ciclomática promedio	≤ 5	2.29 en la lógica de la aplicación	Verde
Calidad	Métodos con complejidad mayor a 10	0	6 de 281 (simulador e importador KML)	Amarillo
Calidad	Tablas con Row Level Security	100 %	100 % (23 de 23)	Verde
Calidad	Cobertura de pruebas	≥ 80 %	Se medirá con PCOV en el sprint 8	Pendiente
Impacto	Relación beneficio/costo	> 1	4.07 (estimado en la factibilidad)	Verde
Impacto	Tiempo de atención de alertas	< 10 minutos	Se medirá en producción	Pendiente
Impacto	Satisfacción de los operadores (SUS)	≥ 70	Se medirá en la capacitación	Pendiente

12.9. Plan de recolección de métricas

Tabla 12.9. Momento, frecuencia y responsable de la medición.

Tipo de métrica	Momento	Frecuencia	Responsable
Proceso	Durante el desarrollo y en cada publicación	En cada cambio y al cerrar el sprint	Scrum Master
Proyecto	Planificación y desarrollo	Diaria (burndown) y en cada revisión de sprint	Product Owner y Scrum Master
Producto: tamaño, complejidad y calidad	Desarrollo y pruebas finales	En cada cambio	Equipo de desarrollo
Producto: rendimiento y confiabilidad	Pruebas finales y producción	Antes de cada versión y de forma continua	Equipo de desarrollo y soporte
Producto: facilidad de uso	Capacitación y prueba piloto	Una vez por versión	Scrum Master
Impacto	Operación	Mensual	Municipalidad

12.10. Resultados esperados

Con la aplicación de estas métricas se espera entregar un sistema **eficaz**, que cumpla el 100 % de los requerimientos y reglas del enunciado con pruebas automatizadas; **rápido**, tanto en la entrega de cambios como en el tiempo de respuesta para los operadores; de **calidad**, con baja complejidad, menos de dos defectos por cada mil líneas de código, sin vulnerabilidades críticas y con una disponibilidad de al menos 99 %; y con un **impacto** positivo y medible para la municipalidad, reflejado en una atención más rápida de las estaciones saturadas, el control de la flota y la recaudación, y una inversión que se recupera en menos de tres años.

13. Implementación de Estándares de Seguridad

La seguridad es uno de los pilares del Transmetro y también del sistema que lo controla. Este capítulo describe los estándares de seguridad que se tomaron como referencia y cómo se implementaron en cada capa de la solución: el navegador de la PC de estación, la aplicación Laravel, la base de datos PostgreSQL y la infraestructura en la nube.

13.1. Estándares de referencia

- **OWASP Top 10 (2021):** lista de los diez riesgos de seguridad más críticos de las aplicaciones web; se usó como guía para revisar cada módulo.
- **OWASP ASVS (nivel 1):** requisitos verificables de autenticación, sesiones, control de acceso, validación de datos y criptografía.
- **ISO/IEC 27001 (Anexo A):** controles de control de acceso, criptografía, registro de eventos, respaldo y gestión de cambios.
- **ISO/IEC 25010:** subcaracterísticas de seguridad del producto: confidencialidad, integridad, no repudio, responsabilidad y autenticidad.

13.2. Controles frente al OWASP Top 10

Tabla 13.1. Riesgos del OWASP Top 10 y controles implementados.

Riesgo OWASP	Control implementado en el sistema	Evidencia
A01 Pérdida de control de acceso	Middleware VerificarRol en cada ruta; el operador solo actúa sobre su estación; Row Level Security en todas las tablas	AutenticacionTest, OperadorTest
A02 Fallas criptográficas	Contraseñas con bcrypt; contacto, dirección e historial educativo cifrados con AES-256; HTTPS y SSL hacia Supabase	Modelos Empleado y Piloto
A03 Inyección	Consultas parametrizadas con Eloquent y PDO; filtros de reportes con parámetros; Blade escapa toda la salida (XSS)	ReportesTest (inyección SQL)
A04 Diseño inseguro	Reglas del enunciado en triggers y restricciones; transacciones con bloqueo de filas para evitar cobros duplicados	ReglasBaseDatosTest, CobroPasajeTest
A05 Configuración incorrecta	Variables de entorno fuera del repositorio (.env); APP_DEBUG desactivado en producción; RLS bloquea la API pública de Supabase	Lista de verificación de publicación
A06 Componentes vulnerables	Versiones con soporte (Laravel 13, PHP 8.4); revisión con composer audit y npm audit antes de publicar	GitHub Actions
A07 Fallas de autenticación	Bloqueo tras 5 intentos fallidos por correo e IP; mensaje genérico; sesión regenerada al entrar e invalidada al salir; sesión que expira al cerrar el navegador	AutenticacionTest, SesionesTest
A08 Integridad del software	Pruebas y revisión de estilo en GitHub Actions antes de cada despliegue; dependencias fijadas con composer.lock y package-lock.json	Flujo de CI/CD
A09 Registro y monitoreo	Bitácora de auditoría con usuario, fecha y datos anteriores y posteriores, consultable por el administrador e inalterable (trg_bitacora_inmutable); monitoreo de disponibilidad con UptimeRobot	BitacoraTest
A10 Falsificación de solicitudes (SSRF)	La aplicación no realiza peticiones a direcciones ingresadas por el usuario	Revisión de código

13.3. Controles por capa

Navegador y comunicación

- Toda la comunicación entre la PC de estación y la aplicación viaja cifrada por HTTPS (TLS).
- Cada formulario incluye un token CSRF que impide que otro sitio envíe operaciones en nombre del usuario.
- Las cookies de sesión son HttpOnly y SameSite, por lo que no pueden leerse desde JavaScript ni enviarse desde otros sitios.

Aplicación

- Autenticación con correo y contraseña cifrada, bloqueo por intentos fallidos y cierre de sesión al cerrar el navegador.
- Autorización por rol en cada ruta y validación de que el operador solo opere su estación.
- Validación de todos los datos recibidos (formatos de placa, DPI, teléfono, montos y estados) antes de guardarlos.
- Mensajes de error en español sin mostrar detalles técnicos al usuario.

Base de datos

- Llaves foráneas, restricciones CHECK y triggers que impiden dejar la información en un estado inválido, aunque se intente modificar fuera de la aplicación.
- Row Level Security activado en las 23 tablas: la API pública de Supabase no puede leer ni modificar datos.
- Solo la aplicación se conecta a la base de datos, con un usuario propio y conexión cifrada.

Infraestructura y operación

- Las credenciales y llaves se guardan como variables de entorno en Render y nunca se suben al repositorio.
- Respaldo diario automático en Supabase y respaldo semanal fuera de la nube, con prueba de restauración mensual (RNF-08).
- Antes de la puesta en producción se cambian las contraseñas de los usuarios de prueba y se desactiva el modo de depuración.

13.4. Verificación de la seguridad

Los controles de seguridad se verifican con pruebas automatizadas que intentan acceder sin sesión, usar funciones de otro rol, superar el límite de intentos de inicio de sesión e inyectar código SQL en los filtros de los reportes. Los resultados se presentan en el capítulo 14. Una vez publicado el sistema se complementará con un análisis estático en SonarQube Cloud y un escaneo con OWASP ZAP.

14. Plan de Pruebas

El plan de pruebas establece qué se va a probar en el Sistema de Control Transmetro, cómo se va a probar, con qué herramientas, en qué entorno y bajo qué criterios se considerará que el sistema está listo para ser entregado a la Municipalidad de Guatemala. Su propósito es comprobar que cada requerimiento funcional y no funcional se cumple, que las reglas del enunciado no pueden ser violadas y que la aplicación responde de forma correcta y segura en la operación diaria de las estaciones.

Las pruebas acompañan todo el ciclo de vida del proyecto. Cada cambio en el código se valida con pruebas automatizadas antes de integrarse, de acuerdo con la definición de terminado del capítulo 11, y al final de cada sprint se ejecuta el conjunto completo. Al final del capítulo se presentan los resultados de la última ejecución completa, realizada el 13 de octubre de 2026.

14.1. Objetivos

Objetivo general: verificar que el Sistema de Control Transmetro cumple los requerimientos funcionales y no funcionales acordados, las reglas de negocio del enunciado y los estándares de calidad definidos, antes de su aceptación por parte de la Municipalidad de Guatemala.

Objetivos específicos:

- Comprobar que cada requerimiento funcional (RF-01 a RF-24) cuenta con al menos un caso de prueba aprobado.
- Demostrar que las reglas del enunciado son rechazadas por la base de datos cuando se intentan violar.
- Verificar que cada usuario únicamente puede usar las funciones de su rol.
- Validar el flujo completo de operación desde el navegador: inicio de sesión, monitoreo, alertas, cobro de pasajes y reportes.
- Medir el rendimiento, la disponibilidad y la compatibilidad del sistema una vez publicado en la nube.
- Registrar, clasificar y dar seguimiento a los defectos encontrados hasta su cierre.

14.2. Alcance

Dentro del alcance: inicio de sesión y roles, mapa de rutas y monitoreo en vivo, catálogos de municipios, líneas y estaciones, accesos y molinetes, flota y parqueos, fallas y mantenimiento, personal, Tarjeta Ciudadana, cobro de pasajes, pantalla del operador, alertas, simulación, horarios y reportes, además de las reglas de la base de datos (restricciones, triggers y procedimiento almacenado).

Fuera del alcance: las funciones de la fase 2 (GPS, control de bahías, roles de piloto y seguridad, modo sin conexión) y los equipos externos (emisión física de tarjetas y molinetes reales), que en este prototipo se representan con la simulación.

14.3. Niveles y tipos de prueba

Tabla 14.1. Niveles y tipos de prueba.

Nivel o tipo	Herramienta	Qué se verifica
Unitarias	Pest (PHP)	Reglas de los modelos: espera de 5 minutos, refuerzo desde el 150 %, tipo de día y horario de servicio
Integración y funcionales	Pest + PostgreSQL	Cada pantalla y acción con su base de datos: formularios, permisos, mensajes, cobros, operador, tarjetas y reportes
Base de datos	Pest + PostgreSQL	Restricciones CHECK, llaves foráneas, triggers y sp_cambiar_parqueo_bus: se intenta violar cada regla
Extremo a extremo (E2E)	Playwright (Chromium)	Recorrido real por la aplicación: clics, menús, formularios, mapa, impresión y errores de JavaScript
Simulación	Pest + transmetro:simular	Avance de buses, subidas, bajadas, cobros, alertas, horarios y tráfico
Seguridad	Pest, Playwright	Acceso sin sesión o con otro rol, bloqueo por intentos, cierre de sesión e inyección SQL
Análisis estático	Laravel Pint, SonarQube Cloud	Estándar de formato, errores, vulnerabilidades, duplicación y deuda técnica
Rendimiento	k6	Tiempo de respuesta con 50 usuarios simultáneos en Render
Disponibilidad	UptimeRobot	Tiempo en línea durante el horario de servicio
Compatibilidad	Chrome, Edge y Firefox	Funciones principales en 1366 × 768
Aceptación (UAT)	Casos de este capítulo	Demostración y ejecución de los casos con el Product Owner

14.4. Entorno de pruebas

Tabla 14.2. Entorno de pruebas.

Componente	Configuración
Aplicación	Laravel 13 sobre PHP 8.4; interfaz con Blade, Tailwind CSS 4 y Leaflet compilada con Vite
Base de datos	PostgreSQL con el esquema separado «pruebas», distinto del que guarda los datos del sistema; cada prueba se ejecuta en una transacción que se revierte al terminar
Integración continua	GitHub Actions ejecuta las pruebas en cada envío de código con un PostgreSQL temporal
Datos de prueba	Red oficial: 8 líneas, 121 estaciones, 173 buses y 545 empleados (170 pilotos, 121 operadores y 254 guardias), tarjetas y horarios oficiales
Pruebas E2E	Servidor local de Laravel y navegador Chromium controlado por Playwright con usuarios de administrador y operador
Ejecución local	probar.bat o php artisan test; vendor/bin/pint --test para el estilo
Entorno publicado	Render y Supabase para rendimiento, disponibilidad y compatibilidad

14.5. Procedimiento

1. Preparar el entorno: cargar los datos de prueba (php artisan db:seed) y limpiar la caché.
2. Verificar el estilo del código con Laravel Pint.
3. Ejecutar las pruebas unitarias, funcionales y de base de datos con Pest.

4. Ejecutar las pruebas de extremo a extremo con Playwright.
5. Registrar como defecto, con su severidad, cualquier prueba fallida.
6. Corregir el defecto, agregar una prueba que lo reproduzca y repetir todo el conjunto (regresión).
7. Registrar los resultados en este documento y en el tablero del sprint.

14.6. Criterios de entrada y de salida

Criterios de entrada: el incremento está integrado en la rama principal y compila; la base de pruebas tiene las migraciones y los datos cargados; los requerimientos están en la matriz de trazabilidad; los usuarios de prueba están disponibles.

Criterios de salida: el 100 % de las pruebas automatizadas se aprueba; no hay defectos críticos ni altos abiertos; cada requerimiento funcional tiene al menos un caso aprobado; el código cumple el estándar de Laravel Pint; las pruebas no funcionales pendientes tienen fecha de ejecución.

14.7. Gestión de defectos

Cada defecto se registra como un issue en GitHub con su descripción, pasos para reproducirlo, resultado esperado, resultado obtenido y severidad. Su ciclo de vida es: abierto, en corrección, corregido, verificado y cerrado. Un defecto solo se cierra cuando una prueba automatizada lo reproduce y, después de la corrección, se aprueba.

Tabla 14.3. Severidad de los defectos.

Severidad	Descripción	Tiempo máximo de corrección
Crítica	Impide usar el sistema, pierde información o permite un acceso no autorizado	Mismo día
Alta	Una función principal no trabaja o viola una regla del enunciado	48 horas
Media	Una función trabaja con errores o muestra un mensaje técnico, pero hay alternativa	Dentro del sprint
Baja	Detalle visual o de redacción que no afecta la operación	Siguiente sprint

14.8. Roles y cronograma de pruebas

Tabla 14.4. Responsables de las pruebas.

Rol	Responsabilidad
Product Owner (municipalidad)	Revisar los criterios de aceptación, participar en las pruebas de aceptación y firmar la carta
Scrum Master (consultora)	Planificar las pruebas de cada sprint, dar seguimiento a los defectos y vigilar los criterios de salida
Equipo de desarrollo (consultora)	Escribir y mantener las pruebas, ejecutar el plan, corregir defectos y documentar resultados

Tabla 14.5. Cronograma de pruebas por sprint.

Sprint	Fechas	Actividad de prueba
4 – 6	14/09 – 10/10	Pruebas unitarias, funcionales y de base de datos escritas junto con cada módulo y ejecutadas en cada cambio; pruebas E2E de la demo funcional
7	12/10 – 17/10	Ejecución completa automatizada y E2E (13/10); plan de pruebas, criterios y carta de aceptación
8	19/10 – 23/10	Publicación, pruebas de carga con k6, monitoreo con UptimeRobot, compatibilidad, SonarQube Cloud, pruebas de aceptación y regresión final

14.9. Casos de prueba

La tabla 14.6 presenta los casos de prueba principales, relacionados con el requerimiento que verifican. Todos están automatizados; el detalle se encuentra en la carpeta tests del repositorio y en el guion de pruebas de extremo a extremo.

Tabla 14.6. Casos de prueba y resultado obtenido.

ID	RF	Caso de prueba	Resultado esperado	Resultado
CP-01	RF-16	Iniciar sesión como administrador	Ingresa y llega al mapa de administración	Aprobado
CP-02	RF-16	Contraseña incorrecta varias veces	Mensaje genérico; tras 5 intentos se bloquea el ingreso	Aprobado
CP-03	RF-16	Un operador abre pantallas del administrador	Acceso denegado (403)	Aprobado
CP-04	RF-16	Abrir pantallas internas sin sesión	Redirige al inicio de sesión	Aprobado
CP-05	RF-01	Consultar municipios y sus estaciones	Cada estación pertenece a un solo municipio	Aprobado
CP-06	RF-02	Consultar el recorrido de la Línea 2	5 estaciones en orden oficial; distancia = suma de tramos	Aprobado
CP-07	RF-02	Consultar San Sebastián	Transbordo entre las Líneas 1 y 2	Aprobado
CP-08	RF-10	Llevar una estación al 150 % de su aforo	Alerta de refuerzo automática; no se repite	Aprobado
CP-09	RF-04	Eliminar un acceso de una estación con 2	La base de datos lo rechaza	Aprobado
CP-10	RF-08	Dejar un acceso sin guardia	Se rechaza con un mensaje claro	Aprobado
CP-11	RF-05	Registrar un bus con placa C387JGL	Se guarda con parqueo y molinetes; placa inválida rechazada	Aprobado
CP-12	RF-11	Asignar más de 2n buses a una línea	trg_buses_max_flota lo rechaza	Aprobado
CP-13	RF-02	Activar una línea sin n a 2n buses	No se activa y se explica el motivo	Aprobado
CP-14	RF-06	Cambiar el parqueo de un bus	Nunca queda sin parqueo; un parqueo ocupado se rechaza	Aprobado
CP-15	RF-07	Registrar, editar y eliminar un empleado	Valida teléfono, DPI y correo; datos cifrados	Aprobado
CP-16	RF-08	Dos operadores en la misma estación	La base de datos lo impide	Aprobado
CP-17	RF-09	El operador abre su estación	Ve aforo, buses detenidos y en camino, accesos y guardias	Aprobado
CP-18	RF-11	El operador pide otro bus	Pedido único; la central envía un bus disponible	Aprobado
CP-19	RF-14	Pase con Tarjeta Ciudadana con viajes gratis	Usa primero los viajes gratis y después el saldo	Aprobado
CP-20	RF-14	Tarjeta sin saldo, bloqueada o inexistente	Pase rechazado; nadie sube	Aprobado

ID	RF	Caso de prueba	Resultado esperado	Resultado
CP-21	RF-14	Pase con tarjeta bancaria	Cobra la tarifa de la línea	Aprobado
CP-22	RF-10	Subir pasajeros a un bus lleno	El molinete no deja subir; alerta de bus lleno	Aprobado
CP-23	RF-12	Bus con menos del 25 % llega a una estación	Espera 5 minutos adicionales	Aprobado
CP-24	RF-13	Ejecutar ciclos de la simulación	Los buses avanzan y guardan cada visita	Aprobado
CP-25	RF-15	El operador atiende alertas	Solo las de su estación; una ya atendida muestra un aviso	Aprobado
CP-26	RF-24	Reportar una falla de prioridad alta	El bus sale de servicio y pasa al taller	Aprobado
CP-27	RF-17	Editar un bus y consultar la bitácora	Muestra quién, cuándo y solo lo que cambió; filtra por módulo, acción, usuario y fecha	Aprobado
CP-28	RF-19	Generar los 7 reportes	Cifras clave, filtros y diseño para PDF	Aprobado
CP-29	RF-19	Verificación de cobros	Cada persona que subió tiene su pago	Aprobado
CP-30	RF-19	Escribir SQL en los filtros	El filtro no permite inyección SQL	Aprobado
CP-31	RF-20	Abrir el mapa de rutas	Las 8 rutas oficiales con color y estaciones	Aprobado
CP-32	RF-21	Consultar la ficha de una tarjeta	Ambos lados con número y QR, saldo y viajes gratis	Aprobado
CP-33	RF-22	Línea fuera de su horario	No salen buses; los que siguen en la calle se retiran	Aprobado
CP-34	RF-23	Hora pico de lunes a viernes	Los tramos tardan un 40 % más	Aprobado
CP-35	RF-18	Cerrar el navegador de la PC de estación	La sesión se cierra	Aprobado

14.10. Resultados de las pruebas

La última ejecución completa del plan se realizó el 13 de octubre de 2026 sobre la versión final del prototipo.

Tabla 14.7. Resumen de resultados.

Nivel	Ejecutadas	Aprobadas	Fallidas	Resultado
Pruebas unitarias, funcionales y de base de datos (Pest)	240	240	0	100 %
Comprobaciones dentro de esas pruebas (aserciones)	1,148	1,148	0	100 %
Pruebas de extremo a extremo (Playwright)	190	190	0	100 %
Errores de JavaScript en el navegador	—	—	0	Sin errores
Estilo del código (Laravel Pint)	—	—	0	Cumple

Tabla 14.8. Pruebas automatizadas por archivo (Pest).

Archivo de prueba	Qué verifica	Pruebas	Aprobadas
ReglasModelosTest	Espera de 5 minutos, refuerzo desde el 150 %, tipo de día y horario	21	21
AutenticacionTest	Inicio y cierre de sesión, bloqueo por intentos y acceso por rol	12	12

Archivo de prueba	Qué verifica	Pruebas	Aprobadas
SesionesTest	Cierre de sesiones abiertas y al cerrar el navegador	2	2
ReglasBaseDatosTest	Accesos, guardias, flota, alerta de aforo, distancia, parqueos y recargas	12	12
LineasOficialesTest	Recorridos, transbordos, flota n a 2n, colores y horarios oficiales	19	19
ImportarKmlTest	Lectura del mapa oficial (KML) de la Municipalidad	7	7
FuncionesTest	Buses, parqueos, estados de líneas y molinetes, personal y tarjetas	23	23
PersonalTest	Formulario de empleados y validación de contactos	4	4
TarjetasTest	Bloqueo, baja, ficha con QR, búsqueda y validación	9	9
CobroPasajeTest	Molinetes del bus, cobros, viajes gratis, saldo y bus lleno	20	20
OperadorTest	Pantalla del operador, pases, bajadas, alertas y fallas	23	23
PantallaPrincipalTest	Mapa principal y panel de cada ruta	6	6
PantallasTest	Carga de todas las pantallas y monitoreo en vivo	17	17
SimulacionTest	Avance de buses, alertas, refuerzos, horarios y tráfico	27	27
ReportesTest	Los 7 reportes, filtros, inyección SQL y verificación de cobros	22	22
BitacoraTest	Bitácora: qué se registra, filtros, datos protegidos y que no se pueda modificar	12	12
ExampleTest (2 archivos)	Pruebas básicas de arranque de la aplicación	4	4
Total		**240**	**240**

Tabla 14.9. Pruebas de extremo a extremo por sección (Playwright).

Sección	Qué se recorrió	Verificaciones	Aprobadas
Acceso	Inicio de sesión, contraseña incorrecta, salida y restricción por rol	6	6
Encabezado	Menús Red, Gestión y Reportes, enlaces y cierre de menús	8	8
Mapa	Las 8 rutas, panel de cada ruta con sus pestañas y estaciones	29	29
Líneas	Listado, ficha, horario oficial y cambio de estado	11	11
Estaciones	Listado, ficha y estado de los molinetes	7	7
Municipios	Listado como catálogo de consulta	3	3
Personal	Filtros, búsqueda, registro, edición y eliminación	12	12
Buses	Flota, filtros, registro, molinetes, parqueo y eliminación	13	13
Tarjetas	Búsqueda, filtros, ficha con QR, bloqueo y baja	14	14
Reportes	Los 7 reportes con filtros, cifras e impresión	30	30
Simulación	Ciclos, inicio, pausa, hora pico y reinicio de aforos	9	9
Operador	Estación, buses, accesos, pasajeros, alertas, pases y fallas	38	38
Bitácora	Consulta, filtros, búsqueda por placa, datos protegidos y acceso solo del administrador	10	10
Total		**190**	**190**

Tabla 14.10. Estado de las pruebas no funcionales.

Prueba	Herramienta	Meta	Estado
Seguridad y roles	Pest, Playwright	Ninguna pantalla accesible sin sesión o con otro rol	Aprobado
Integridad	Pest + PostgreSQL	Cada regla del enunciado rechazada por la base	Aprobado
Estilo del código	Laravel Pint	Sin observaciones	Aprobado
Rendimiento	k6	95 % en menos de 2 s con 50 usuarios	Pendiente (tras publicar)
Disponibilidad	UptimeRobot	99 % o más en horario de servicio	Pendiente (tras publicar)
Compatibilidad	Chrome, Edge, Firefox	Funciones principales en 1366 × 768	Parcial (Chromium)
Calidad del código	SonarQube Cloud	0 críticos, menos de 3 % de duplicación	Pendiente (sprint 8)
Usabilidad	Cuestionario SUS	70 puntos o más	Pendiente (sprint 8)

Tabla 14.11. Defectos encontrados y corregidos.

ID	Severidad	Defecto	Corrección	Estado
D-01	Alta	Los buses seguían circulando después del cierre de la línea	Se retiran al terminar su vuelta y, a más tardar, 15 minutos después del cierre	Cerrado
D-02	Alta	La carga de datos se detenía por falta de memoria (128 MB)	La carga ajusta su límite de memoria antes de iniciar	Cerrado
D-03	Alta	Error interno en el formulario de un bus al agregar mantenimiento	Se corrigió la vista y se agregó su prueba	Cerrado
D-04	Media	Error técnico (422) al atender una alerta ya atendida	Aviso «Esa alerta ya fue atendida»	Cerrado
D-05	Media	Error de JavaScript al resaltar la estación del operador	La estación se marca después de ubicar el mapa	Cerrado
D-06	Media	Pruebas de horarios fallaban al cambiar la fecha	Fechas relativas al día de ejecución	Cerrado
D-07	Baja	Las tablas de los reportes se cortaban al imprimir	Diseño de impresión con ajuste de texto y numeración	Cerrado
D-08	Baja	Texto blanco ilegible sobre el amarillo de la Línea 6	Color de texto según el color oficial de cada línea	Cerrado

14.11. Conclusión de los resultados

El prototipo aprobó el 100 % de sus pruebas automatizadas: 240 pruebas funcionales y de base de datos con 1,148 comprobaciones y 190 verificaciones de extremo a extremo, sin errores en el navegador. Cada requerimiento funcional cuenta con al menos un caso de prueba aprobado y no existen defectos abiertos. Las pruebas no funcionales que dependen de la publicación en la nube y de la participación de los operadores se ejecutarán según el cronograma antes de la firma de la carta de aceptación.

15. Criterios de Aceptación

Los criterios de aceptación son las condiciones verificables que el Sistema de Control Transmetro debe cumplir para que la Municipalidad de Guatemala lo reciba como terminado. Se redactaron a partir de los requerimientos de este documento y de los puntos del enunciado, de modo que cada uno pueda comprobarse con una demostración o con un caso del plan de pruebas.

15.1. Criterios generales

- El sistema cumple los requerimientos funcionales del alcance acordado (RF-01 a RF-24).
- Las reglas del enunciado se aplican en la base de datos y no pueden violarse desde la aplicación.
- El 100 % de las pruebas automatizadas se aprueba y no hay defectos críticos ni altos abiertos.
- Cada usuario accede únicamente a las funciones de su rol.
- Los mensajes de error se muestran en español y explican qué hacer.
- Se entregan el código fuente, la base de datos con sus scripts, la documentación, el manual de usuario y este documento.

15.2. Criterios funcionales

Tabla 15.1. Criterios de aceptación por requerimiento funcional.

RF	Criterio de aceptación	Verificación	Estado
RF-01	Cada estación y línea pertenece a una sola municipalidad y se consulta con ella	CP-05	Cumple
RF-02	Cada línea muestra su recorrido oficial, distancia total calculada, horario y accesos	CP-06, CP-07, CP-13	Cumple
RF-03	Cada estación muestra municipio, dirección, aforo, ocupación, líneas y ubicación	E2E Estaciones	Cumple
RF-04	Mínimo 2 accesos por estación, un molinete por acceso y al menos un guardia	CP-09, CP-10	Cumple
RF-05	Se registran, editan y retiran buses; un bus en una sola línea; sin piloto no sale a ruta	CP-11	Cumple
RF-06	Todo bus tiene parqueo; el cambio nunca lo deja sin parqueo	CP-14	Cumple
RF-07	El personal se registra con sus datos comunes y de su puesto, validados y cifrados	CP-15	Cumple
RF-08	Un operador por estación; ningún acceso queda sin guardia	CP-10, CP-16	Cumple
RF-09	El operador ve y opera solo su estación: buses, accesos, alertas, pases y fallas	CP-17, CP-18	Cumple
RF-10	Alerta automática al 150 % del aforo; un bus lleno no deja subir más pasajeros	CP-08, CP-22	Cumple
RF-11	Ninguna línea supera 2n buses; las alertas envían un bus disponible	CP-12, CP-18	Cumple
RF-12	Un bus con menos del 25 % espera 5 minutos más en cada estación	CP-23	Cumple
RF-13	Cada viaje registra bus, línea, horas y visitas con su ocupación	CP-24	Cumple

RF	Criterio de aceptación	Verificación	Estado
RF-14	Se cobra con Tarjeta Ciudadana o bancaria; se rechazan tarjetas bloqueadas o sin saldo	CP-19 a CP-21	Cumple
RF-15	Las alertas registran tipo, estación, bus y fechas; el operador atiende solo las suyas	CP-25	Cumple
RF-16	Las pantallas exigen sesión y rol; bloqueo tras 5 intentos	CP-01 a CP-04	Cumple
RF-17	Las operaciones quedan en la bitácora y el administrador puede consultarla y filtrarla	CP-27	Cumple
RF-18	Las PC de estación usan HTTPS y nunca se conectan directo a la base	CP-35; al publicar	Pendiente
RF-19	Reportes de estaciones, líneas y buses asignados, más los de operación, con filtros y PDF	CP-28 a CP-30	Cumple
RF-20	El mapa muestra las 8 líneas oficiales y la posición de los buses en ruta	CP-31	Cumple
RF-21	La tarjeta muestra saldo, viajes gratis e historial; se bloquea o se da de baja	CP-19, CP-32	Cumple
RF-22	Cada línea tiene su horario oficial y fuera de él no circulan buses	CP-33	Cumple
RF-23	La simulación mueve buses, cobra pasajes y genera alertas; solo el administrador la controla	CP-24, CP-34	Cumple
RF-24	Una falla alta saca el bus de servicio y al resolverla vuelve a estar disponible	CP-26	Cumple

15.3. Criterios no funcionales

Tabla 15.2. Criterios de aceptación no funcionales.

RNF	Criterio medible	Estado
RNF-01 Seguridad	Todas las rutas exigen sesión y rol; contraseñas cifradas; CSRF; bloqueo por intentos	Cumple
RNF-02 Integridad	Cada regla del enunciado tiene una prueba que intenta violarla	Cumple
RNF-03 Disponibilidad	99 % o más en el horario de servicio	Pendiente
RNF-04 Rendimiento	95 % de operaciones en menos de 2 s con 50 usuarios	Pendiente
RNF-05 Escalabilidad	Nuevas líneas, estaciones o municipios sin cambiar código	Cumple
RNF-06 Usabilidad	Operador capacitado sin conocimientos técnicos; SUS de 70 o más	Pendiente
RNF-07 Auditoría	Operaciones importantes con usuario, fecha y datos anteriores y nuevos; bitácora inalterable	Cumple
RNF-08 Respaldo	Respaldo diario y semanal; restauración mensual probada	Pendiente
RNF-09 Mantenibilidad	Pint y 240 pruebas en cada cambio; complejidad promedio menor a 5	Cumple
RNF-10 Compatibilidad	Chrome, Edge y Firefox en 1366 × 768	Parcial
RNF-11 Conectividad	HTTPS desde cada estación; sin operaciones a medias	Pendiente
RNF-12 Datos personales	Contacto, dirección e historial cifrados; solo el administrador los ve	Cumple
RNF-13 Portabilidad	Entorno nuevo en menos de 30 minutos sin cambiar código	Cumple

Los criterios pendientes dependen de la publicación en la nube o de la participación de los operadores y se verificarán según el cronograma de la tabla 14.5; los parciales se completarán antes de la entrega final (sprint 8).

15.4. Procedimiento de aceptación

1. La consultora entrega el sistema publicado, la documentación y los resultados de las pruebas.
2. Se realiza una demostración con el Product Owner, ejecutando los casos de la tabla 14.6 con usuarios de administrador y de operador.
3. El Product Owner revisa cada criterio de las tablas 15.1 y 15.2 y registra sus observaciones.
4. La consultora corrige las observaciones en los plazos de la tabla 14.3 y repite las pruebas de regresión.
5. Cumplidas las condiciones de aceptación, el representante de la municipalidad firma la carta de aceptación.

15.5. Condiciones para la aceptación

- Todos los criterios funcionales de la tabla 15.1 se encuentran en estado «Cumple».
- No existen defectos críticos ni altos abiertos.
- Los criterios no funcionales pendientes cuentan con su medición o con un compromiso de fecha aceptado por la municipalidad.
- Se cambiaron las contraseñas de los usuarios de prueba antes de la puesta en producción.

16. Carta de Aceptación del Proyecto

Guatemala, _____ de _____ de 20____

Señora
Wendy Celeste Nathaly Estrada González
Consultora en Desarrollo de Software y Aseguramiento de la Calidad
Presente

Asunto: Aceptación del proyecto «Sistema de Control Transmetro»

Estimada consultora:

Por este medio, la Municipalidad de Guatemala hace constar que ha recibido a entera satisfacción el proyecto **Sistema de Control Transmetro**, desarrollado e implementado conforme a la propuesta técnica y económica aprobada y a los requerimientos funcionales y no funcionales establecidos en el documento del proyecto.

Se recibieron los siguientes entregables:

- Aplicación web en operación en la nube, con acceso desde la PC de trabajo de cada estación.
- Base de datos con sus reglas de negocio, scripts de creación y consultas de los reportes.
- Código fuente en el repositorio del proyecto y documentación técnica.
- Manual de usuario y capacitación del personal administrador y de los operadores de estación.
- Plan de pruebas con sus resultados y la evidencia del cumplimiento de los criterios de aceptación.

Luego de la revisión de la documentación, la ejecución de las pruebas de aceptación y la demostración del sistema, se verificó que este cumple con los criterios de aceptación acordados para la administración de la red, el control de la flota y del personal, el monitoreo del aforo con alertas automáticas, el cobro de pasajes con Tarjeta Ciudadana y tarjeta bancaria, los reportes y la seguridad de acceso por rol.

En virtud de lo anterior, la Municipalidad de Guatemala **acepta formalmente** el proyecto y autoriza el pago correspondiente al hito de aceptación, de acuerdo con la forma de pago establecida en la propuesta. Las observaciones menores pendientes, si las hubiera, se detallan a continuación y serán atendidas en los plazos acordados:

_____

_____

Sin otro particular, nos suscribimos atentamente.

Por la Municipalidad de Guatemala	**Recibí conforme**
_____	_____
Firma	Firma
Nombre: _____	Wendy Celeste Nathaly Estrada González
Cargo: _____	Consultora en Desarrollo de Software
(Sello)	y Aseguramiento de la Calidad

17. Conclusiones y Recomendaciones

17.1. Conclusiones

- El Sistema de Control Transmetro centraliza la información de las líneas, estaciones, accesos, buses, parqueos, personal y pagos que hoy se maneja de forma aislada, y responde a los 18 puntos del enunciado, como lo demuestra la matriz de trazabilidad del capítulo 5.
- Aplicar las reglas del enunciado directamente en la base de datos, mediante 17 triggers, 55 restricciones CHECK y un procedimiento almacenado, garantiza que ninguna operación deje la información en un estado inválido, aunque se intente modificar fuera de la aplicación.
- La alerta automática al 150 % del aforo, la alerta de bus lleno y el envío de buses de refuerzo permiten actuar a tiempo ante la saturación de las estaciones, que era una de las principales deficiencias de los controles actuales.
- La conexión de cada PC de estación por Internet mediante HTTPS a una aplicación en la nube resuelve el punto 17 del enunciado sin requerir una red privada entre estaciones, con un costo bajo y escalable.
- El enfoque de aseguramiento de la calidad, con 240 pruebas automatizadas, 190 verificaciones de extremo a extremo, métricas de software e integración continua, permitió detectar y corregir defectos antes de la entrega y respalda la aceptación del sistema con evidencia objetiva.
- La inversión es factible: la propuesta tiene una relación beneficio/costo de 4.07 y se recupera en menos de un año (8 meses).

17.2. Recomendaciones

- Iniciar la implementación con el piloto en las Líneas 1 y 2 y ampliar a las demás líneas después de evaluar sus resultados.
- Conectar los validadores y contadores reales de los molinetes a la aplicación para reemplazar la simulación con datos de la operación.
- Contratar los planes de pago de Render y Supabase para asegurar la disponibilidad y los respaldos diarios.
- Capacitar a los operadores antes de la puesta en marcha y medir su satisfacción con el cuestionario SUS.
- Evaluar en una fase 2 la ubicación de los buses por GPS, el control de bahías, los roles de piloto y seguridad y un modo de operación sin conexión.
- Cambiar las contraseñas de prueba, desactivar el modo de depuración y ejecutar el plan de pruebas completo antes de cada publicación.

18. Repositorio del Proyecto

Todo el código fuente, los scripts de la base de datos, las pruebas automatizadas y la documentación del proyecto se almacenan en un repositorio de GitHub, con control de versiones e integración continua.

Tabla 18.1. Enlaces del proyecto.

Recurso	Enlace o ubicación
Repositorio del código fuente	https://github.com/WendyEstrada04/Transmetro_PGACS
Pruebas automatizadas (GitHub Actions)	https://github.com/WendyEstrada04/Transmetro_PGACS/actions
Script de la base de datos	Script_Creacion_Estructuras_BD_PostgreSQL.sql (sección 9.1)
Consultas de los reportes	docs/Consultas_Reportes.sql (sección 9.2)
Aplicación web publicada (Render)	https://transmetro-fm62.onrender.com
Usuarios de demostración	Administrador: admin@transmetro.test / admin123 · Operador: operador@transmetro.test / operador123
Presentación ejecutiva y video	_____

Estructura principal del repositorio:

- **app/:** modelos, controladores y servicios (CobroPasaje, SimuladorOperacion, Reportes, ImportadorKml).
- **database/migrations y database/seeders:** estructura de la base de datos, reglas de negocio y datos oficiales de la red.
- **resources/views:** pantallas de la aplicación.
- **tests/:** pruebas automatizadas con Pest.
- **docs/:** consultas SQL de los reportes y documentación del proyecto.
- **bat/:** archivos para iniciar el sistema y ejecutar las pruebas en Windows.